

Running Platformer Learning Guide Document

Learning Goals

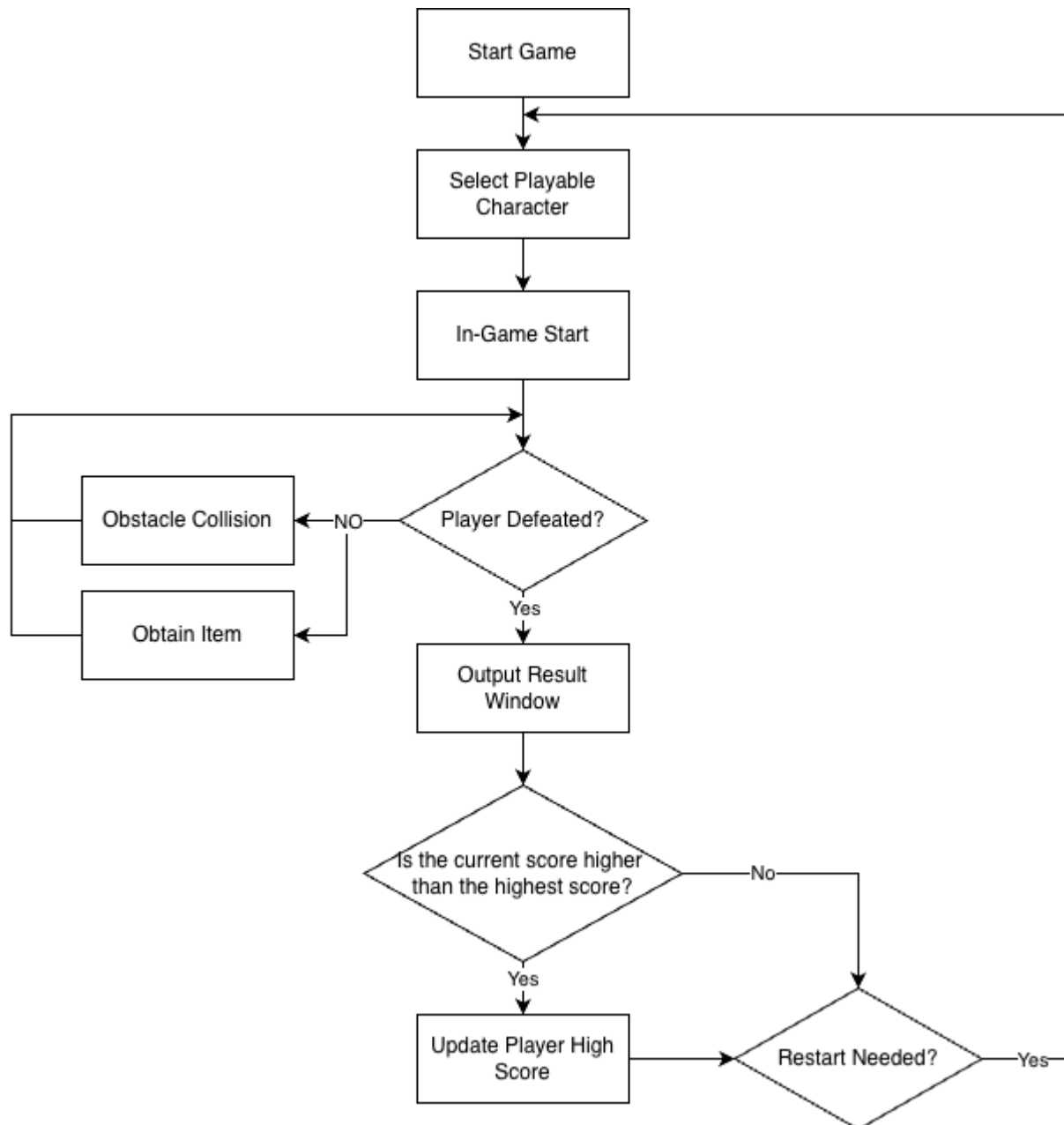
- Learn how to implement core running platformer game functions using the guide document and the sample world.
 - Learn how to set the map up based on the game that is to be created in MapleStory Worlds.
 - Understand Logic and use it to design the game's structure.
 - Modify and utilize the world based on the lessons.
-

Learning Core Functions

- **Physics Component:** Learn about how to use the default Body component (Rigid, Kinematic, Sideview) as well as the PhysicsBody and PhysicsCollider components, which have the physics engine applied to them.
- **Procedural Map Creation:** Learn how to naturally link tiles beyond the random generation of platforms that the character can run on.
- **Object Pool:** Learn about the principles of the object pool techniques in order to manage multiple platform tiles and items.



Core Gameplay Loop



1. Game Start: Start the game from the title screen or view the controls for the game.

****2. Select Play Character:** When the Start Game button is pressed on the title screen, the UI for selecting a character is displayed.

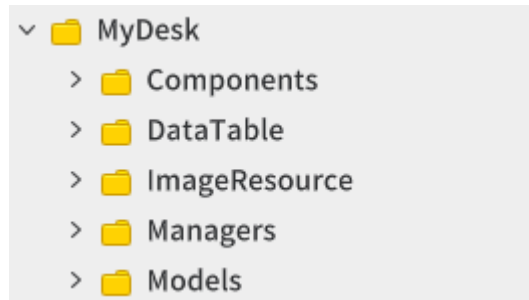
3. In-Game Start: Proceed with a countdown and start the game proper. A scroll speed will apply for each character, making the tiles and background elements move in the opposite direction of the character, and you can now interact with items and obstacles.

4. Character HP Status Check: Check if the HP reached 0 as it continually decreases over time in accordance with the game end condition.

5. Results Check: Displays the current score and the previous high score on the results window, and if the current score is higher than the high score, the user's high score is now updated.

6. Restart Request Check: Return to the first title screen upon requesting a return to the title screen from the results window.

Folder Composition



Name	Description
Components	The folder containing components added to entities to actualize the world.
DataTable	The folder containing the DataSet needed to compose the sample world.
ImageResource	The folder containing the ImageResource needed for game creation. It currently only holds the Image Resource that is used to create the floor tile and obstacles.
Managers	The folder containing scripts created with the MapleStory Worlds Logic function.
Models	The folder containing the entities that have gone through Modelization.

Implementation Order

- 💡 **Basic Training:** The training material for MapleStory Worlds's basic functions and the fundamental functions of each target genre.
- 📖 **Advanced Training:** Category of training for learning and implementing functions visible in the sample world. Using these materials can increase the functions for the genre and understanding of the training materials.

[💡 Basic Training] 1. Composing a World Environment

- Learn about the map types provided by MapleStory Worlds and their relationships with each Body component.
- Learn what kind of game creation environment is needed when making a running platformer game.
- Learn about the PhysicsRigidbody and PhysicsCollider components.
- Apply them to an actual world and learn which settings are required.

[💡 Basic Training] 2. Understanding Logic

- Learn about what role Logic serves.
- Understand the concept of the Manager and how to use it.
- Understand the compositional reasoning and objectives of GameLogic, DataSetLogic, and UIManagerLogic.

[💡 Basic Training]3. Implementing the Scroll System

- Learn about how the character and environmental elements move when making a running platformer game.
- Learn about how to control the background elements that are visible in the world.
- Learn how to change the appearance of the environmental element based on the passage of time after the game starts.

[💡 Basic Training]4. Manage the Map Through Procedural Creation Methods

- Learn about the procedural creation method in a running platformer and why it's needed.
- Create floor tiles that will be used in the game and learn how to compose it as a Model.
- Create a DataSet so that the floor tile can be created through procedural generation based on the data table.
- Implement an infinite scrolling environment by creating floor tiles utilizing procedural creation methods.

[💡 Basic Training] 5. Creating an Object Pool

- Understand the concept of an object pool and what its objective is.
- Implement the object pool Logic.
- See for yourself how the object pool functions by creating and returning one.
- Learn how to create new tiles when a floor tile is destroyed.

[💡 Basic Training]6. Implementing the Player Character

- Learn how to allow the character to jump and slide in a running platformer game.
- Learn how to compose and create a DataSet needed to create various characters.
- Learn how to revert the character's state and information when the game is restarted.
- Learn how to implement skills for each character that activates when the player makes the skill key input.
- Implement the function that can provide the player various characters to choose from and how characters can be selected.

[💡 Basic Training]7. Implementing the Life System

- Learn how to create a function that decreases the character's HP continuously over time once the main game begins.
- Learn how to create a UI element that the user can view as the HP continuously decreases over time.

[💡 Basic Training] 8. Composing UI

- Learn what a UIGroup is and how to use and edit it.
- Create UI elements used in the title screen and learn how to edit the UI.
- Learn how to control the UI elements in the title screen through code using Manager Logic.
- Create UI elements that should be marked in-game and implement it so it's synchronized in real time.
- Implement a UI that displays the results window that shows each score when the game ends.

[📖 Advanced Training] 9. Implementing Obstacle Objects

- Learn how to place obstacle elements on floor tiles that are continuously moving.
- Learn about how to set it so that collisions between characters and obstacles can be detected.
- Implement the function that decreases the character's HP when the character collides with an obstacle.

[📖 Advanced Training] 10. Implementing Items

- Learn how to place item elements on floor tiles that are continuously moving.
- Learn how to compose a DataSet to compose an item with various effects.
- Learn how to set effects and unique effects based on the saved item data table.
- Implement how various item effects are activated and the actual action that's triggered when the character and the item collide.

[📖 Advanced Training] 11. Implementing a Data Save System

- Learn how the high score is updated when the current score is higher than the previous high score when the player character dies.
- Create the system so that the previous high score is loaded and visible in the title screen if there is one when the player launches the world.

[💡 Basic Training] Implementing a World Environment

Video Timeline

- You can follow along by using the timeline under the training video.
- Implementing a World Environment: 00:00:01

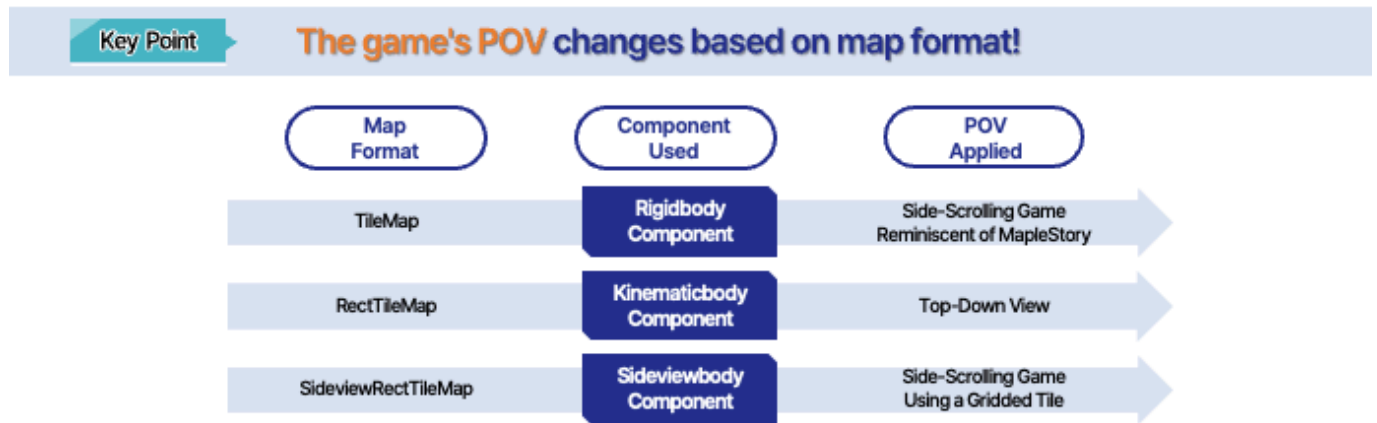
Learning Goals

- Understand the traits of the various TileMap modes that are provided by MapleStory Worlds and the correlation of Body components appropriate for each map.
- Analyze the traits of the running platformer genre and compose the optimized game creation environment yourself.
- Learn the core functions and properties of the PhysicsRigidbody and PhysicsCollider components to implement the character movement and collision detection.
- Apply the learned physics system to actual entities and practice adjusting the physics setting values that fit the game design through test play.

Practical Exercises to Do After Watching the Video

- Implement a map environment that fits a running platformer.
- Register components needed for the interaction between character and floor tiles.
- Test if the created tiles can be stepped on in order to allow for movement.

MapleStory Worlds Map Mode



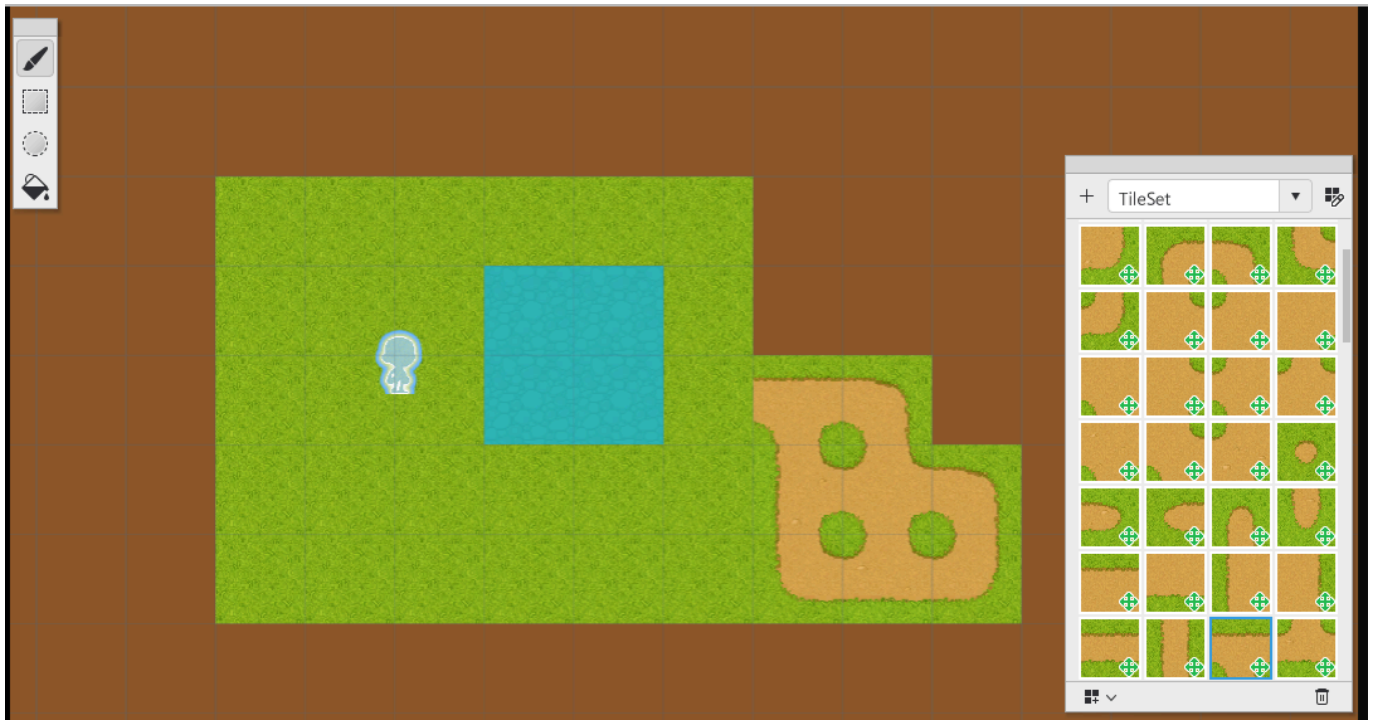
TileMap



- A map format that is set up as the default when creating a world for the first time in MapleStory Worlds.
- A world that is best suited for the feel of MapleStory.
- Formulate a map by utilizing Tiles provided by MapleStory Worlds to create platforms and place entities.

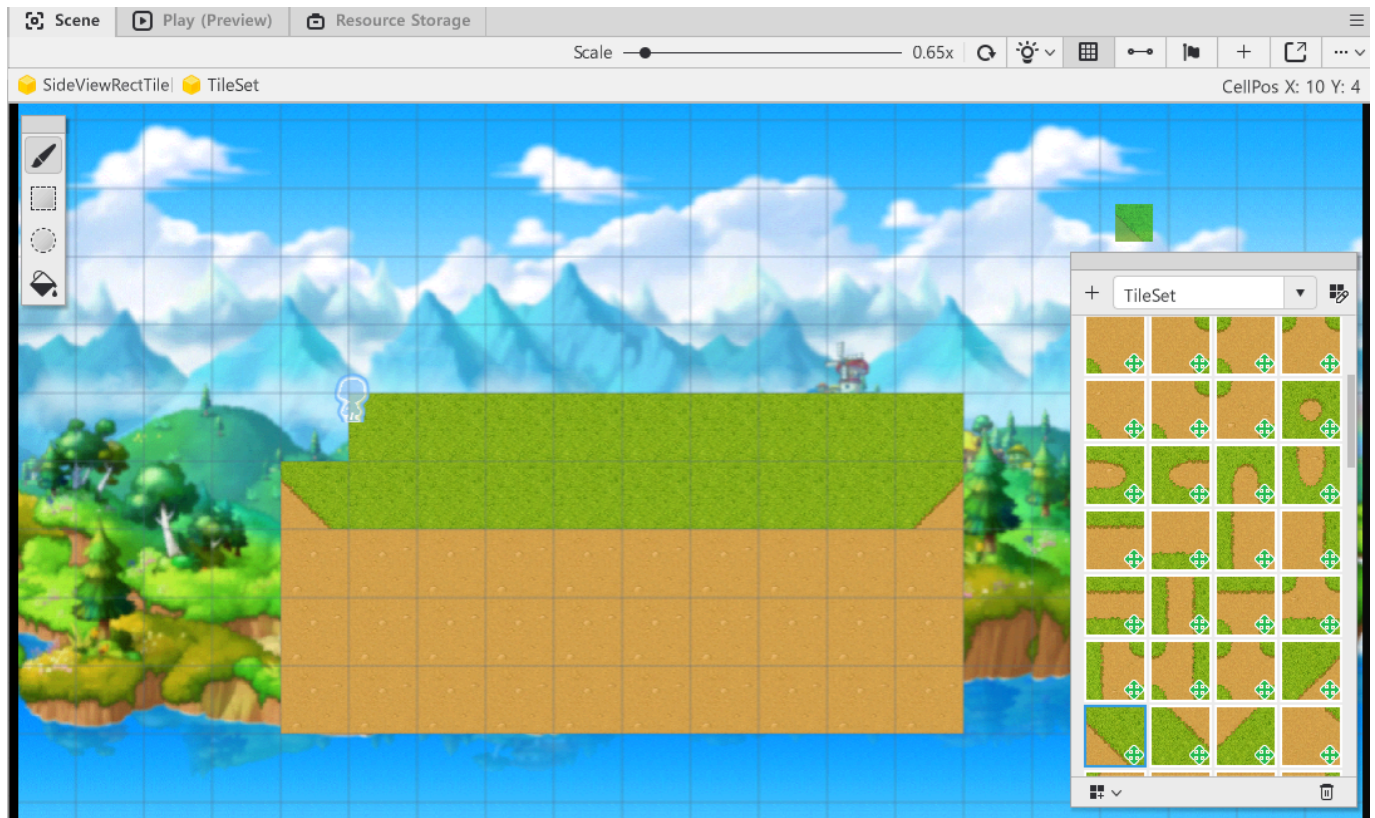
- Create a ramp based on how the tiles are placed.
- A useful map format when you want to make a side scrolling-type game reminiscent of MapleStory.

RectTileMap



- A map format that is effective in making top-down view type games.
- Create your own tile sets or use the tile set resources provided by MapleStory Worlds to formulate a map.
- You can formulate a map by placing tiles in a gridded format.
- You can create mobile and immobile areas through each tile.
- Create a game where you can move in 4 or 8 directions, unlike with the TileMap and SideViewRectTileMap methods.

SideviewRectTileMap



- A map format that is effective in making side scrolling-type games.
- Create your own tile sets or use the tile set resources provided by MapleStory Worlds to formulate a map.
- You can formulate a map by placing tiles in a gridded format.
- The character moves **through** tiles if the tile set doesn't prevent it, unlike with RectTileMap.
- So if you formulate a map with SideViewRectTileMap, the tile set that prevents movement must be used to create **platforms**.

BodyComponent

- The BodyComponent is required to interact with tiles that are platforms in MapleStory Worlds.
- Different BodyComponents are needed based on the map type.
- There are 3 main types of BodyComponents.

RigidbodyComponent

▼

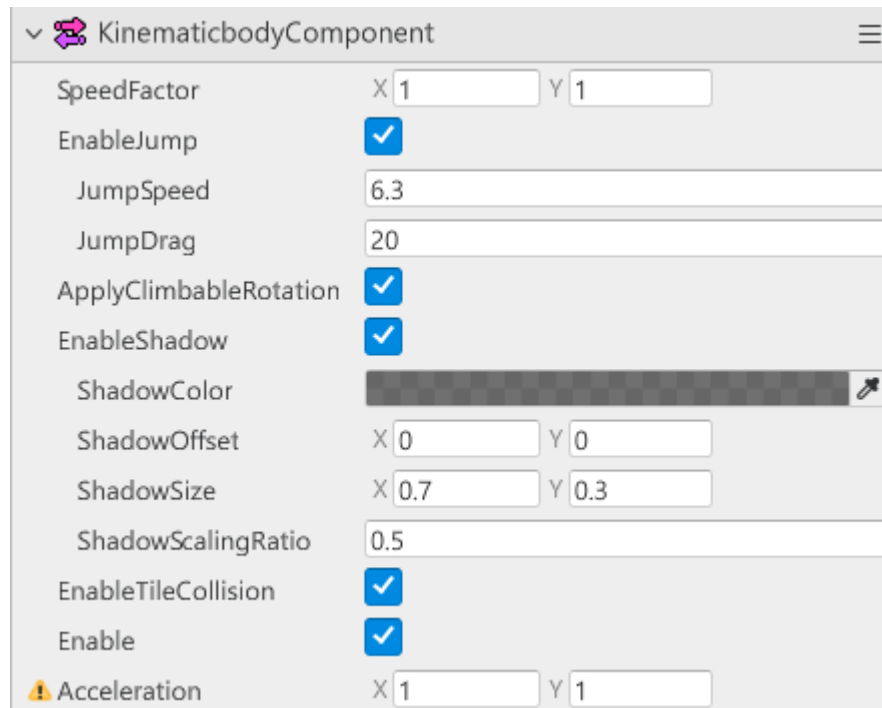
RigidbodyComponent

☰

Mass	100
Gravity	1
JumpBias	0.03
WalkJump	1.23
DownJumpSpeed	2
WalkSpeed	1.4
WalkAcceleration	1
WalkDrag	1
WalkSlant	0.9
AirAccelerationX	1
AirDecelerationX	1
FallSpeedMaxX	1
FallSpeedMaxY	1
LayerSettingType	All ▼
IsBlockVerticalLine	☐
IgnoreMoveBoundary	☐
ApplyClimbableRotation	☒
IsolatedMove	☐
KinematicMove	☐
Enable	☒

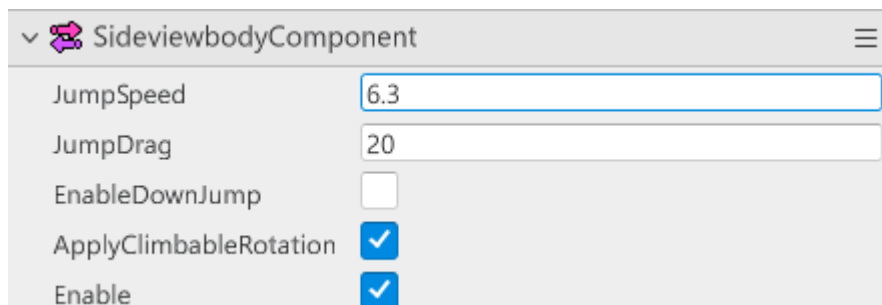
- Controls the character's physics with MapleStory Worlds's TileMap method.
- If the map is a TileMap but doesn't have a RigidBodyComponent, the character and tiles will not interact.
- Provides various physics property values that control the jumps, gravity, and more for characters within RigidBodyComponent.
- The target values can be changed to change the physics methods.

KinematicComponent



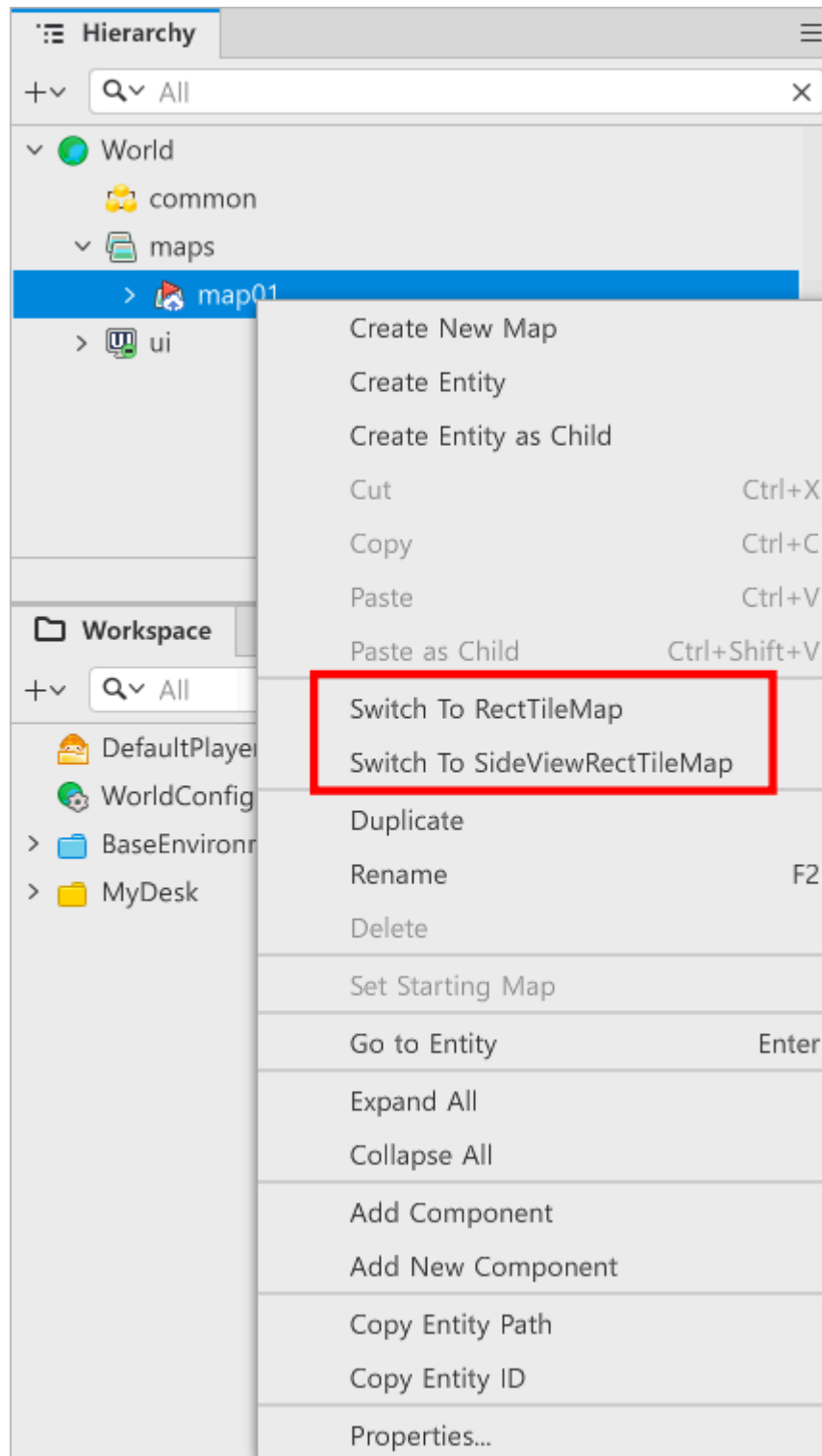
- Controls the character's physics for the MapleStory Worlds's RectTileMap type.
- If the map is RectTileMap but there is no KinematicBodyComponent, no interactions will happen with the RectTileMap tiles.
- You can adjust the speed, jump availability, shadow status, etc. for characters within KinematicBodyComponent.

SideviewbodyComponent



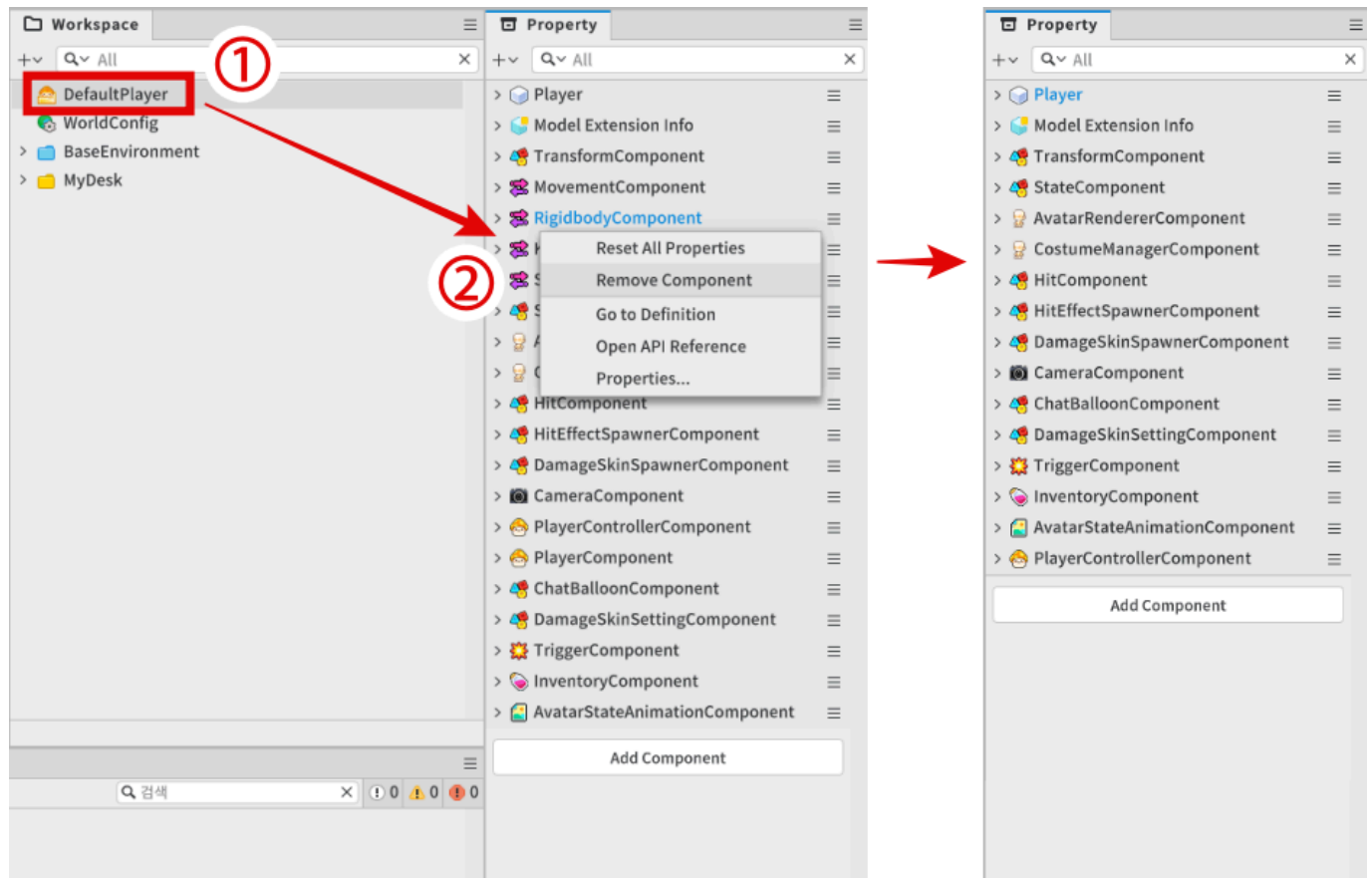
- Controls the character's physics if MapleStory Worlds is using the SideViewRectTileMap method.
- If this component doesn't exist, the character won't be able to interact with the map's tiles.
- Controls the character's jump speed and gravitational acceleration.

Changing Map Formats



- Right-click the map you want to change from the Hierarchy window.
- Click Switch To RectTileMap or Switch To SideViewRectTileMap.
 - If the current map isn't a TileMap, Switch To TileMap will be visible and the conversion to the current map state will be hidden.
- Click on Switch To SideViewRectTileMap to convert the map to SideViewRectTileMap to create a running platformer game.

Removing the DefaultPlayer's Existing Component

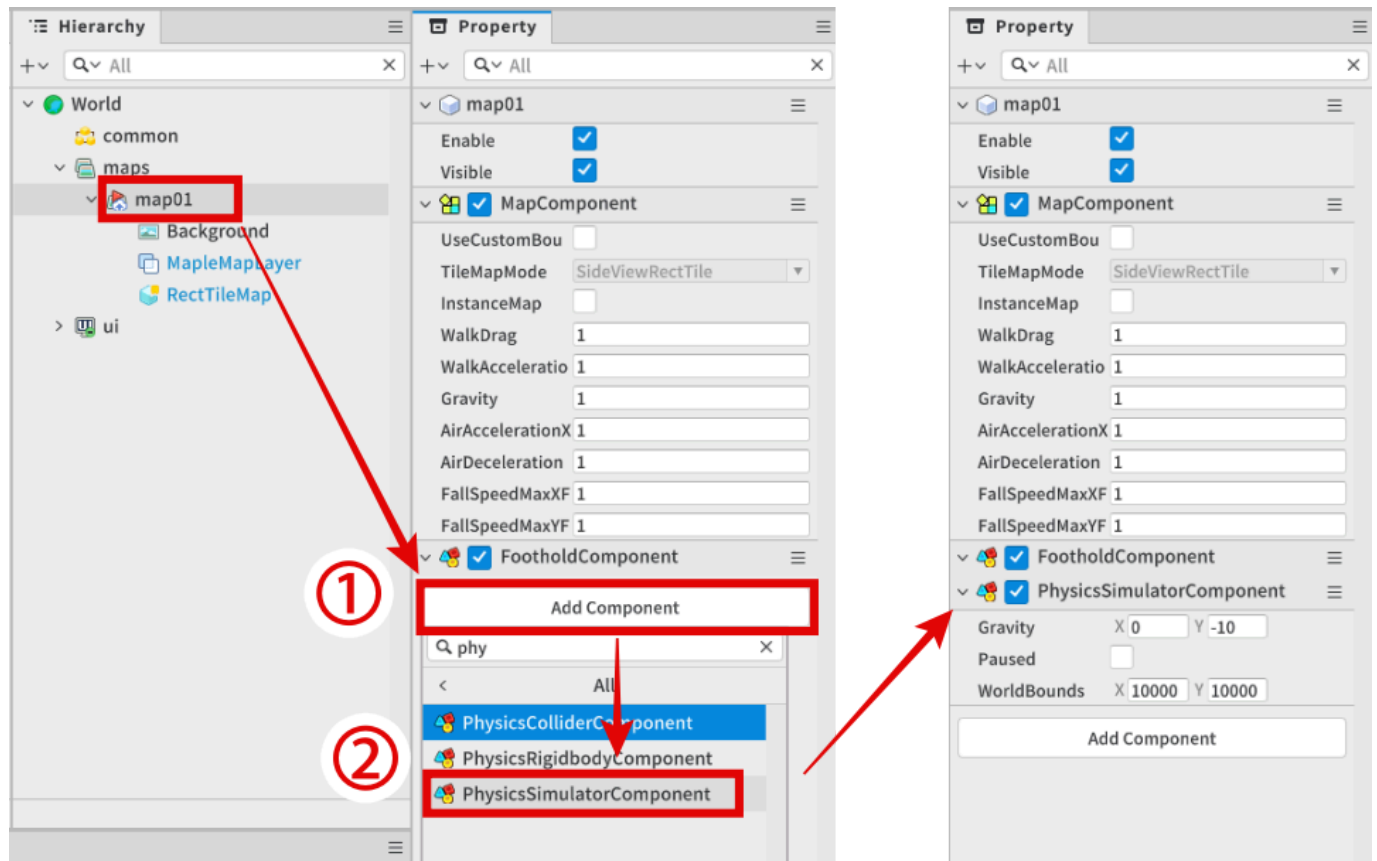


1. Select the DefaultPlayer(Model) from WorkSpace that will be the template for the character that the player will control.
2. To prevent interference of the physics and interaction function from the Property tab, remove the target component using Remove Component. (Rigidbody, Kinematicbody, Sideviewbody, Movement, Player)

Apply a Physics System Utilizing the Physics Component

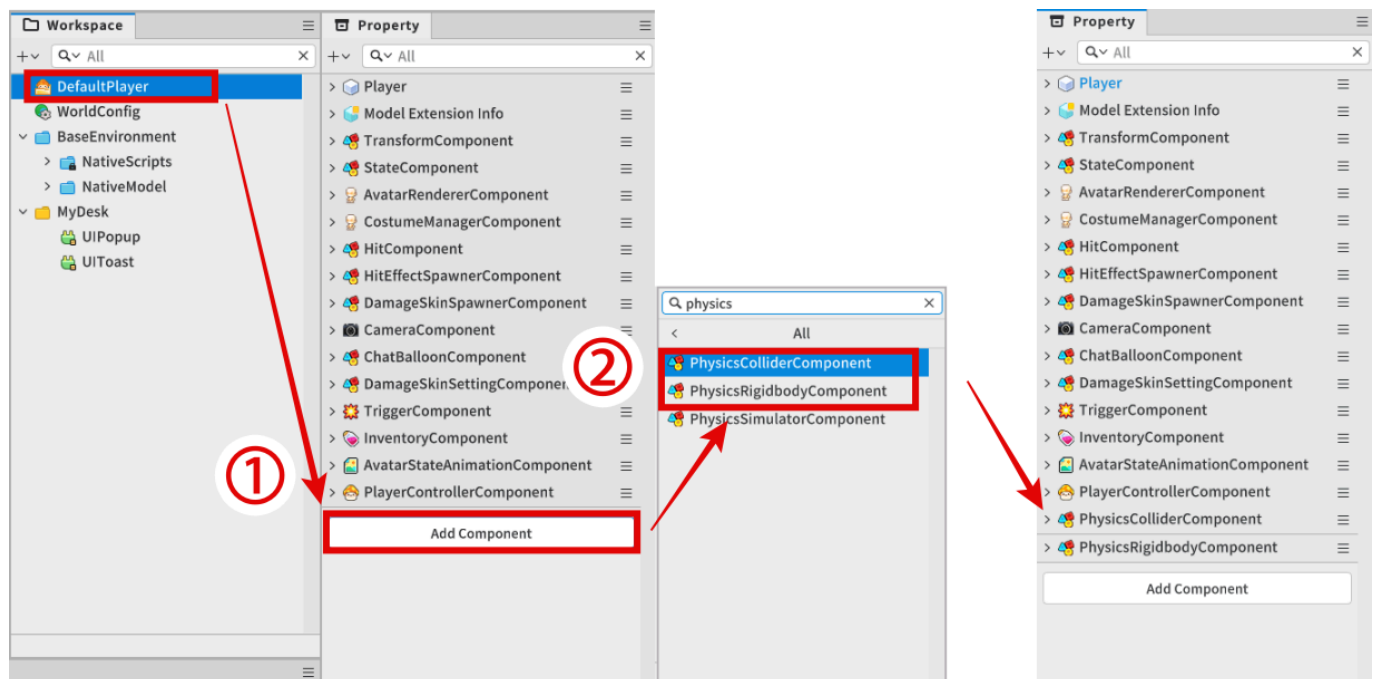
- With how the running platform game works, floor tiles need to be continuously created and destroyed. So in order to make management smooth and to apply independent functions, the floor tile itself will be implemented as a whole entity.
- A separate Physics setting is needed, since it's not terrain created as a TileSet.

Registering PhysicsSimulator to Map Entity



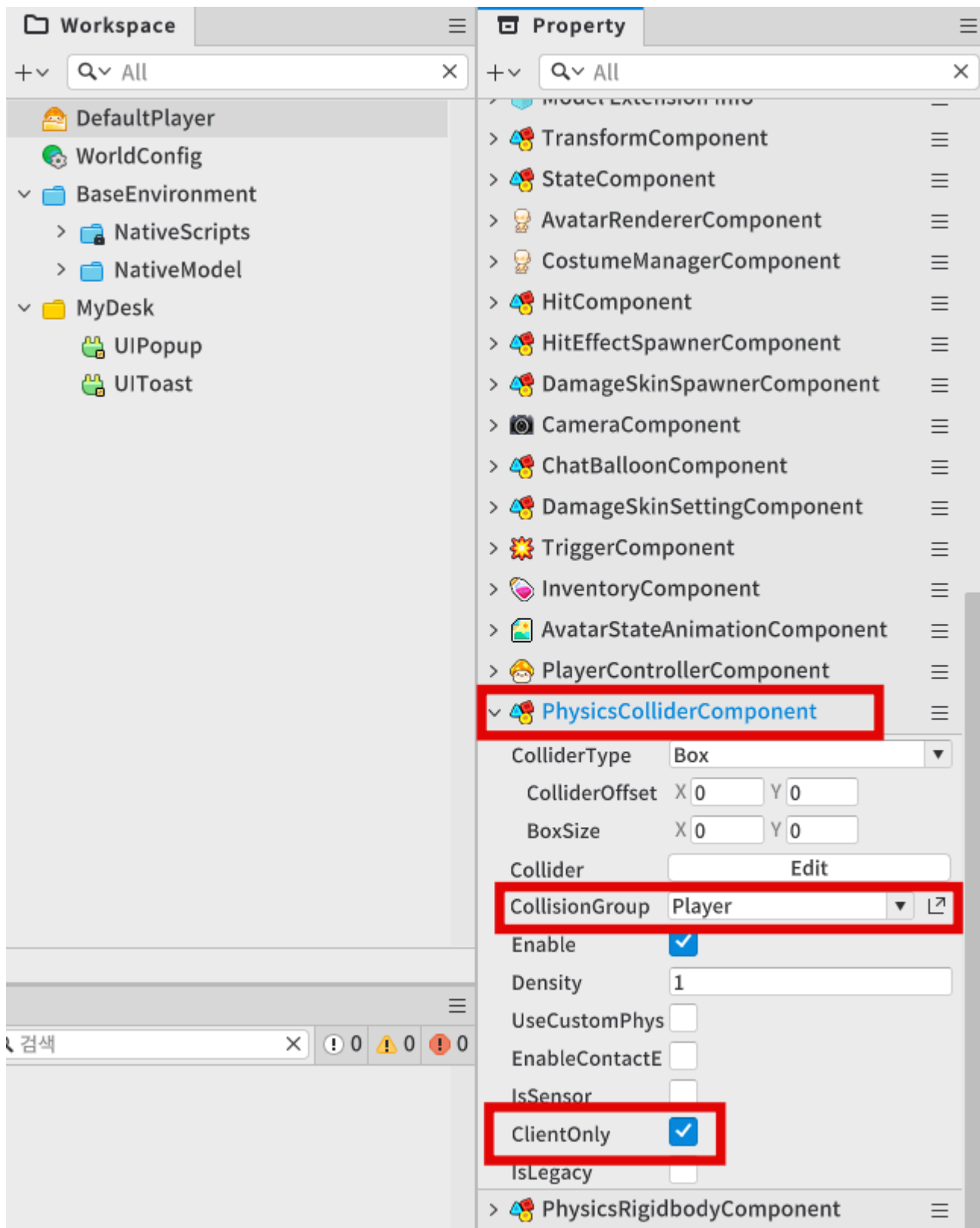
1. Select the Map entity from the Hierarchy tab after clicking AddComponent.
2. Register PhysicsSimulatorComponent.

Registering Character PhysicsComponent



1. Select the original DefaultPlayer and click the AddComponent button from the Property tab.
2. Register PhysicsCollider and PhysicsRigidbodyComponent.

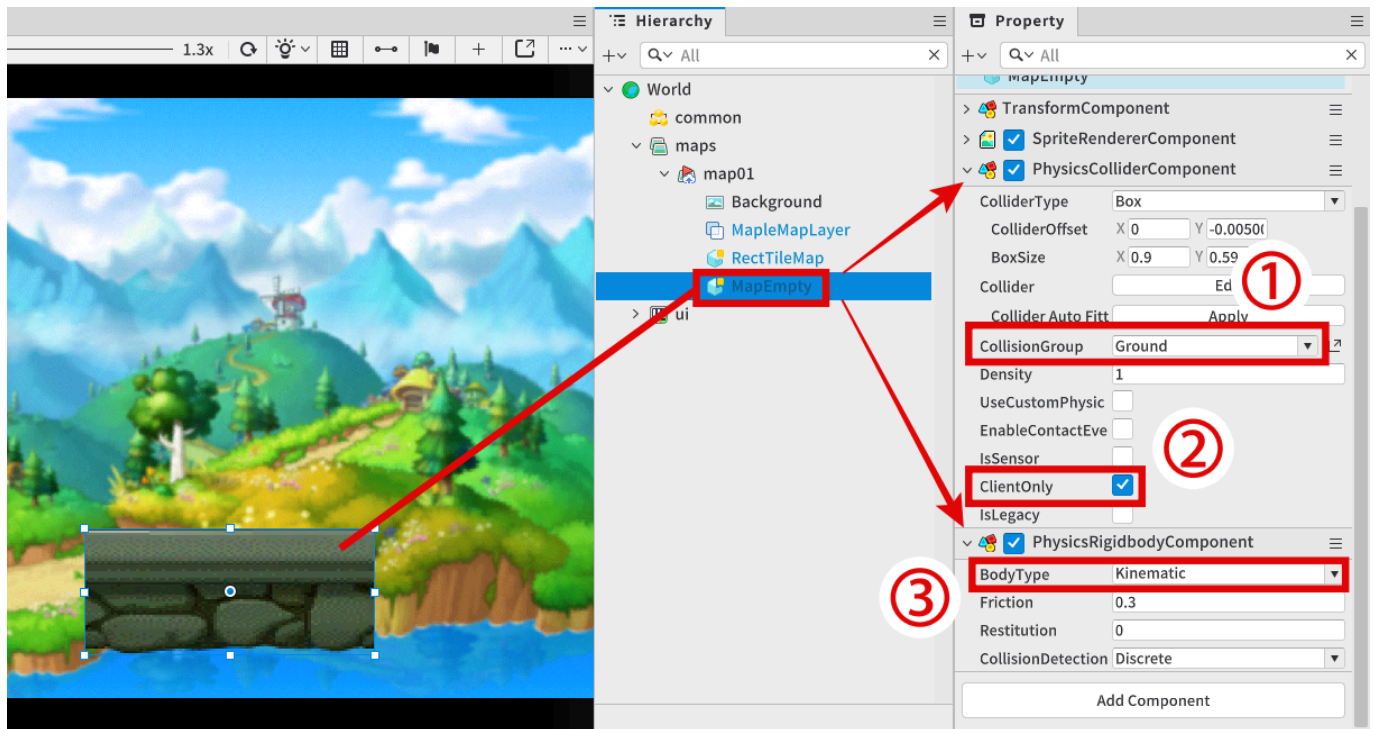
Setting a PhysicsColliderComponent



- Add a 'Player' group to CollisionGroup to set collision targets.
- Check the ClientOnly item from the internal properties of PhysicsColliderComponent, since the game doesn't interact with other players.

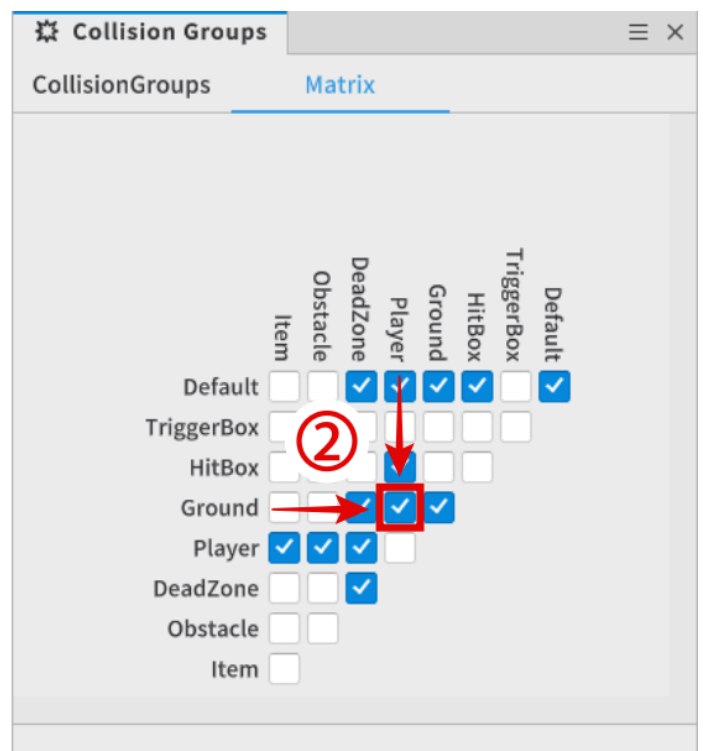
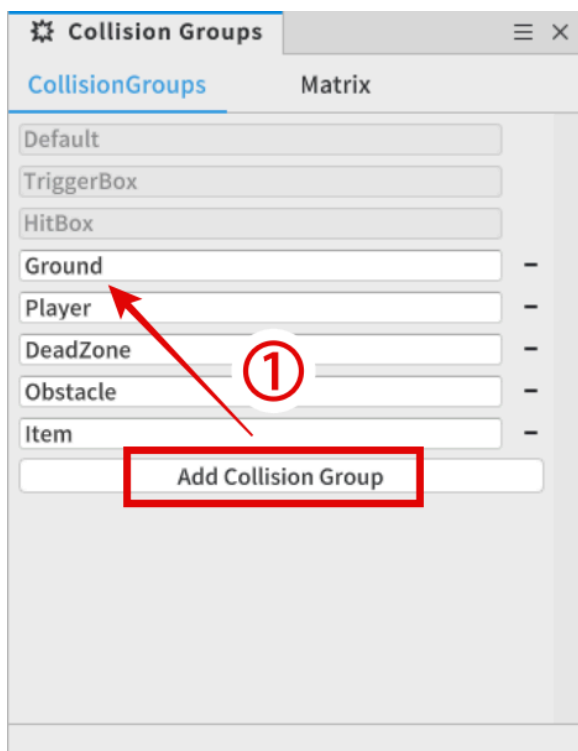
Floor Tile Entity Composition and Interaction Test

Setting the Test Floor Tile



- Create entities for testing and register PhysicsRigidbody and PhysicsColliderComponent like with characters.
1. Set CollisionGroup to Ground, so it can interact with characters.
 2. Check the ClientOnly item from the PhysicsColliderComponent internal properties, since it only works in a local environment like with characters.
 3. Set BodyType to Kinematic from the PhysicsRigidbodyComponent internal properties, since the entity that will be used as the floor will have to execute movement whole in a static state.

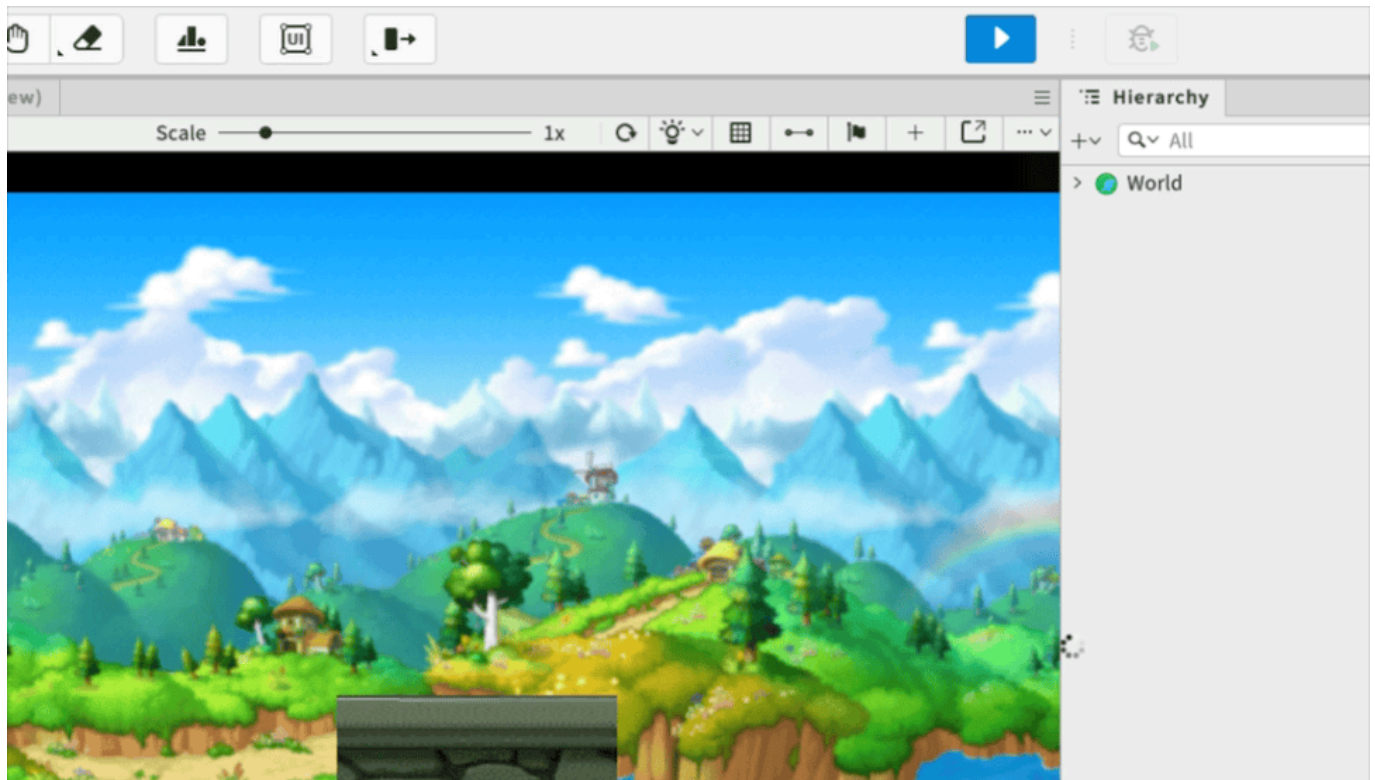
Adding and Setting CollisionGroup



1. Create a new CollisionGroup item using the CollisionGroup's Add Collision Group.

2. Confirm whether the Player and Ground lines are checked in the Matrix tab, so the collision between Player and Ground can be detected.

Testing Interactions



- Confirm if the character can stand on top of the tile entity while being affected by gravity as intended in a world environment that's launched through the play button on the top of the editor.

Minimum Implementation Standards

- Compose the running platformer game map that a character can normally run in.
 - The character shouldn't be floating in the air or be going through the floor tile.

Usage and Extension Direction

- Control the size of the character by adjusting the Scale value of the TransformComponent for the playable character and edit the collider size with the collider Edit button in PhysicsCollider. The game difficulty can be set through such settings.

[💡 Basic Training] Understanding Logic

Video Timeline

- You can follow along by using the timeline under the training video.
- Understanding Logic: 00:13:54

Learning Goals

- Understand the core concept of the Logic that functions Globally in MapleStory Worlds and clearly identify the differences it has between components that are owned by a normal entity.
- Understand the necessity of the 'manager' that centralizes control over various systems in the game and learn how to lower the project code complexity and how to increase the maintainability.
- Design the GameLogic, DataSetLogic, and UIManagerLogic that serve as the skeleton of the running platformer and implement a structure so that each logic exchanges data organically.

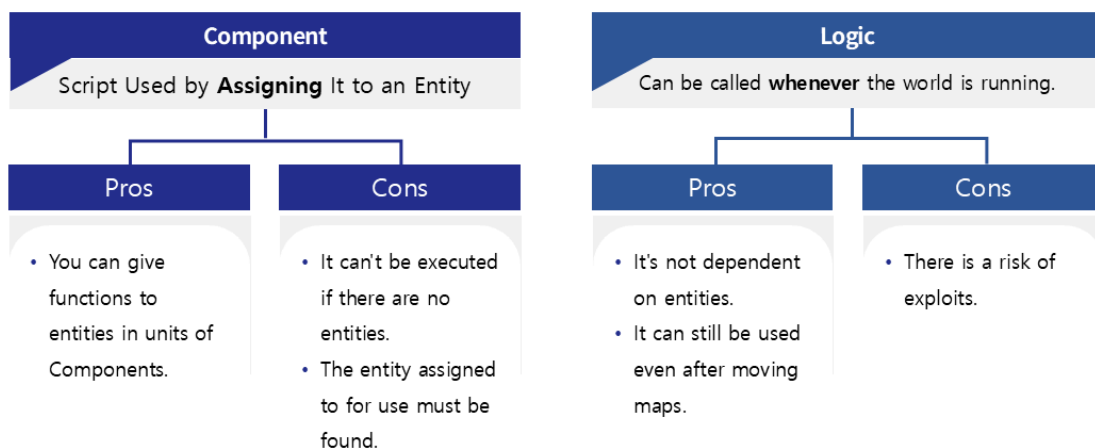
Practical Exercises to Do After Watching the Video

- Analyze what kind of functions can be focused on in terms of management and create them with a Logic script.
- Test if the created Logic script can be called as intended from the outside (components other than Logic).

What is Logic?

Key Point

Script Differences Based on **Whether the Entity is Dependent or Not**

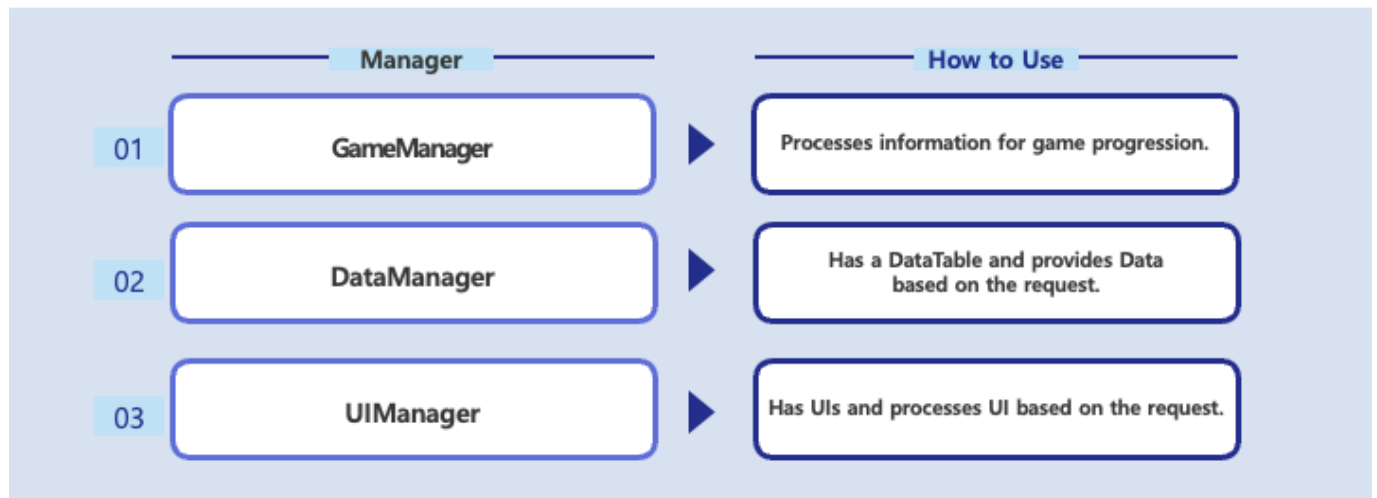


- It is one of the script types provided by MapleStory Worlds.
- When creating a Logic, it will be marked with a green plug icon within the Workspace.
- Also, unlike Components, it can access scripts even when not added to an entity.

Manager Concepts

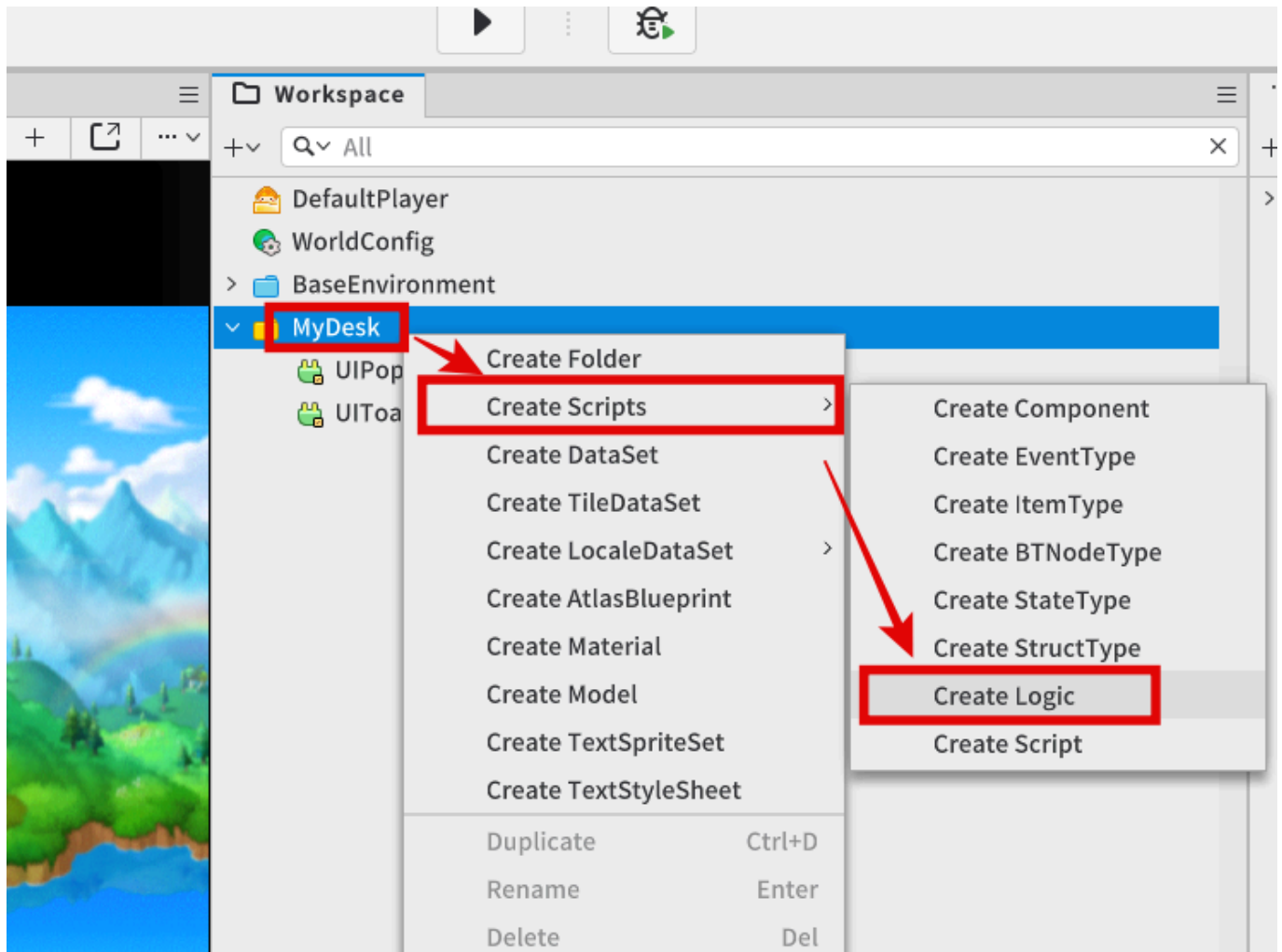
Key Point

Designed to control specific functions through the Manager.



- Logic can be created as a manager that executes specific functions or roles.
- The manager of a single function or role is called a Manager.
- When there are issues with a specific function, it can be fixed quickly and conveniently by only changing the content of the Manager.
 - Case 1: When a character completes damage calculation and delivers it to a monster.
 - Each time the damage formula changes or each time a new element is added to damage calculation, the formula needs to be changed.
 - All character codes must be individually found and edited.
 - Case 2: When making a request for damage calculation from a Logic-created Manager.
 - Change the damage formula inside the Logic.
 - The Logic handles the damage calculation of all characters and monsters, so there are no more elements that require editing.
 - Even if there are issues with the damage formula in future, editing the Manager can solve them.

How to Create Logic



- Right-click MyDesk from Workspace.
- Click Create Scripts from the menu screen.
- Click Create Logic to create a new Logic.

Manager Logic Usage Within the Sample World

Name	Usage
GameLogic	The Manager Logic that was created to manage functions that must be processed based on game progression. It is in charge of processing needed for the game, such as HP and score control, game start, game end, saving user data, etc.
DataSetLogic	The Logic for using and managing DataSet. It would be too much trouble to load each DataSet individually whenever data is needed. Once the game starts, DataTableLogic caches all DataSets and returns the cached data whenever needed.
UIManagerLogic	The Manager Logic for managing the UI. When a component or GameLogic needs to do processing related to UI, the request is sent to Manager Logic. UIManagerLogic has the components for all the UI, so it is in charge of requests related to UI.

- The sample world creates and uses Logic like in the above formats.

- When you need to create an additional function or extend one, you can declare your own Logic and create one for your use.

Logic Usage Example Within the Sample World

GameLogic Example

- Processes the function that continuously decreases HP and increases the score once the game begins.
- If the HP that is managed reaches 0, the results screen is requested and compares the current score to the previous high score and updates the user's high score as needed.
- Executes the timer function used for stage transitions that happen in set intervals when the main game begins.

DataSetLogic Example

- Reads the original data such as items, tiles, character info, etc. using DataService, which is a built-in function for MapleStory Worlds.
- Serves the role of attaching a tag (Key) to create an easily-readable Lua table variable format so that inputting the name is enough to find something in the data that was read.

UIManagerLogic Example

- Consolidates the reference path for UI elements, such as the title screen, character selection window, HP and score display, results pop-up, etc. for management.
- Gathers complex pathing to search each UI element into one and ensures ease of access is possible when another script needs to control specific UI controls.
- Executes the pop-up control function that only turns the necessary UI groups on or off based on the game state (start, play, end) while an entity is set as a reference.

Writing GameLogic Code

```
@Logic
script GameLogic extends Logic

property number discript = 0 -- The logic used to control the game's
overall flow.

property ScrollingController scrollingController = nil

property number currentScore = 0 -- The variable needed to contain the
current game score value.

@Sync
property number bestScore = 0 -- The variable needed to contain the
player's high score.

property number scoreIncreaseTimer = 0 -- The timer variable needed to
regularly update the score.
```

```
property number currentStage = 1 -- The variable needed to save the current
stage number.

property number stageTimer = 0 -- The timer variable needed to measure the
timing when the stage changes.

property number hp = 0 -- The variable needed to save the player's HP
value.

property boolean isEnd = true -- The state variable that records that the
game ended.

property boolean isMobile = true -- The variable that saves whether the
launching environment is a mobile environment.

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- This method is called when the world starts.

-- Proceeds with resetting the game logic.
self:Init()

-- Saves whether the world progression is done on mobile as a state
variable.
self.isMobile = Environment:IsMobilePlatform()

-- Plays the background music when the world starts.
_SoundService:PlayBGM("f5bb92d91e134c699e7bb2cf1aaca8e6",0.4)
end

@ExecSpace("ClientOnly")
method void Init()
-- Proceeds with resetting the main logic that manages the overall game.

-- Links the internal member references.
self:ConnectRefs()

-- Proceeds with resetting the other core logics.
self:OrderInit()
-- Resets the score value.
self:SetScore(0)
-- Resets the current HP value.
self:SetHP(100)
end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- This method links the internal member references.

-- Links the scrolling controller entity.
self.scrollingController =
_EntityService:GetEntityByPath("/maps/MainGame/Scrolling").ScrollingControl
ler
end
```

```
@ExecSpace("ClientOnly")
method void OrderInit()
-- Proceeds with the reset command of the core logics and components from
the game logic.

-- Proceeds with the scrolling controller reset.
self.scrollingController:Init()
-- Proceeds with data set logic reset.
_DataSetLogic:Init()
-- Proceeds with the UI manager logic reset.
_UIManagerLogic:Init()
-- Proceeds with the object pool logic reset.
_ObjectPoolLogic:Init()
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- This method is called for each frame.

-- If the game isn't in progress,
if self.isEnd == true then
    -- the method is halted.
    return
end

-- Automatically reduces HP by the delta value.
self:AutoDecreaseHp(delta)
-- Automatically increases the score by the delta value.
self:AutoIncreaseScore(delta)

-- /Branch 1/ If there is time remaining in the stage timer,
if self.stageTimer > 0 then
    -- the delta value is deducted from the stage timer value.
    self.stageTimer -= delta
-- /Branch 2/ If there is no time remaining in the stage timer,
else
    -- the stage change is called.
    self:StageChange()
    -- Resets the stage timer value to the value set in Config.
    self.stageTimer = _Config.stageChangeTime
end

end

@ExecSpace("ClientOnly")
method void AutoIncreaseScore(number delta)
-- This method increases the score every increaseScoreIntervalSecond
(interval).

-- /Branch 1/ If the measuring timer value is smaller than the Interval
value that is the standard,
if self.scoreIncreaseTimer < _Config.increaseScoreInterval then
```

```
-- the delta value is added to the timer and the method ends.
self.scoreIncreaseTimer += delta
return
end

-- /Branch 2/ If the measuring timer value is equal to or larger than the
Interval value that is the standard (the point when a specific interval is
reached),

-- the score is added.
self:AddScore(_Config.increaseScore)
-- Resets the measuring timer.
self.scoreIncreaseTimer = 0
end

@ExecSpace("ClientOnly")
method void AutoDecreaseHp(number delta)
-- This method automatically decreases the HP.

-- Delivers the value that must be deducted from the current HP.
self:AddHP(delta * _Config.decreaseHpValue)
end

@ExecSpace("ClientOnly")
method void SetScore(number value)
-- This method sets the value that was delivered from the parameter as the
Score value.

-- Parameter 1 number value: The value that is set as the score value.

-- Sets the current score value to a specified value.
self.currentScore = value
-- Sets the Text registered to UIManager as a specified value.
_UIManagerLogic:SetScoreText(value)
end

@ExecSpace("ClientOnly")
method void AddScore(number addValue)
-- This method adds the value that was delivered from a parameter to the
Score value.

-- Parameter 1 number addValue: Receives the value that is added to the
existing score value.

-- Adds the received value to the current score.
self.currentScore += addValue
-- Delivers the current score value to the Text registered to UIManager.
_UIManagerLogic:SetScoreText(self.currentScore)
end

@ExecSpace("ClientOnly")
method void SetHP(number value)
-- This method sets the value that was delivered from a parameter as the Hp
value.
```

```

-- Parameter 1 number value: Receives the value that is set as the Hp
value.

-- Sets the HP value as a specified value. (Limit the minimum and maximum
using clamp.)
self.hp = math.clamp(value,0,_Config.maxHp)

-- Applies the HP value to the slider registered to UIManager.
_UIManagerLogic:SetHpSliderValue(self.hp)
end

@ExecSpace("ClientOnly")
method void AddHP(number addValue)
-- This method adds the values that were delivered from a parameter to the
HP value.

-- Parameter 1 number addValue: Receives the HP value that will be added.

-- Adds the parameter value to the current HP.
self.hp += addValue

-- Limits the minimum and maximum using clamp.
self.hp = math.clamp(self.hp,0,_Config.maxHp)

-- Applies the HP value to the slider registered to UIManager.
_UIManagerLogic:SetHpSliderValue(self.hp)

-- If the HP value is equal to or smaller than 0,
if self.hp <= 0 then
    -- the end game logic is executed.
    self:EndMainGame()
end
end

@ExecSpace("Client")
method void EndMainGame()
-- This method ends the game. (client-exclusive method)

-- Converts to the end game state.
self.isEnd = true

-- Sets the character as not visible.
_UserService.LocalPlayer.Visible = false

-- Deactivates the joy stick if the launching environment is mobile when
the game ends.
_UIManagerLogic:ShowMobileJoyStick(false)

-- Designates the values that will be set in the results window pop-up and
outputs it on the screen. (Delivers the high score and current score.)
_UIManagerLogic:ShowResultPopup(tostring(math.floor(self.bestScore)),tostring(math.floor(self.currentScore)))

```

```
-- Proceeds with returning (deactivating) all tile set objects in the
scrolling controller.
self.scrollingController:ReturnAllTiles()

-- Decreases the background scrolling speed.
self.scrollingController.background.ScrollRate = 0.5

-- If the current score is higher than the high score that's managed within
the client,
if self.currentScore > self.bestScore then
    -- the high score value is replaced with the current score.
    self.bestScore = self.currentScore
    -- Updates the high score value in the server, since the high score has
    been changed.
    self:UpdateScore(self.bestScore)
end

-- Applies the main title's high score value.
_UManagerLogic:UpdateTitleBestScore(self.bestScore)
end

@ExecSpace("ClientOnly")
method void GameStart(number selectCharacterId)
-- This method is called when the main game starts.

-- Loads the character's speed value that was selected in the character
selection window from CharacterInfoTable.
local scrollSpeed =
_DataSetLogic.characterInfoTable[selectCharacterId].Speed
-- Registers the default speed value.
self:SetDefaultScrollSpeed(scrollSpeed)

-- Resets the current (starting) stage to 1.
self.currentStage = 1

-- Requests a stage change.
self:StageChange()

self:SetScore(0)

-- Resets the score timer value.
self.scoreIncreaseTimer = 0

-- Resets the HP value.
self:SetHP(100)

-- Sets the stage timer to the change time that's set in config.
self.stageTimer = _Config.stageChangeTime

-- Displays the mobile joy stick for mobile environments.
_UManagerLogic:ShowMobileJoyStick(true)

-- Proceeds with resetting the scrolling controller.
self.scrollingController:Reset()
```



```
-- Proceeds with resetting the player character.
_UserService.LocalPlayer.CustomPlayerController:ResetCharacter()

-- Sets the UI that should be provided to the user as visible.
_UIManagerLogic:ShowPlayerInfo(true)

-- Proceeds with resetting the skill logic.
_SkillLogic:Reset()

-- Only plays the resource effect to prevent the skill effect resource
buffering.
_SkillLogic:PlaySkillEffect()

-- Applies the image as unique skill icons that each character has.
_UIManagerLogic:RegistSkillIcon()

-- Proceeds with the resetting of item logic.
_ItemLogic:Reset()

-- Proceeds with the 3, 2, 1 count before the main game starts. (Includes
Wait method.)
_UIManagerLogic:CountTimerStart()

-- Converts the game state to in-progress.
self.isEnd = false

end

@ExecSpace("ClientOnly")
method void SetDefaultScrollSpeed(number speed)
-- Sets the character's default scrolling speed value.
-- Why Default and Current Are Divided: This is to separately save the
character's default speed, since the speed value changes when a booster
item is used.

-- Parameter 1 number speed: Receives the scrolling speed value that will
be applied when the game is running.

-- Sets the scrolling component's default speed value as the parameter
value.
self.scrollingController.defaultSpeed = speed
-- Sets the scrolling component's current speed value as the parameter
value.
self.scrollingController.currentScrollSpeed = speed
end

@ExecSpace("ClientOnly")
method void StageChange()
-- This method changes the stage.

-- Increases the stage number by 1, but sets to loop the resource element
using the % operator.
self.currentStage = (self.currentStage %
```

```
(_DataSetLogic.stageResourceCount+1)) + 1

-- Converts to the background screen that fits the current stage number.
self.scrollingController:ChangeBackground()
end

@ExecSpace("Server")
method void LoadData()
-- This method loads the recorded information. (server-exclusive method)
-- Server-Side Execution: Needed to load the data managed by the server.

-- Utilizes the data storage service to get data storage named 'MyData'.
local data = _DataStorageService:GetGlobalDataStorage("MyData")

-- Gets the Value value corresponding to the 'BestScore' key within that
data storage.
local errorCode,value = data:GetAndWait("BestScore")

-- Registers a temporary score value variable.
local tempScore

-- If no data is stored,
if value == nil then
    -- sets the temporary score value to 0.
    tempScore = 0

-- If there is any stored data,
else
    -- sets the temporary score value to the existing score value.
    tempScore = tonumber(value)
end

-- Updates the bestScore value.
-- When a property declared with Sync changes its value,
-- the value is synchronized on the client side as well, and the
OnSyncProperty method is called automatically.
self.bestScore = tonumber(tempScore)
end

@ExecSpace("Server")
method void SaveScore(number score)
-- This method saves the user's information.

-- This method saves the score to data storage. (server-exclusive method)

-- Utilizes the data storage service to get data storage named 'MyData'.
local data = _DataStorageService:GetGlobalDataStorage("MyData")

-- Registers the 'score' value to the Value value that corresponds to the
'BestScore' key in the target storage.
local errorCode = data:SetAndWait("BestScore", tostring(score))
end

@ExecSpace("Server")
```

```

method void UpdateScore(number score)
-- This method updates the score value that's managed server-side. (server-
exclusive method)

-- Parameter 1 number score: Receives the score value managed by the
client.

-- Sets the server-side bestScore value as the parameter value.
self.bestScore = score
end

@ExecSpace("ClientOnly")
method void OnSyncProperty(string name, any value)
-- This method is called automatically when there is a value change server-
side limited to properties that were declared by Sync.
-- However, if the changed value from the server is the same as the
existing client value, this method isn't called.

-- If the changed property name is bestScore,
if name == "bestScore" then
    -- a high score text update in the title screen is requested to
    UIManagerLogic.
    _UIManagerLogic:UpdateTitleBestScore(toNumber(value))
end
end

@EventSender("Service", "UserService")
handler HandleUserEnterEvent(UserEnterEvent event)
-- This method is called when accessing users and entering.

-- Loads the data from server-side.
self:LoadData()

end

@EventSender("Service", "UserService")
handler HandleUserLeaveEvent(UserLeaveEvent event)
-- This method is called when a user leaves.

-- Executes the method that registers the high score to the server.
self:SaveScore(self.bestScore)
end

end

```

Writing DataSetLogic Code

```

@Logic
script DataSetLogic extends Logic

```

```

property string discript = "" -- The logic needed to manage the data that's
managed in the target world.

property table tileSetInfoTable = {} -- The table needed to contain the
tile set data.

property number totalTileSetCount = -1 -- The variable that saves the
number of tile types.

property table itemInfoTable = {} -- The table needed to contain the items'
data.

property table tilesFloorInfoTable = {[1]={} , [2]={}} -- The table that
contains the floor information of the tile set. / ex table[1] = {2,4} Tile
set no. 2 and 4 will start from Fl. 1.

property table characterInfoTable = {} -- The table that saves character
information.

property table skillInfoTable = {} -- The table that saves skill
information.

property table stageRenderInfoTable = {} -- The table that saves resource
information that needs to be rendered by each stage.

property number stageResourceCount = 0 -- The number of stage resource
types.

@ExecSpace("ClientOnly")
method void Init()
-- This method resets the DataSetLogic component.

-- Resets the tile set table.
self:InitTileSetInfoTable()
-- Resets the item table.
self:InitItemInfoTable()
-- Resets the character table.
self:InitCharacterInfoTable()
-- Resets the skill table.
self:InitSkillInfoTable()
-- Resets the stage render table.
self:InitStageRenderInfoTable()
end

@ExecSpace("ClientOnly")
method void InitTileSetInfoTable()
-- This method is needed to save the data set that's related to tile
object.

-- This variable is needed to access the name of the data set that is saved
in world space as local.
local dataSetName = "TileSetInfo"

```

```

-- The number of Rows of the target data set.
local rowCount = self:GetDataSetRowCount(dataSetName)

-- Saves the number of tile set types.
self.totalTileSetCount = rowCount

-- This variable is needed to contain the data in each line.
local rowData

-- Starts iterations a number of times equal to the number of data set Rows
starting from 1.
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- The number of floors for the first tile of the tile set object.
        FirstFloor =
tonumber(_DataService:GetCell(dataSetName,i,"FirstFloor")),
        -- The number of floors for the last tile of the tile set object.
        LastFloor =
tonumber(_DataService:GetCell(dataSetName,i,"LastFloor")),
        -- The original entity model (prefab) Id needed to create a tile
set object.
        EntryID = _DataService:GetCell(dataSetName,i,"EntryID")
    }

    -- Saves the data saved to rowData to the table.
    self.tileSetInfoTable[i] = rowData

    -- Saves the number of floors currently registered in the tile set as a
variable.
    local firstTileFloor = self.tileSetInfoTable[i].FirstFloor

    -- /Branch 1/ If the first tile floor number is Fl. 1,
    if firstTileFloor == 1 then
        -- saves to the Fl. 1 table list.
        table.insert(self.tilesFloorInfoTable[1],i)
    -- /Branch 2/ If the first tile floor number isn't Fl. 1,
    else
        -- saves to the Fl. 2 table list.
        table.insert(self.tilesFloorInfoTable[2],i)
    end
end

end

@ExecSpace("ClientOnly")
method number GetDataSetRowCount(string Name)
-- Returns the number of Rows of the target data set.

-- Parameter 1 string Name: Receives the name of the target data set.

-- Returns the number of Rows of the verified data set.
return _DataService:GetRowCount(Name)

```

end

@ExecSpace("ClientOnly")

method void InitItemInfoTable()

-- This method is needed to save the item data set.

-- This variable is needed to access the name of the data set that is saved in world space as local.

local dataSetName = "ItemInfo"

-- The number of Rows of the target data set.

local rowCount = self:GetDataSetRowCount(dataSetName)

-- This variable is needed to contain the data in each line.

local rowData

-- Iterates a number of times equal to the number of Rows of the data set starting from 1.

for i = 1 , rowCount , 1 do

 rowData =

 {

 -- Saves the item name.

 Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),

 -- Saves the item's RUID.

 RUID = tostring(_DataService:GetCell(dataSetName,i,"RUID")),

 -- Saves the unique effect value that the item has.

 Value = tonumber(_DataService:GetCell(dataSetName,i,"Value")),

 -- Saves the item creation chance.

 SpawnRate =

 tonumber(_DataService:GetCell(dataSetName,i,"SpawnRate")),

 -- Saves the item model (prefab) EntryID value.

 EntryID = tostring(_DataService:GetCell(dataSetName,i,"EntryID"))

 }

 -- Saves the data saved to rowData to the table.

 self.itemInfoTable[i] = rowData

end

end

@ExecSpace("ClientOnly")

method void InitCharacterInfoTable()

-- This method is needed to save the character data set.

-- This variable is needed to access the name of the data set that is saved in world space as local.

local dataSetName = "CharacterInfo"

-- The number of Rows of the target data set.

local rowCount = self:GetDataSetRowCount(dataSetName)

-- This variable is needed to contain the data in each line.

local rowData

-- Iterates a number of times equal to the number of Rows of the data set

```

starting from 1.
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- Saves the character name.
        Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
        -- Saves the unique character scrolling speed value.
        Speed = tonumber(_DataService:GetCell(dataSetName,i,"Speed")),
        -- Saves the character's unique skill Id.
        SkillId = tonumber(_DataService:GetCell(dataSetName,i,"SkillId")),
        -- Saves the RUID value that will be set in the Costume renderer,
        which will be attached to a character.
        RUID = tostring(_DataService:GetCell(dataSetName,i,"RUID"))
    }

    -- Saves the data saved to rowData to the table.
    self.characterInfoTable[i] = rowData
end
end

@ExecSpace("ClientOnly")
method void InitSkillInfoTable()
-- This method is needed to save the skill data set.

-- This variable is needed to access the name of the data set that is saved
in world space as local.
local dataSetName = "SkillInfo"

-- Verifies the number of rows in the target data set.
local rowCount = self:GetDataSetRowCount(dataSetName)

-- Saves the number of internal member resource types.
self.stageResourceCount = rowCount

-- This variable is needed to contain the data in each line.
local rowData

-- Starts iterations a number of times equal to the number of data set Rows
starting from 1.
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- Saves the skill name.
        Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
        -- Saves the skill use effect's unique value.
        Value = tonumber(_DataService:GetCell(dataSetName,i,"Value")),
        -- Saves the skill effect RUID value.
        EffectRUID =
tostring(_DataService:GetCell(dataSetName,i,"EffectRUID")),
        -- Saves the in-game UI RUID value.
        UI_RUID = tostring(_DataService:GetCell(dataSetName,i,"UI_RUID"))
    }

    -- Saves the data saved to rowData to the table.

```



```

        self.skillInfoTable[i] = rowData
    end
end

@ExecSpace("ClientOnly")
method void InitStageRenderInfoTable()
-- This method is needed to save the resource information data set for each
stage.

-- This variable is needed to access the name of the data set that is saved
in world space as local.
local dataSetName = "StageRenderInfo"

-- Verifies the number of rows in the target data set.
local rowCount = self:GetDataSetRowCount(dataSetName)

-- This variable is needed to contain the data in each line.
local rowData

-- Iterates a number of times equal to the number of Rows of the data set
starting from 1.
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- Saves the floor tile RUID value that will be applied for each
stage.
        TileRUID =
tostring(_DataService:GetCell(dataSetName,i,"TileRUID")),
        -- Saves the obstacle object RUID value that will be applied for
each stage.
        ObstacleRUID =
tostring(_DataService:GetCell(dataSetName,i,"ObstacleRUID")),
        -- Saves the background RUID value that will be applied to each
stage.
        BackgroundRUID =
tostring(_DataService:GetCell(dataSetName,i,"BackgroundRUID"))
    }

    -- Saves the data saved to rowData to the table.
    self.stageRenderInfoTable[i] = rowData
end
end

end

```

Writing UIManagerLogic Code

```

@Logic
script UIManagerLogic extends Logic

```

```
property number discript = 0 -- The logic needed to manage the overall UI,
entities, and internal values.

property SliderComponent hpSlider = nil

property Entity toastUIGroup = nil

property TextComponent inGameCurrentScoreText = nil

property Entity resultPopup = nil

property Entity resultBestScoreText = nil

property Entity currentScoreText = nil

property Entity resultToMenuButton = nil

property Entity mainTitlePanel = nil

property Entity characterSelectPopup = nil

property SpriteGUIRendererComponent skillRenderer = nil

property TextComponent titleBestScore = nil

property Entity inGameScoreText = nil

property TextComponent counterText = nil

property Entity titleGameStartButton = nil

property Entity defaultUIGroup = nil

property Entity mobileSlideButton = nil

property Entity mobileJumpButton = nil

@ExecSpace("ClientOnly")
method void Init()
-- This method proceeds with resetting UIManagerLogic.

-- Links the references of internal members.
self:ConnectRefs()

-- Activates the main canvas that contains the UI entities when proceeding
with a reset.
self.toastUIGroup.Enable = true
-- Links the method that reverts to the initial game launch state when the
main menu button is clicked from the results window.
self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
-- Links the method that displays the character selection window when the
start game button is pressed from the title screen.
self.titleGameStartButton:ConnectEvent(ButtonClickEvent,self.ShowCharactersS
```

```
selectPopup)
end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- This method reference links the members that are internally required.

-- Links the toastUIGroup entity.
self.toastUIGroup = _EntityService:GetEntityByPath("/ui/ToastGroup")
-- Links the defaultUIGroup entity.
self.defaultUIGroup = _EntityService:GetEntityByPath("/ui/DefaultGroup")

-- Links the main title panel entity.
self.mainTitlePanel =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle")
-- Links the main title screen high score text component.
self.titleBestScore =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UITitleBestScorePanel/UITitleBestScore").TextComponent
-- Links the start game button entity in the main title screen.
self.titleGameStartButton =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UIButtonGameStartBtn")

-- Links the character selection window pop-up entity.
self.characterSelectPopup =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UICharacterSelect")

-- Links the UI hpSlider reference, located on the top of the in-game screen.
self.hpSlider =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UI_HPbar").SliderComponent
-- Links the UI ScoreText (score value) reference, located on the top of the in-game screen.
self.inGameCurrentScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UIInGameScore").TextComponent
-- Links the UI ScoreText (only displays the 'Score' text) reference, located on the top of the in-game screen.
self.inGameScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UIScoreText")
-- Links the in-game skill icon component.
self.skillRenderer =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UI_Skill").SpriteGUIRendererComponent
-- Links the start standby timer text upon starting the in-game game start.
self.counterText =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UICounterText").TextComponent
-- Links the joy stick button entity to execute the character's jump function in the in-game and mobile environment.
self.mobileJumpButton =
_EntityService:GetEntityByPath("/ui/DefaultGroup/Button_Jump")
-- Links the joy stick button entity to execute the character's slide
```

```

function in the in-game and mobile environment.
self.mobileSlideButton =
_EntityService:GetEntityByPath("/ui/DefaultGroup/Button_Slide")

-- Links the results screen pop-up entity.
self.resultPopup =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup")
-- Links the high score text entity within the results window.
self.resultBestScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup/UIBestScoreText/UIBestScore")
-- Links the current score text entity within the results window pop-up.
self.currentScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup/UIScoreText/UIScore")
-- Links the menu button entity within the results window pop-up.
self.resultToMenuButton =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup/UISMainMenuBtn")
)
end

@ExecSpace("ClientOnly")
method void UpdateTitleBestScore(number score)
-- This method sets the high score value that will be displayed in the main
title screen.

-- Parameter 1 number score: Sets the score value for the title screen with
the received value.

-- If there is no score value,
if score == nil or score <= 0 then
    -- the main title screen's high score text value is set to '0'.
    self.titleBestScore.Text = "0"

-- If there is a score value,
else
    -- the main title screen's high score text is registered as the
received value. Anything past the decimal point is dropped using
math.floor.
    self.titleBestScore.Text = tostring(math.floor(score))

end
end

@ExecSpace("ClientOnly")
method void ShowCharacterSelectPopup()
-- This method displays the character selection window.

-- Loads the character selection pop-up using entity service.
local characterSelectPopup =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UICharacterSelect")

-- Activates pop-up entities.
characterSelectPopup.Enable = true

```

end

```
@ExecSpace("ClientOnly")
method void ShowMobileJoyStick(boolean active)
-- This method displays the joy stick controller to control the character
in a mobile environment.
```

```
-- Parameter 1 boolean active: Activates if true, deactivates if false.
```

```
-- If the current platform environment is on mobile,
```

```
if _GameLogic.isMobile == true then
```

```
-- the activation and deactivation of the UI canvas that includes the
joy stick controller will be controlled with a parameter value.
```

```
self.defaultUIGroup.Enable = active
```

```
end
```

```
end
```

```
@ExecSpace("ClientOnly")
```

```
method void CountTimerStart()
```

```
-- This method proceeds with a countdown before the game starts once the
main game is launched.
```

```
-- Activates the count text object.
```

```
self.counterText.Entity.Enable = true
```

```
-- Sets the counter text value to '3'.
```

```
self.counterText.Text = "3"
```

```
-- Sets logic execution on standby for 1 second.
```

```
wait(1)
```

```
self.counterText.Text = "2"
```

```
wait(1)
```

```
self.counterText.Text = "1"
```

```
wait(1)
```

```
-- Deactivates the count text object, since the countdown is over.
```

```
self.counterText.Entity.Enable = false
```

```
end
```

```
@ExecSpace("ClientOnly")
```

```
method void ShowPlayerInfo(boolean active)
```

```
-- This method deactivates the UI element that needs to be provided to the
player in-game.
```

```
-- Parameter 1 boolean active: Activates if true, deactivates if false.
```

```
-- Sets the in-game hp slider activation state with a parameter value.
```

```
self.hpSlider.Entity.Enable = active
```

```
-- Sets the activation state of the text object that displays the current
score value in-game with a parameter value.
```

```
self.inGameCurrentScoreText.Entity.Enable = active
```

```
-- Sets the activation state of the in-game 'score' text object with a
parameter value.
```

```
self.inGameScoreText.Enable = active
```

```
-- Sets the activation state of the in-game skill icon object with a
```

```
parameter value.
self.skillRenderer.Entity.Enable = active
end

@ExecSpace("ClientOnly")
method void RegistSkillIcon()
-- This method registers the current in-game character's skill icon.

-- Loads the skill icon's RUID value from the table based on the skill
information registered to skillLogic.
self.skillRenderer.ImageRUID =
_DataSetLogic.skillInfoTable[_SkillLogic.skillId].UI_RUID
-- Sets the alpha value to the max value to display the skill as still
available.
self.skillRenderer.Color.a = 1
end

@ExecSpace("ClientOnly")
method void ShowResultPopup(string bestScore, string currentScore)
-- This method displays the results window pop-up.

-- Registers the high score text.
self.resultBestScoreText.TextComponent.Text = bestScore

-- Registers the current score text.
self.currentScoreText.TextComponent.Text = currentScore

-- Activates the results window pop-up.
self.resultPopup.Enable = true;
end

@ExecSpace("ClientOnly")
method void ReturnToMenu()
-- This method reverts to the main menu state after the game ends.

-- Deactivates the UI elements that are displayed in-game.
self:ShowPlayerInfo(false)

-- Deactivates the character selection window pop-up.
self.characterSelectPopup.Enable = false
-- Deactivates the results window pop-up that displays after the game ends.
self.resultPopup.Enable = false
-- Activates the panel object that needs to be displayed in the main menu
title.
self.mainTitlePanel.Enable = true
end

@ExecSpace("ClientOnly")
method void SetScoreText(number value)
-- This method sets the value that's delivered as a parameter as the UI
Score text.

-- Parameter 1 number value: Receives the value that will be set as the
score value.
```

```

self.inGameCurrentScoreText.Text = tostring(math.floor(value))
end

@ExecSpace("ClientOnly")
method number SetHpSliderValue(number value)
-- This method sets the hp slider value as a parameter value and returns
the Value value of the current slider.

-- Parameter 1 number value: Receives the value that will be set as the
slider.

-- Sets the slider value as the received value and sets the max HP value,
so it can't go over a set amount using clamp.
self.hpSlider.Value = math.clamp(value, 0, _Config.maxHp)

-- Returns the current slider's value.
return self.hpSlider.Value
end

end

```

Minimum Implementation Standards

- Create a Logic that will manage and make use of future DataSets.
 - Manages the DataSet by creating a DataSetLogic based on the sample world.
- Create a Logic that handles the game's progression and overall management.
 - Create a sample world-based GameLogic to implement processing based on game state.

Usage and Extension Direction

- Change the function that adds to the score in set intervals in seconds with its own score-granting formula to change the game's flow.
- Create a 'Config' logic that manages the fixed values that **isn't intended to change** using Logic.
 - EX: Change the game's difficulty by changing the previously set max HP value.
 - EX: Change the game's difficulty by changing how much the HP is decreased over time.

[💡 Basic Training] Implementing the Scroll System

Video Timeline

- You can follow along by using the timeline under the training video.
- Implementing the Scroll System: 00:25:46

Learning Goals

- Understand the core principles of the scroll system that moves the background in the opposite direction of the character to make it look like the character is continuously running.
- Write a script that can smoothly move the entity at a consistent speed over time without being affected by frames by utilizing the OnUpdate method and delta time.
- Adjust the game's difficulty by controlling the speed of the background and floor tiles.
- Understand how the stage environment changes dynamically based on the game's progress.

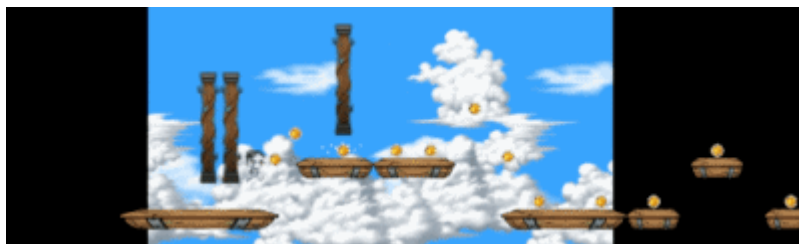
Practical Exercises to Do After Watching the Video

- Change the BackgroundComponent's ScrollRate value in the Map entity's child Background entity to set the appropriate background movement speed.

Understanding the Scroll System's Basic Structure and Principles

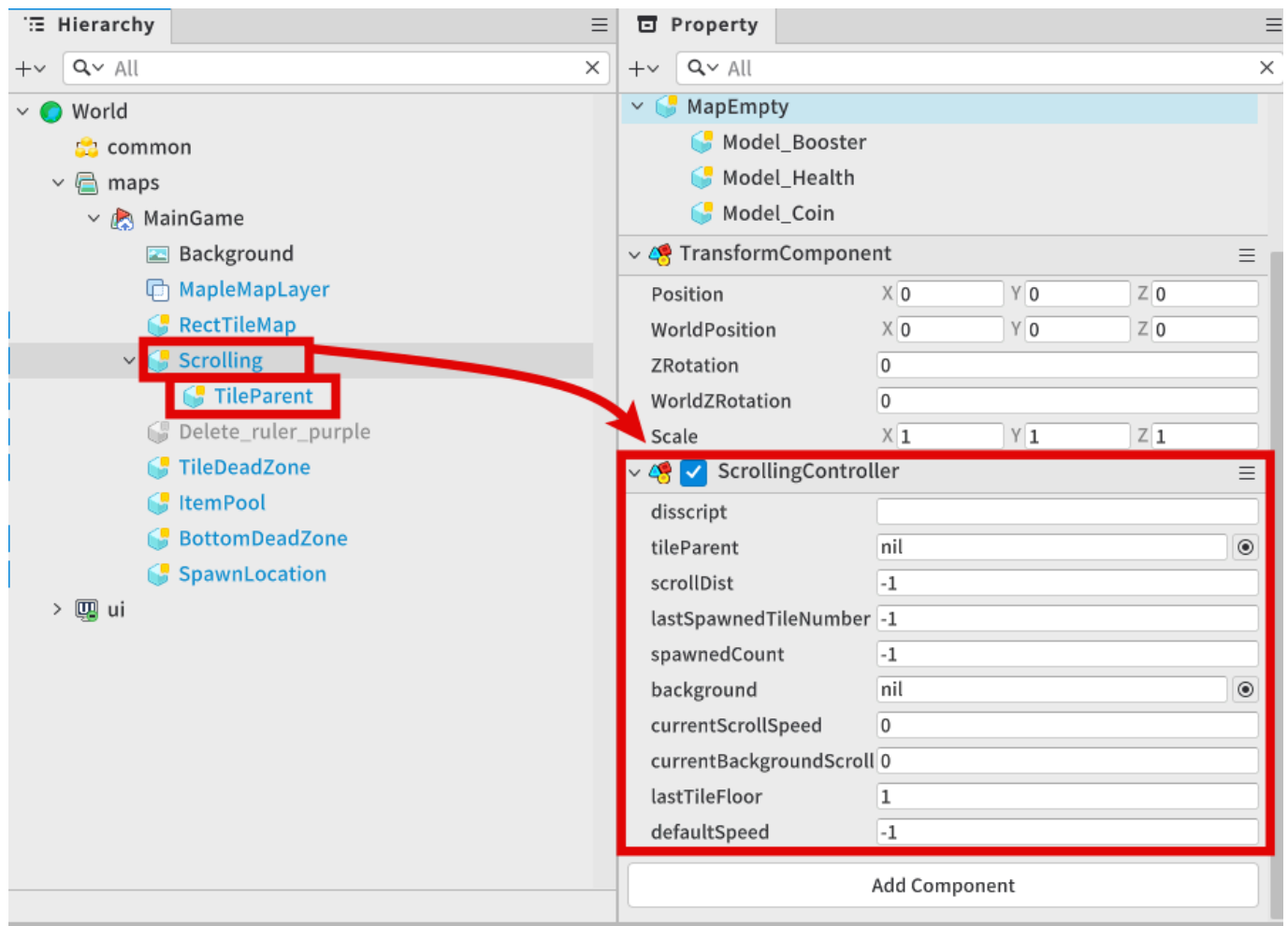
- If the character is allowed to endlessly run to the right in a running platformer game, you will be forced to create an infinitely long map.
- You may run into a critical error in the system if the character's coordinate values become infinitely larger.
- The scroll system needs to be implemented to overcome the physical limits and use the infinite map with stability.

Relative Scrolling



- The camera and player character don't move and are placed in a fixed location.
- Instead, the floor tile that the character is standing on, obstacles, and background elements are continuously moved to the left, the opposite direction.
- This creates the illusion that the player character is moving to the right.
- As a result, you can create an environment where the character infinitely runs to the right while staying in place regardless of the size of the map.

Creating ScrollingController



- Register the ScrollingController component to an empty entity.
 - ScrollingController is the subject that manages the game's overall scroll environment.
 - Set the created entity's TransformComponent Position value to (0,0,0).
- Create an empty entity and set it as the child entity of an entity that has an attached ScrollingController component.
 - This entity serves the role of a parent entity that's meant to move the entire tile set.
 - Set the created entity's TransformComponent Position value to (0,0,0) to place it on the same line as the parent.

Composing ScrollingController Component

- The code is required to control the background and floor tile.
- Write the following code.
- The target code is based on Plain Text.

```
@Component
script ScrollingController extends Component

property string discript = "" -- This logic is for the overall management
of the side-scrolling movement in a running platformer game.

property Entity tileParent = nil

property number scrollDist = -1 -- This variable saves the current scroll
```

```
distance. // The distance per tile set is 10 in config.

property number lastSpawnedTileNumber = -1 -- The tile number that was
created very last.

property number spawnedCount = -1 -- This variable is a record of how many
tiles are spawned.

property BackgroundComponent background = nil

property number currentScrollSpeed = 0 -- This variable saves the current
scrolling speed value.

property number currentBackgroundScrollSpeed = 0 -- This variable saves the
current background scrolling speed value.

property number lastTileFloor = 1 -- This variable saves the number of tile
floors that was spawned last.

property number defaultSpeed = -1 -- The scroll speed value that's set upon
character selection.

@ExecSpace("ClientOnly")
method void Scroll(number moveDistance)
-- Moves the activated TileSet object by the distance that received the
currently-set scroll.

-- Moves the parent object (TileParent).
self.tileParent.TransformComponent:Translate(moveDistance,0)
end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- This method reference links members that are internally necessary.

-- Links the Entity of the object that will be the parent of all tile sets.
self.tileParent =
_EntityService:GetEntityByPath("/maps/MainGame/Scrolling/TileParent")

-- Links the Background component.
self.background =
_EntityService:GetEntityByPath("/maps/MainGame/Background").BackgroundCompo
nent
end

@ExecSpace("ClientOnly")
method void Init()
-- This method proceeds with an internal reset for a Scrolling Controller
component.

-- Proceeds with internal reference linking.
self:ConnectRefs()

-- Sets the background screen scrolling speed value to the background
```

```

scrolling speed set in Config.
self.background.ScrollRate = _Config.backgroundScrollSpeed

-- Sets the scrolling speed value of the actual background object.
self.currentBackgroundScrollSpeed = _Config.backgroundScrollSpeed

-- Resets the number of created tile sets.
self.spawnedCount = 0
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- This method is called for each frame.

-- If the main game is closed,
if _GameLogic.isEnd == true then
    -- the Update method action is stopped.
    return
end

-- Executes the floor object movement with the set scrolling speed.
self:Scroll(-1 * delta * self.currentScrollSpeed)
end

@ExecSpace("ClientOnly")
method void SpawnRandomTile()
-- This method creates random tile sets.

-- Extracts a random, not duplicated number and the latest created tile
number.
local randomNumber = self:GetRandomTileNumber()

-- Creates a tile with the randomly-extracted number.
self:SpawnTile(randomNumber)
end

@ExecSpace("ClientOnly")
method number GetRandomTileNumber()
-- This method extracts a random tile number.

-- This process loads the table that contains the tile set numbers that can
be linked with the floor count of the tile set that was spawned last.
-- ex) End Point Floor Number: Fl. 2 / Table containing the tile set number
starting with Fl. 2 0
-- ex) End Point Floor Number: Fl. 1 / Table containing the tile set number
starting with Fl. 2 X
-- This is to create a linked tile set.

-- This is the core logic for sequential map creation.

-- When assuming that Fl. N was the end point of the tile that was spawned
last,
-- a table that contains the tile set numbers that can start with Fl. N is
loaded.

```

```

local tileSetFloorInfoTable =
_DataSetLogic.tilesFloorInfoTable[self.lastTileFloor]

-- Confirms the target table's total count.
local tileSetTableTotalCount = #tileSetFloorInfoTable

-- Extracts a random number with a range from 1 to the number of tile
types.
local randomNumber = math.random(1, tileSetTableTotalCount)

-- Extracts the 'randomNumber' tile set number from the table that only
contains the relevant floor number.
local selectedTileNumber = tileSetFloorInfoTable[randomNumber]

-- This logic is to prevent consecutive tile spawn creations. (If you want
to allow for consecutive tile set creations, don't execute the syntax
below.)

-- If the randomly-drawn number is the same as the the previously-spawned
tile number (another random draw takes place to prevent the same tile being
selected again),
if selectedTileNumber == self.lastSpawnedTileNumber then

    -- a random number is extracted again. (Extracts a random element from
the tile set elements that match the relevant floor number.)
    local tempNumber = math.random(1, tileSetTableTotalCount)

    -- If the random number is a duplicate number again,
    if tempNumber == randomNumber then
        -- the randomly-drawn number value is increased by 1 and then the
remaining operator is utilized.
        tempNumber = (tempNumber + 1) % tileSetTableTotalCount + 1

        -- This is why the first starting value of the data set is 1 and,
if the product of remaining calculation is 0,
        if tempNumber == 0 then
            -- the tempNumber value is set to 1.
            tempNumber = 1
        end

    end

    -- Selects the randomly-drawn index 'tempNumber' element from the table
that only contains the relevant floor number.
    selectedTileNumber = tileSetFloorInfoTable[tempNumber]
end

-- Returns the target tile set number.
return selectedTileNumber
end

@ExecSpace("ClientOnly")
method void SetScrollSpeed(number speed)

```

```

-- This method controls the floor tile object's speed.

-- Parameter 1 number speed: Receives the intended speed setting.

-- Sets the current speed value as the speed value that was received from a
parameter.
self.currentScrollSpeed = speed
end

@ExecSpace("ClientOnly")
method void SetBackgroundScrollSpeed(number speed)
-- This method adjusts the background scroll speed.

-- Parameter 1 number speed: Receives the intended speed setting.

-- Sets the background scroll speed as the parameter value.
self.background.ScrollRate = speed
end

@ExecSpace("ClientOnly")
method void ReturnAllTiles()
-- This method returns (deactivates) all created tile set objects.

-- Stores 'TileSet' components into a table by accessing the tileParent
children recursively.
local tileSetTable =
self.tileParent:GetChildComponentsByTypeName("TileSet",true)

-- Traverses a number of times equal to the number of tables that contains
'TileSet' components.
for i = 1, #tileSetTable, 1 do

    -- If the object of each element is on (activated) (to prevent only
objects that are turned on from being returned),
    if tileSetTable[i].Entity.Enable == true then

        -- accesses as a tile set component for each element.
        local tempTileset = tileSetTable[i].Entity.TileSet
        -- Retrieves all items that were placed as the children of the tile
set.
        tempTileset:ReturnAllItems()
        -- Returns the relevant tile set as an object pool again.
        _ObjectPoolLogic:ReturnTile(tempTileset.Entity)
    end

end

end
end

method void SpawnTile(number tileSetNumber)
-- This method spawns the tile set object.

-- Parameter 1 number tileSetNumber: Receives to create the target
parameter number's tile set.

```

```

-- Receives the 'parameter' tile object from the object pool.
local spawnedTile = _ObjectPoolLogic:GetTile(tileSetNumber)

-- Records the floor of the end point for the tile set that was spawned
from the object pool.
self.lastTileFloor =
_DataSetLogic.tileSetInfoTable[tileSetNumber].LastFloor

-- Sets the X-axis distance from the parent object that act as the basis of
the tile set.
-- spawnedCount(Previously-Created Tile Set Count) * 10(Tile Set Prefab's
Standard Distance Unit)
spawnedTile.TransformComponent.Position = Vector3(self.spawnedCount *
10,0,0)

-- Increases the number of created tile set by 1.
self.spawnedCount += 1
-- Replaces the tile set number that was created last with the tile set
number that was fetched from the object pool.
self.lastSpawnedTileNumber = tileSetNumber

-- Proceeds with a reset of a tile set that was fetched from the object
pool.
spawnedTile.TileSet:Init()
end

@ExecSpace("ClientOnly")
method void Reset()
-- This method resets the member elements needed for game progression.

-- Resets the number of created tiles to 0.
self.spawnedCount = 0
-- Resets the position of the 'tileParent' object, which is the parent of
all the tile set objects.
self.tileParent.TransformComponent.Position = Vector3(0,0,0)
-- Sets the background scrolling speed to the value that was set as a basis
in Config.
self:SetBackgroundScrollSpeed(_Config.backgroundScrollSpeed)

-- Creates a tile set in sequence from 1 to 5 when the game starts.

-- Executes a loop from 1 to 5
for i = 1, 5, 1 do
    -- and spawns tile index i.
    self:SpawnTile(i)
end
end

@ExecSpace("ClientOnly")
method void ChangeBackground()
-- This method changes the background.

-- Sets the background with the background RUID value that was saved in the

```

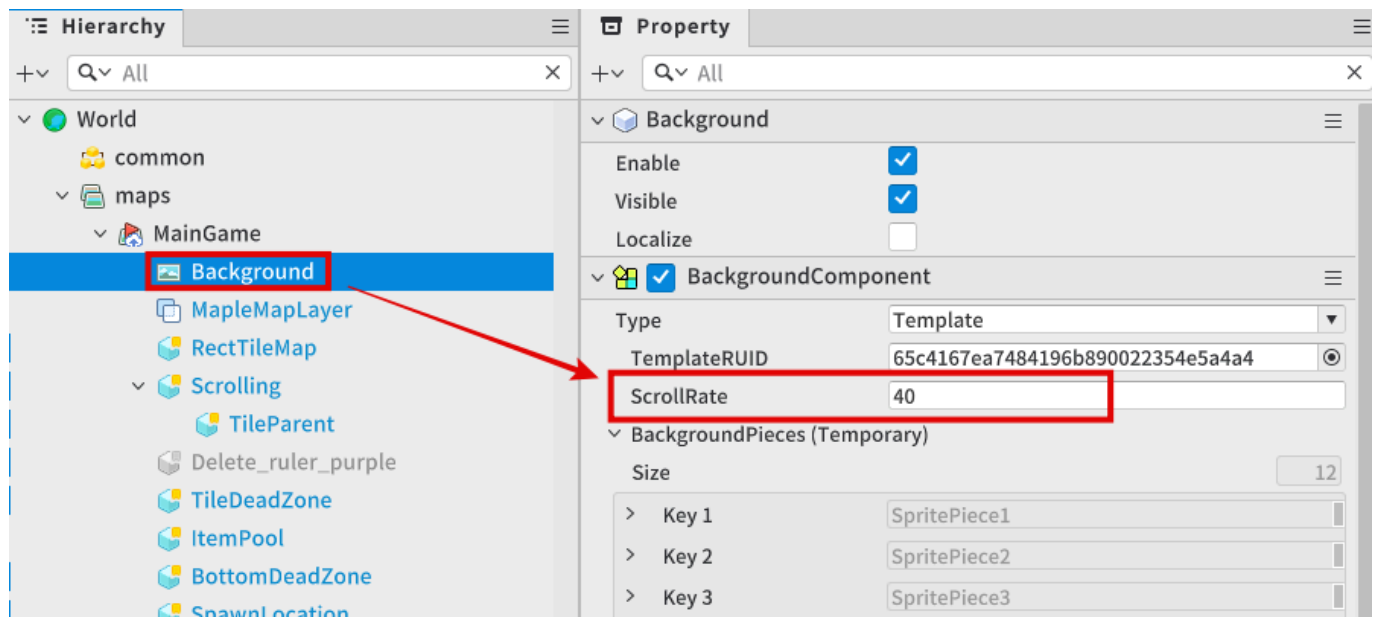
```

data set referencing the current stage number that's registered to the game
logic.
self.background:ChangeBackgroundByTemplateRUID(_DataSetLogic.stageRenderInf
oTable[_GameLogic.currentStage].BackgroundRUID)
end

end

```

BackgroundComponent's ScrollRate



- Access the Background entity to control the background image's scroll speed.
- Set the BackgroundComponent ScrollRate value.
- Even if the editor isn't in a play state, you can view directly from the Scene tab's screen.

Changing ScrollRate with Code

- The method to control the background scroll speed in real time while the game is running.
- References the SetBackgroundScrollSpeed method registered in the ScrollingController script.

```

void SetBackgroundScrollSpeed (number speed)
{
    self.background.ScrollRate = speed
}

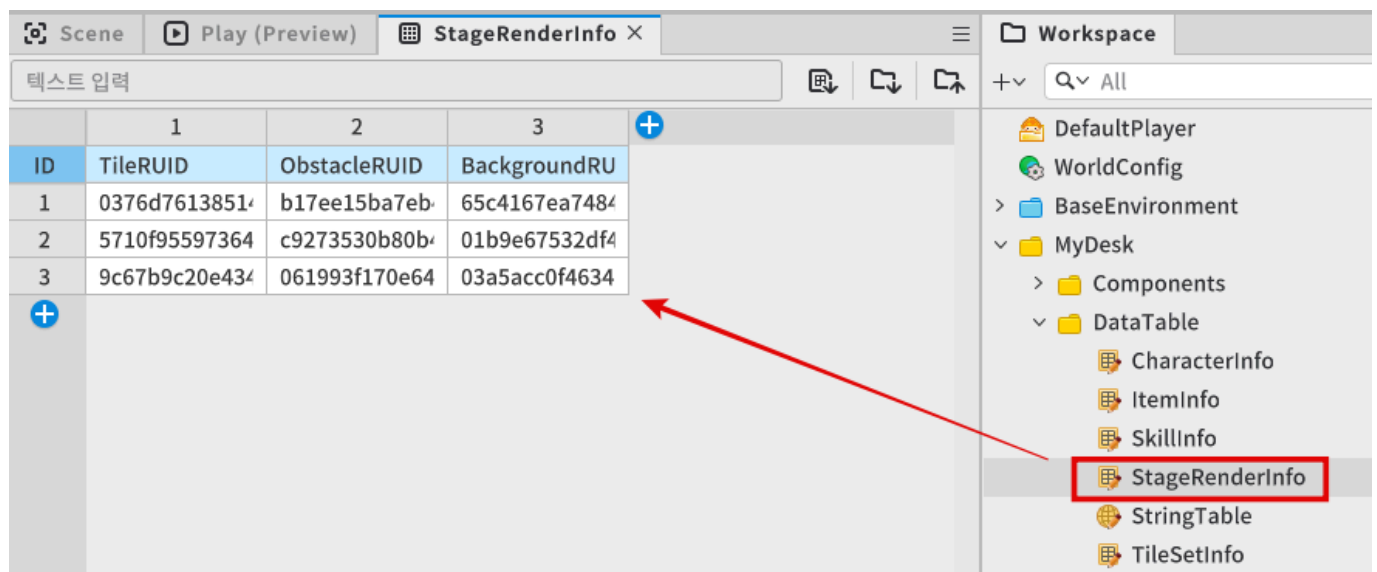
```

How to Change Stages in Real Time



- Set so that the background, floor tile, and the appearances of obstacles change once a set amount of time passes after the game starts.

Registering the Appearance DataSet that will be Applied to Each Stage



Name	Description
TileRUID	The resource ID value of the floor tile which interacts with the character.
ObstacleRUID	The resource ID value of the obstacle element that appears with the floor tile.
BackgroundRUID	The resource ID value that will be applied as the background with the backgroundComponent's templateRUID value.

- A DataSet is needed to save the RUID values that need to be applied to each stage.

- The sample world is divided in to stages 1, 2, and 3 like in the image, and the floor tile RUID, obstacle RUID, and background image RUID values are saved for each stage.

How to Change Stages

- The GameLogic's currentStage number type variable manages the data that manages the stage numbers.
- The stage change function is called from GameLogic's OnUpdate method after a set amount of time.
- The sample world only has a total of 3 registered stages, but it is created with a structure of lopping stages with 1,2,3 -> 1,2 using the % operator.
- Confirms the GameLogic's currentStage number and applies the RUID value that corresponds to the current number for floor tiles and obstacles when the entity is activated.
- The following is a portion of the GameLogic code.

```
GameLogic
number currentStage = 1 -- The variable that records the current stage
information.
number stageTimer = 0 -- Records how many seconds remain until the stage
changes.

void OnUpdate(number delta) -- The method that's called every frame.
{
    if(stageTimer > 0) then -- If there is time remaining in the timer,
        self.stageTimer -= delta -- The delta value is deducted from the
timer.

    else -- If there is no time left in the timer,
        self.StageChange() -- Calls the StageChange method.
        self.stageTimer = _Config.stageChagneTime -- Resets the timer to
the earlier time value.
}

void StageChange() -- The method that changes stages.
{
    -- Create with a structure that can continuously loop the stage numbers
in sequence of 1,2,3,1.
    self.currentStage = (self.currentStage % (totalStageCount+1))+1
    scrollingController:ChangeBackground()
}
```

Floor Tile Appearance Application Example

- When an entity is activated in the component that executes the floor tile function as provided in the sample world, the appearance is applied from within the Init method.
- Learn about the TileSet component script in the next chapter, [Manage the Map Through Procedural Creation Methods](#).

```

TileSet

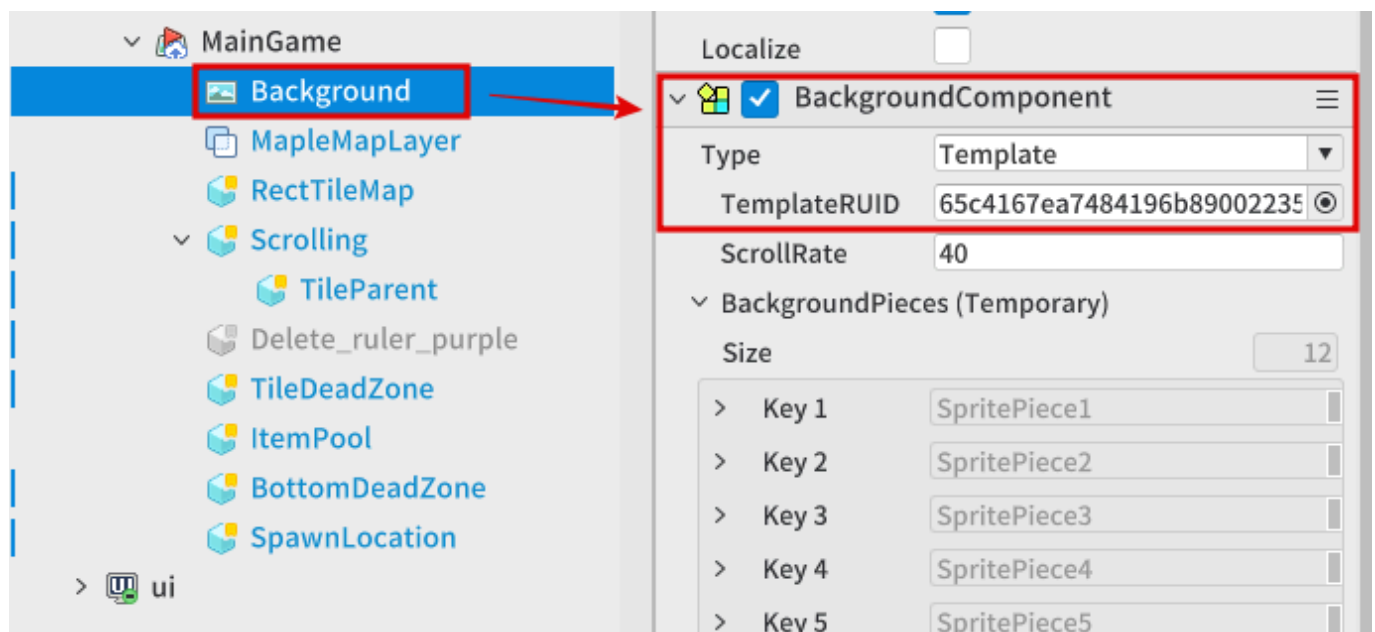
void Init()
{
    local renderers =

    self.Entity:GetChildComponentsByTypeName("SpriteRendererComponent", false)

    renderer.SpriteRUID =
        _DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].TileID
}

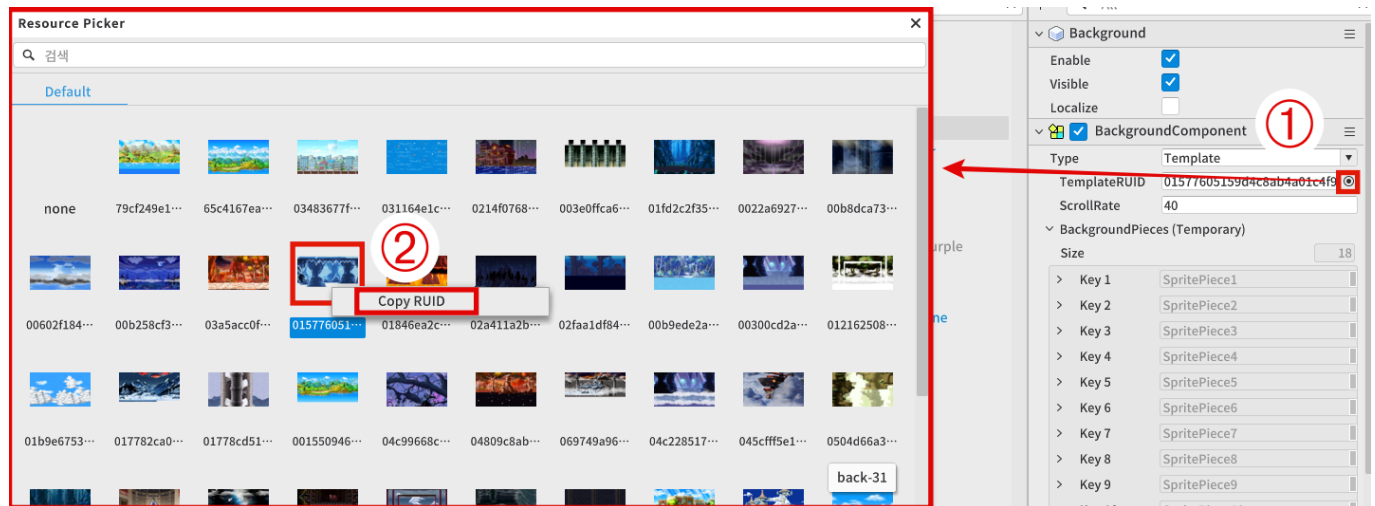
```

Changing Background Appearances



- The method provided from BackgroundComponent must be used to change the RUID of the BackgroundComponent, which manages the background image.
- Confirm the current Background Type.
- Use the BackgroundComponent's ChangeBackgroundByTemplateRUID method, since the Template type is currently in use.
 - MapleStory Worlds official document, BackgroundComponent [Link](#)

Confirming the backgroundTemplateRUID Value



1. Open the BackgroundComponent's RUID resource picker window to check the background's TemplateRUID value.
2. Extract the value by right-clicking the target resource and then clicking Copy RUID.

Changing backgroundTemplateRUID

- Change the background image through code from the ScrollingController component when GameLogic is calling the stage change function.
- Checks the registered background type and utilizes the RUID change method that's provided by the appropriate BackgroundComponent.

ScrollingController

BackgroundComponent background

```
void ChangeBackground()
{
    self.background.ChangeBackgroundByTemplateRUID(
        _DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].BackgroundRUID)
}
```

Minimum Implementation Standards

- Implement the basic floor tile creation and movement functions.

Usage and Extension Direction

- Add the RUID that manages the RUID for floor tiles, obstacles, and backgrounds to the DataSet to create a more varied stage.
- Adjust the stage change timer to change stages at the desired timing.

[💡 Basic Training] Manage the Map Through Procedural Creation Methods

Video Timeline

- You can follow along by using the timeline under the training video.
- Manage the Map Through Procedural Creation Methods: 00:33:49

Learning Goals

- Understand the concept of the procedurally-generated infinite maps and their practical need.
- Learn about dividing maps into set units of chunks and how to logically match the starting floor and ending floor of each chunk so that a natural map can be created.
- Create an infinite map through a script and implement a way to create a new floor tile entity on the floor tile spawn point.

Practical Exercises to Do After Watching the Video

- Add new tile information by adding data to the floor management table within DataSetLogic.
- Press the play button to confirm if a random tile set that fits the requirements is selected and spawned exactly at the X coordinate at the end.
- Test in-game to confirm that the heights of the platforms in the created map don't look awkward and that there are no gaps between tiles.

Understanding the Concepts of Procedural Map Generation

- Procedural map generation refers to a system that creates terrain in real time from beginning to end according to set rules and algorithms instead of placing them by hand.
- This is a core piece of technology that is needed to extend the map length indefinitely and have different terrain patterns each time for the infinite running game genre.

Overcoming the Limits of an Infinite Map

- Not only would creating a map in the editor that the player will be running across for a running platformer game mean an overwhelming workload, it would also place a massive burden on the system's memory.
- To resolve this issue, the 'procedural generation' technique is implemented so that new map pieces can be created in real time as the game play continues.
- Using this technique is like placing tracks in front of a train that's currently running, which functionally extends the map an infinite amount while also making management efficient.

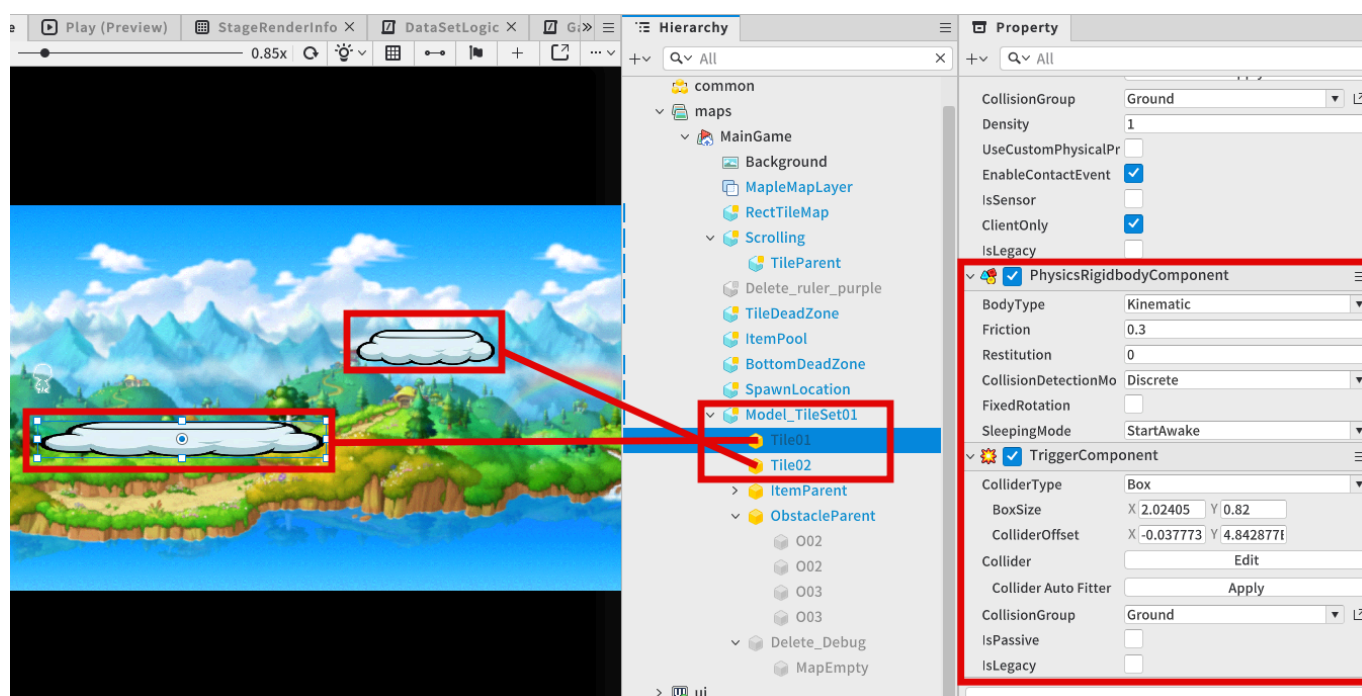
Management of Tile Set (Chunk) Units and Randomness

- Each pixel and tile isn't being randomly created for the map in real time. In actuality, map fragments in set lengths (chunks) will be created in advance and managed in various patterns.
- The script chooses one of the already created chunks at random in accordance with the rules and attaches it to the end of the previous chunk like one would a lego block.
- This allows for the map to be structurally stable while also giving the player a unique terrain type to adapt to each time they start a run, therefore increasing replayability.

Creating Map Chunks

- Chunks refer to a map fragment of a set size that was created in advance for the procedural generation system to use when connecting terrain.

Creating Floor Tile Entities

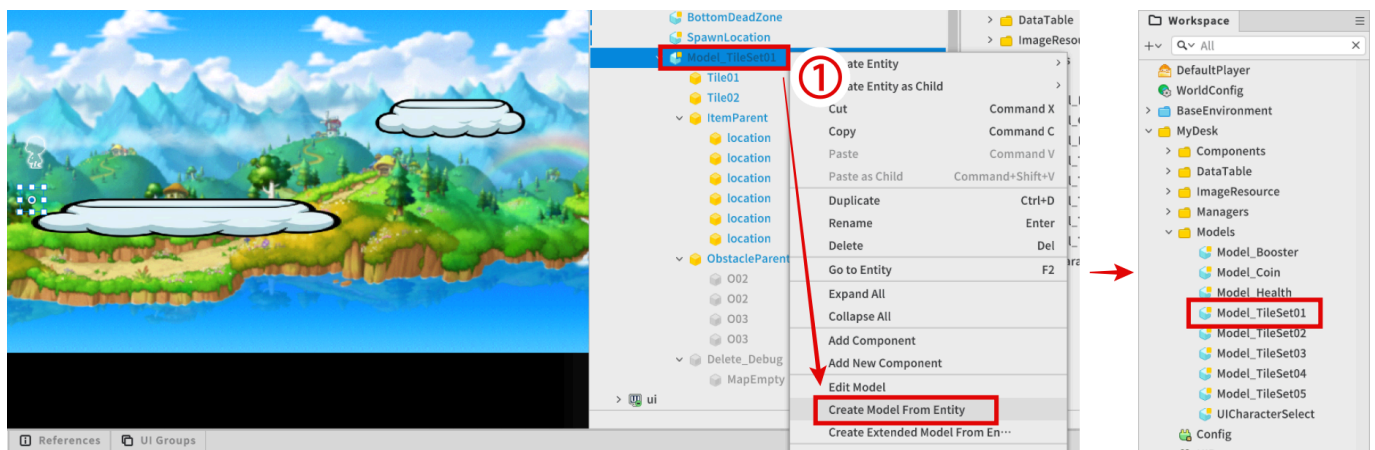


- Create an empty entity that will be the tile's parent in order to create a single chunk.
- The coordinates of the parent entity should be set as the starting point (0,0,0) to ensure a smooth working experience when placing tiles in the editor.
- Register multiple child entities and attach SpriteRendererComponents.
- Set the SpriteRendererComponent's RUID value to register their appearances.
- Register PhysicsRigidbody and PhysicsCollider components to each floor entity with the exception of the parent entity so that they can interact with the character.
- Change the collider's position and size through the collider edit button, which can be found in the PhysicsCollider component.



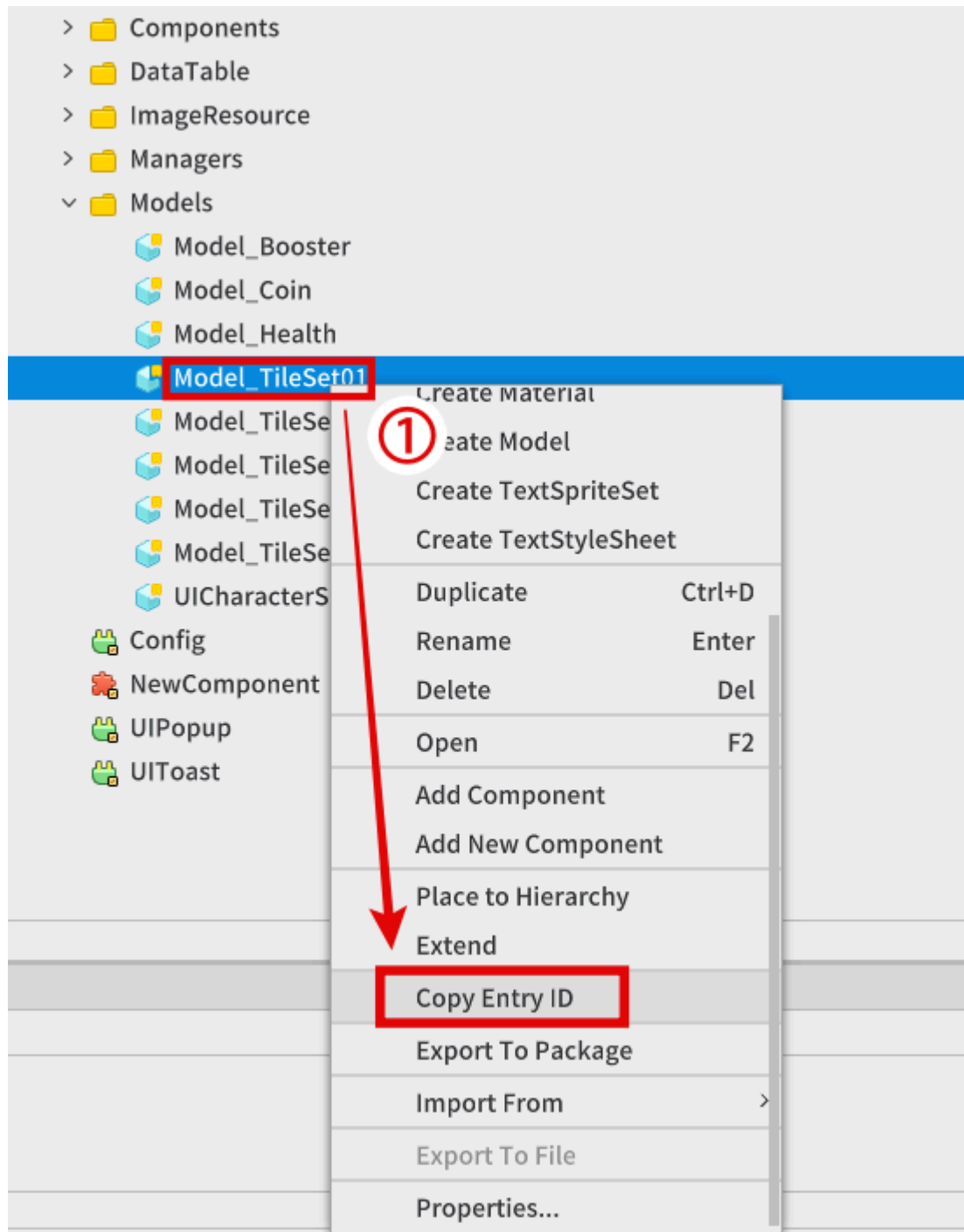
⚠ All chunks are created in the same position in set intervals, so if each chunk has different lengths or heights, it will make the map feel broken or incomplete. In the sample world, the x-axis interval for a single tile set is 10.

Registering Created Floor Tile Set Models



- If generated chunk maps are needed for creation calls, the entity needs to be registered as a model so it may be placed as an entity right away.
- Right-click the created tile set entity and then select Create Model From Entity.
- You can view that the target entity's model has been generated from Workspace.

Generating Created Models



- Since a single original floor tile has been created, it will be easy to generate that tile as many times as needed from the hierarchy.
- Right-click the generated Model from Workspace and select CopyEntry ID to copy the Entry ID value.
- Create the entity that is identical to the model in the world using the EntryID and SpawnService from the code.
 - MapleStory Worlds official document, SpawnService [Link](#)

Spawn Utilization Example

```
void Spawn()
{
```

```
    local entryID = "model://02e63fc1-cca0-4b98-ada3-d5a36e26f7de"
```



```

    _SpawnService:SpawnByModelId(entryID,"TempEntity",Vector3.zero,nil)
}

```

Registering the Starting Point and End Point Floor Number Information for Each Tile

	1	2	3
ID	FirstFloor	LastFloor	EntryID
1	1	2	model://02e63i
2	2	1	model://d1ca0
3	1	2	model://ec391v
4	2	1	model://d00b8
5	1	1	model://30079

- Create a data table to directly manage the starting and ending points for each tile.
- Each tile's FirstFloor and LastFloor information, which records the first starting point's floor number and ending point's floor number, and the EntryID value of each Model is registered in the table.

Registering Each Tile Data to the DataSet

①

ID	FirstFloor
1	1
2	2
3	1
4	2
5	1

②

TableName	FirstFloor	TilesNumber
tilesFloorInfoTable	[1]	1,3,5
	[2]	2,4

Red arrows indicate the mapping: ID 1 and 3 map to [1], ID 2 and 4 map to [2].

1. This is a portion of the actually saved DataSet(TileSetInfo) structure.
2. This graph displays how it's saved to the table from DataSetLogic.

- The sample world's floor tiles are made up as Fl. 1 and Fl. 2. So tileFloorInfoTable is also separated into 2 arrays, [1] and [2], so they can be independently managed and accessed.


```

-- DataSetLogic's tileFloorInfoTable Save Code

table tilesFloorInfoTable = {[1]={}, [2]={}}

void InitTileSetInfoTable()
{
    -- This method is needed to save the data set that's related to tile
    object.

    -- This variable is needed to access the name of the data set that is
    saved in world space as local.
    local dataSetName = "TileSetInfo"

    -- The number of Rows of the target data set.
    local rowCount = self:GetDataSetRowCount(dataSetName)

    -- Saves the number of tile set types.
    self.totalTileSetCount = rowCount

    -- A variable to hold the data for each line.
    local rowData

    -- Starts iterations a number of times equal to the number of data set
    Rows starting from 1.
    for i = 1 , rowCount , 1 do
        rowData =
        {
            -- The number of floors for the first tile of the tile set
            object.
            FirstFloor =
tonumber(_DataService:GetCell(dataSetName,i,"FirstFloor")),
            -- The number of floors for the last tile of the tile set
            object.
            LastFloor =
tonumber(_DataService:GetCell(dataSetName,i,"LastFloor")),
            -- The original entity model (prefab) Id needed to create a
            tile set object.
            EntryID = _DataService:GetCell(dataSetName,i,"EntryID")
        }

        -- Saves the data stored as rowData to a table.
        self.tileSetInfoTable[i] = rowData

        -- Saves the number of floors currently registered in the tile set
        as a variable.
        local firstTileFloor = self.tileSetInfoTable[i].FirstFloor

        -- /Branch 1/ If the first tile floor number is Fl. 1,
        if firstTileFloor == 1 then
            -- saves to the Fl. 1 table list.
            table.insert(self.tilesFloorInfoTable[1],i)
        -- /Branch 2/ If the first tile floor number isn't Fl. 1,

```

```

        else
            -- saves to the Fl. 2 table list.
            table.insert(self.tilesFloorInfoTable[2],i)
        end
    end
end
}

number GetDataSetRowCount(string Name)
{
    -- Returns the number of Rows of the target data set.

    -- Parameter 1 string Name: Receives the name of the target data set.

    -- Returns the number of Rows of the verified data set.
    return _DataService:GetRowCount(Name)
}

```

Utilizing Each Floor Tile Data Registered to DataSet

- The tilesFloorInfoTable table for each floor that was saved can be categorized as follows.
 - tilesFloorInfoTable[1]: The table containing the tile set numbers that start from Fl. 1
 - tilesFloorInfoTable[2]: The table containing the tile set numbers that start from Fl. 2
- Verifies the floor number of the end point for the floor tile that was generated last and extracts a random tile number from within the table for each floor.
- Updates the lastFloor value of the tile set that spawns last as a method to register the last tile's floor number.

```

-- Part of the method that generates the ScrollingController's tile

number lastTileFloor

void SpawnTile(number tileSetNumber)
{
    -- Receives the 'parameter' tile object from the object pool.
    local spawnedTile = _ObjectPoolLogic:GetTile(tileSetNumber)

    -- Records the floor of the end point for the tile set that was spawned
    from the object pool.
    self.lastTileFloor =
    _DataSetLogic.tileSetInfoTable[tileSetNumber].LastFloor
}

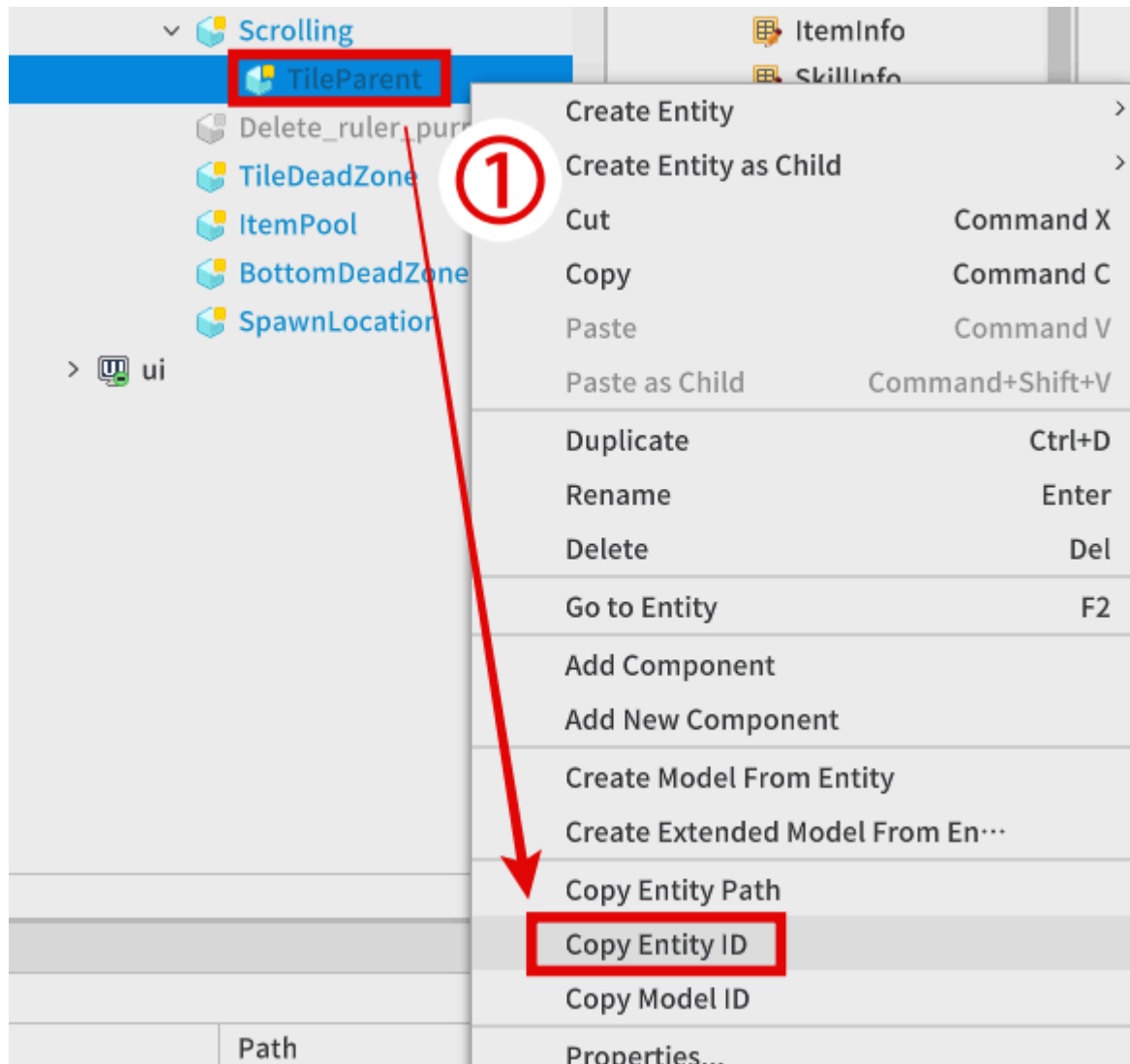
```

Principles of Moving Multiple Chunk Maps



- The TileParent of Scrolling's child entity that is only in charge of actual movement in the world.
- Each chunk map tile set entity doesn't execute movement individually; they are registered as the child entity of the TileParent that actually executes movement to make the movement automatic.
- In other words, the TileParent moves continuously in the -X-axis direction (from right to left) when the game starts and, when registered as a child of a TileParent entity, the child entity also moves automatically.

Registering as the TileParent Entity's Child



- You can load entities from the hierarchy through EntityService.
- Right-click the entity placed in the hierarchy and copy the ID value through Copy Entity ID in order to load the target entity's Entry ID.
- The SpawnService's SpawnByModelId method can set the Parent entity of the entity that will be generated with the 4th Parameter.
- Set the Parent when generating a new entity from the generated floor tile set model.

--Example use of SpawnService's SpawnByModelId method

```
void SpawnTile()
{
    // Load the EntryID from chunk map tile No. 2 from DataSetLogic
    local entryID = _DataSetLogic.tileSetInfoTable[2].EntryID

    // The name of the newly-generated entity
    local entityName = "TempEntity"

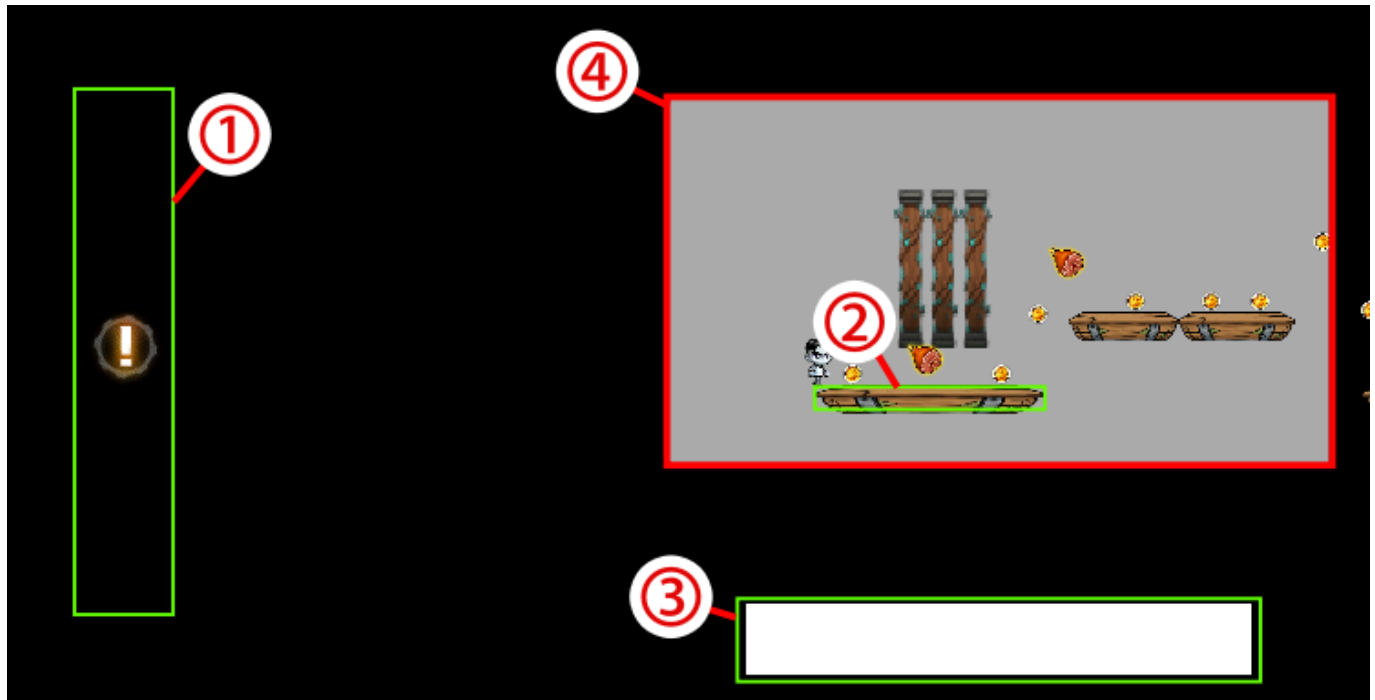
    // The coordinates that will be generated
    local spawnPosition = Vector3(50,0,0)

    // The parent entity of the entity that will be generated
    local tileParentEntity = _EntityService.GetEntity("d24b5917-1821-4743-
```

```
ae07-77cc9bfc2d0a")
```

```
_SpawnService:SpawnByModelId(entryID,entityName,spawnPosition,tileParentEntity)
}
```

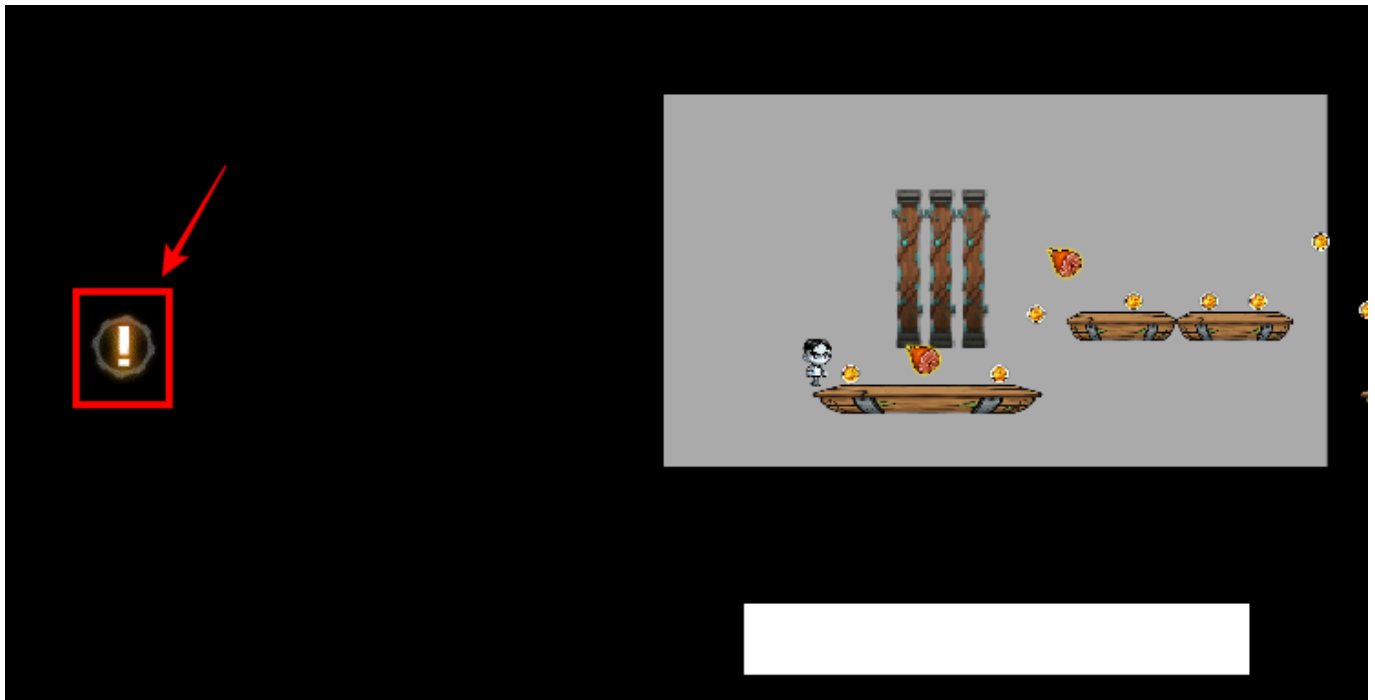
Removing a Tile Set That Leaves the Screen Area



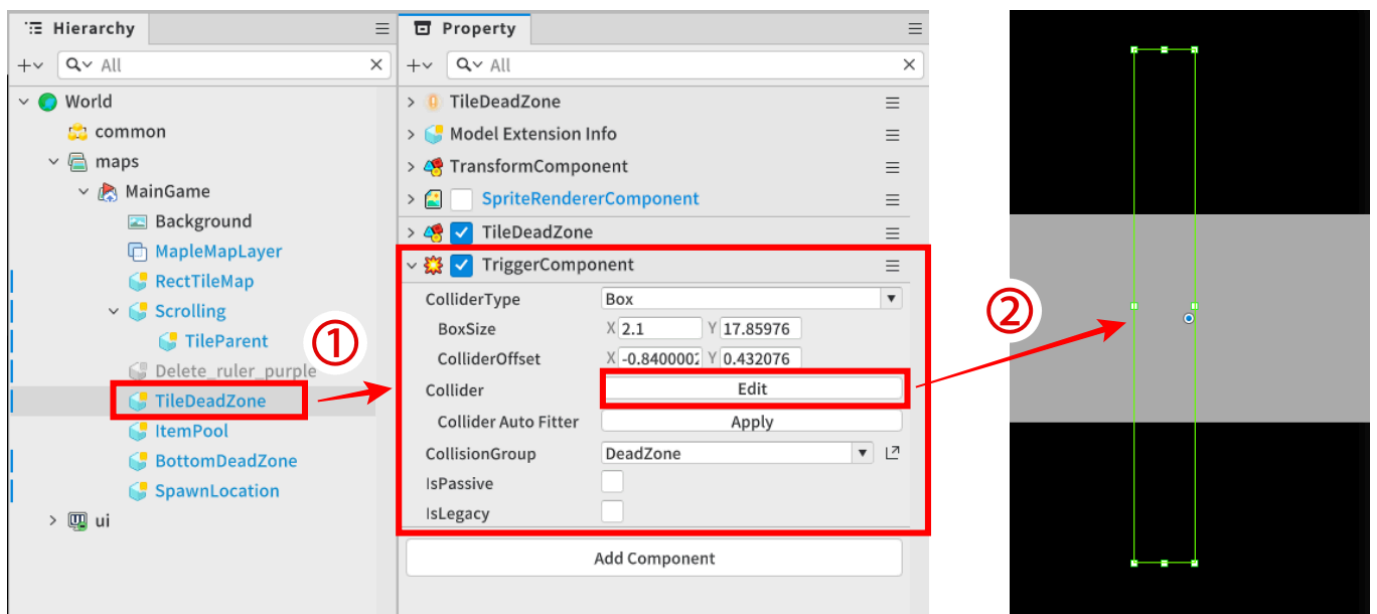
Category	Target	Description
1	DeadZone	Placed to detect a collision of the floor tile that moves to the left and to remove the target entity.
2	Tile Trigger	The Collider needed to interact with DeadZone.
3	DeadZone	The entity that needs to immediately end the game when the character falls below.
4	View Area	The camera area that's actually visible in a real Play environment.

- Tile entities that have completely gone beyond the left-side border of the screen no longer need to be rendered by the system.
- An entity that can detect a collision with the floor tile and remove the target tile set needs to be placed.

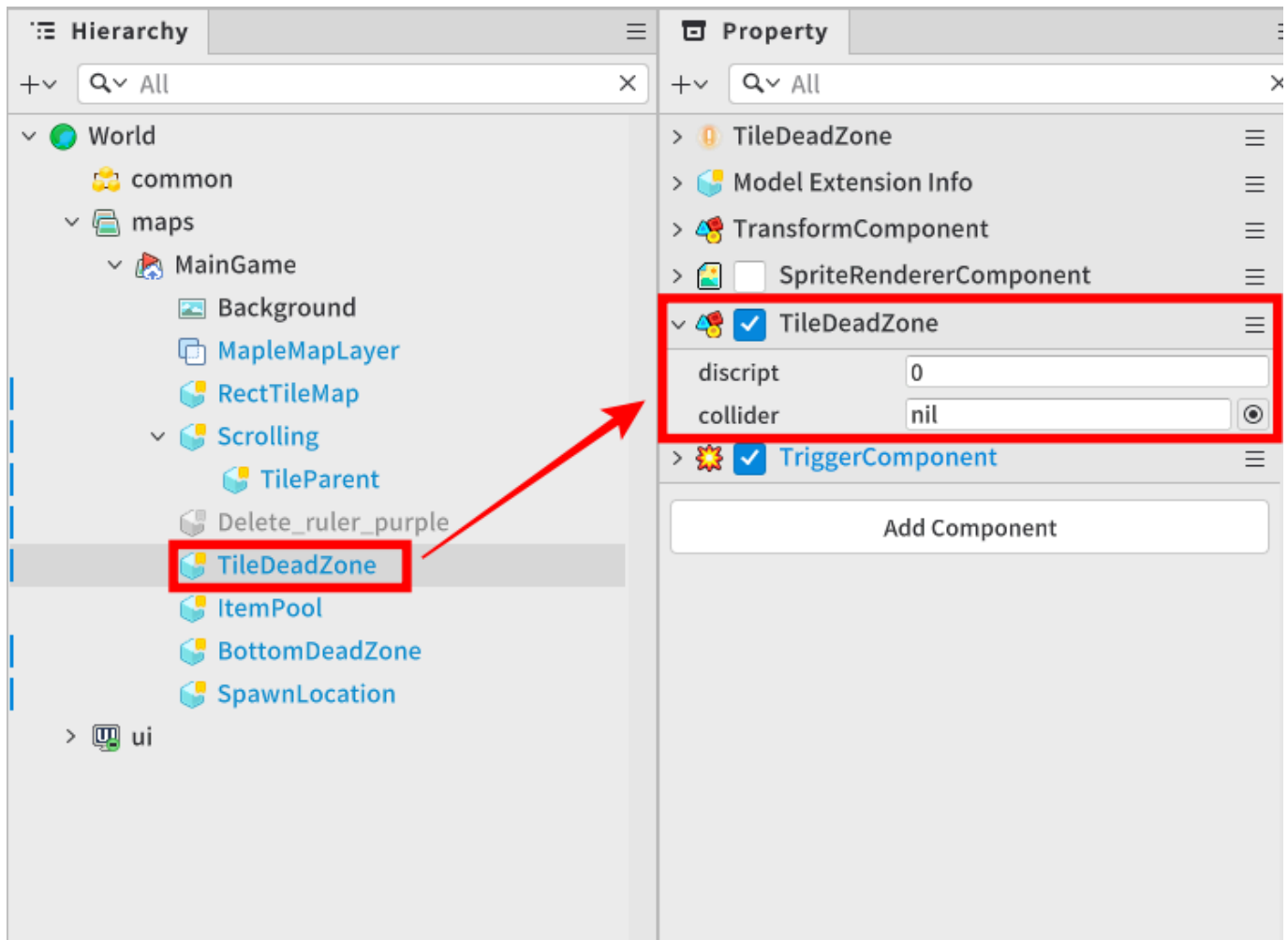
Creating a DeadZone That Interacts with Tile



- Create a new entity that's a far distance away from the camera area so that the removal of the floor tiles isn't visible to the player.



1. Register a TriggerComponent to the entity that will serve the role of TileDeadZone.
2. Set the collision detection range using the Collider's Edit button from the attached TriggerComponent.



1. Create a TileDeadZone component script and register it to TileDeadZone.
2. Register the code content using the TileDeadZone's Plain Text.

```
@Component
script TileDeadZone extends Component

property number discript = 0 -- The component that manages the destruction
(deactivation) of tile objects.

property PhysicsColliderComponent collider = nil

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- Links the references of the internally required members.

-- Links the PhysicsCollider component needed to detect colliding bodies.
self.collider = self.Entity.PhysicsColliderComponent
end

@ExecSpace("ClientOnly")
method void Init()
-- This method resets the TileDeadZone component.

-- Executes a reference link that was declared as a variable.
self:ConnectRefs()
```

```

end

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- This method is called when the world is launched.

-- Resets the target component.
self:Init()
end

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- This event is triggered when a trigger collision is detected.

-- Saves the entity of the trigger collision.
local TriggerBodyEntity = event.TriggerBodyEntity
-- Verifies the Collision group's id value of the collided trigger.
local TriggerGroup =
tostring(TriggerBodyEntity.TriggerComponent.CollisionGroup.Id)

-- If the returned string is 'Ground' upon inserting the above group id
value as the key value in the table register in Config,
-- in other words, if the colliding body that's detected as a trigger is
Ground,
if _Config.triggerGroup[TriggerGroup] == "Ground" then

    -- executes a tile set object return.
    _ObjectPoolLogic:ReturnTile(TriggerBodyEntity.Parent)

    -- Calls the scrolling controller for a random tile creation that will
connect to the last tile.
    _GameLogic.scrollingController:SpawnRandomTile()
end
end

end

```

Minimum Implementation Standards

- The starting floor number and end floor number matching data acts precisely and without any gaps between tiles.
- New map chunks are generated at the right time matching the scroll speed so that the character doesn't fall off the map due to a missing platform.

Usage and Extension Direction

- Register various floor tile models (Model) to expand or change the types in order to adjust the random generation weight so that higher difficulty map chunks appear more frequently.

[💡 Basic Training] Creating an Object Pool

Video Timeline

- You can follow along by using the timeline under the training video.
- Creating an Object Pool: 00:44:52

Learning Goals

- Understand why performance drops when entities (objects) are created and destroyed and then understand why the object pooling technique is critical in solving it.
- Be able to implement a material structure that can generate map chunks or obstacle entities that are needed when the game starts in advance and store them in a pool.
- Write a method that doesn't delete but rather deactivates entities that leave the screen, returns them to the pool, and re-sets their positions when needed again for future use.

Practical Exercises to Do After Watching the Video

- Write a code that spawns a number of tile set entities as set in the game initialization stage and then stores them as inactive using logic script.
- Test if a tile that scrolls to the left-most DeadZone doesn't get deleted but instead has its properties reset and returned to the object pool, and then enters a standby state.
- Confirm that the procedural map generation does not make brand new entities each time but rather takes out entities that are in standby in the object pool and attaches them to the end coordinates of the previous tile.

What is an Object Pool?

- Many entities are used when executing the game.
- Of those entities, bullets or platforms that are spawned frequently must be removed from the screen after a set time.
- However, spawning and destroying entities comes with taking up memory, and when those processes repeat over a short window of time, it could cause the computer to be overwhelmed (frame drops).
- However, if the activation is removed instead of creating and eliminating entities, it can lower the load on your computer.
- This is because the memory is used and reused like flipping a switch.
- In other words, the **Object Pool** is a Pool where created entities can be placed and pulled out for use before being returned to the pool, optimizing the normal creation and destruction process.

Creating an Object Pool

- The sample world's object pool is created as a central Manager where it can be accessed from all places like GameLogic or DataSetLogic, to easily rent out entities.

- Create a new logic in Workspace and write as an ObjectPoolLogic.

Writing ObjectPoolLogic Scripts

- Write the following code.
- The target code is based on Plain Text.

```
@Logic
script ObjectPoolLogic extends Logic

property number discript = 0 -- The logic used to effectively manage
objects that have frequent creation and destruction.

property table tilePool = {} -- The table that fulfills the role of a pool
to manage tile set objects.

property table itemPool = {} -- The table that fulfills the role of a pool
to manage item objects.

property Entity ItemPoolParent = "9e9e98bd-4f7b-46ed-9a93-f0e7d74fac1c"

property table itemDataSetIDTable = {"Coin" = 1 ,["Health"] = 2 ,
["Booster"] = 3} -- key : Item Name , value : The table meant to save the
Index number for the data set.

@ExecSpace("ClientOnly")
method void ReturnTile(Entity entity)
-- This method returns a tile object to the object pool.

-- Parameter 1 entity: Receives the object that will be returned
(deactivated).
-- Parameter 2 tileNumber: Receives the tile number.

-- Extracts the tile set entity's number. (This is to manage the tile set
objects by number in the table.)
local tileNumber = self.GetTileNumber(entity)

-- Checks whether the pool (table) that corresponds to the tile number is
valid.
if invalid(self.tilePool[tileNumber]) == false or self.tilePool[tileNumber]
== nil then
    -- Creates a new table space if there is no table.
    self.tilePool[tileNumber] = {}
end

-- Saves the target entity (object) to the target table.
table.insert(self.tilePool[tileNumber],entity)
-- Deactivates the entity (object).
entity.Enable = false
end

@ExecSpace("ClientOnly")
```

```
method Entity GetTile(number tileNumber)
-- This method returns the designated tile entity from the tile pool.

-- Parameter 1 number tileNumber: Receives a specified tile set number.

-- This is to discard the decimal point for tileNumber using the math.floor
method. ex) 4.0 => 4
tileNumber = math.floor(tileNumber)

-- Proceeds with checking the target pool (table).

-- If there is no pool (table) that manages the received tileNumber tile
objects,
if invalid(self.tilePool[tileNumber]) == false or self.tilePool[tileNumber]
== nil then
    -- a new table space is created.
    self.tilePool[tileNumber] = {}
end

-- /Branch 1/ Newly-created and returned if none exist.

-- If there are no objects that are contained in the target pool (a new
object creation takes place from the model (prefab)),
if #self.tilePool[tileNumber] <= 0 then

    -- checks the EntryID saved to the table set.
    local entryID = _DataSetLogic.tileSetInfoTable[tileNumber].EntryID
    -- Saves the name in the already saved (prefab) name format.
    local name = "Model_TileSet0"..tileNumber

    -- Creates a target model (prefab) object in a temporary position.
    local tempTile =
    _SpawnService:SpawnByModelId(entryID,name,Vector3.up*100,_EntityService:Get
    Entity("d24b5917-1821-4743-ae07-77cc9bfc2d0a"))

    -- Returns the target object.
    return tempTile
end

-- /Branch 2/ Returns the last element if one does exist.

-- View the number of target tables.
local tableCount = #self.tilePool[tileNumber]

-- Receives the last element returned from the target table and removes the
target entity from the table.
local tempTile = table.remove(self.tilePool[tileNumber],tableCount)

-- Activates the selected object.
tempTile.Enable = true

-- Returns the object.
return tempTile
```

```
end
```

```
@ExecSpace("ClientOnly")
```

```
method Entity GetItem(string itemName)
```

```
-- This method receives the returned item object from the object pool.
```

```
-- Parameter 1 string itemName: Receives the item name.
```

```
-- Loads the table that contains the relevant items from the item pool.
```

```
local targetTable = self.itemPool[itemName]
```

```
-- When the table has no internal elements,
```

```
if #targetTable == 0 then
```

```
-- proceeds with additional creations of the target item a number of  
times equal to the creation number set in config.
```

```
self.CreateItem(itemName,_Config.itemPoolInitCount)
```

```
end
```

```
-- Loads the entity from the intended table. (table's last element)
```

```
---@type Entity
```

```
local targetEntity = targetTable[#targetTable]
```

```
-- Removes the last element of the table, since the entity has been saved.
```

```
table.remove(targetTable,#targetTable)
```

```
-- Activates the entity (object).
```

```
targetEntity.Enable = true
```

```
-- Returns the saved entity.
```

```
return targetEntity
```

```
end
```

```
@ExecSpace("ClientOnly")
```

```
method void ReturnItem(Entity entity)
```

```
-- This method returns the item object to the object pool again.
```

```
-- Parameter 1 Entity entity: Delivers the item object entity that will be  
returned.
```

```
-- Confirms the name of the returning item.
```

```
local itemName = entity.Item.name
```

```
-- Slots entity into the target table.
```

```
table.insert(self.itemPool[itemName],entity)
```

```
-- From the tile set's existing Parent (an object that was preregistered to  
place the tile set item),
```

```
-- moves the Parent to ItemPool again.
```

```
entity:AttachTo(self.ItemPoolParent)
```

```
-- Deactivates the entity (object).
```

```
entity.Enable = false
```

```
end
```

```
@ExecSpace("ClientOnly")
method void CreateItem(string itemName, number createCount)
-- This method creates a new item object from the model (prefab) and places
it into the target pool.

-- Parameter 1 string itemName: Receives the name of the item that will be
created.
-- Parameter 2 number createCount: Determines the amount to be created and
how many objects will be created and placed inside the object pool.

-- Declares a temporary table.
local tempTable

-- Sets the Id number that will be referenced by the data set.
local dataSetID = self.itemDataSetIDTable[itemName]

-- Loads the EntryID value of the target item model from the data set.
local entryID = _DataSetLogic.itemInfoTable[dataSetID].EntryID
-- Loads the Name value of the target item model from the data set.
local name = _DataSetLogic.itemInfoTable[dataSetID].Name
-- Loads the Value value of the target item model from the data set.
local value = _DataSetLogic.itemInfoTable[dataSetID].Value

-- Determines if the target table is valid.

-- /Branch 1/ If no target pool (table) exists within the target item pool,
if isValid(self.itemPool[name]) == false or self.itemPool[name] == nil then
    -- a new temporary table is created.
    tempTable = {}
    -- Sets the internal pool (table) from the item pool as the temporary
table.
    self.itemPool[name] = tempTable

-- /Branch 2/ If the target pool is already in the target item pool,
else
    -- the temporary table (pool) will be declared as a table that was
previously in use.
    tempTable = self.itemPool[name]
end

-- Performs a loop as many times as parameter2 createCount.
for i = 1 , createCount, 1 do
    -- Utilizes SpawnService to create the object.
    local entity = _SpawnService.SpawnByModelId(entryID,name,Vector3.up *
100, self.ItemPoolParent)

    -- Proceeds with configuring the created object.

    -- Sets the item's unique value of the created object.
    entity.Item.value = value
    -- Sets the name variable of the item of the created object.
    entity.Item.name = name

    -- Inserts the entity created for the target table (pool).
```

```

        table.insert(tempTable,entity)

        -- Disables the created entity object.
        entity.Enable = false
    end
end

@ExecSpace("ClientOnly")
method void InitItemPool()
    -- This method resets the item pool.

    -- Loads the table that contains the item data set from DataSetLogic.
    local itemDataTable = _DataSetLogic.itemInfoTable

    -- Creates an amount of the Coin item equal to the creation count set in
    Config.
    self:CreateItem("Coin",_Config.itemPoolInitCount)
    -- Creates an amount of the Health item equal to the creation count set in
    Config.
    self:CreateItem("Health",_Config.itemPoolInitCount)
    -- Creates an amount of the Booster item equal to the creation count set in
    Config.
    self:CreateItem("Booster",_Config.itemPoolInitCount)
end

@ExecSpace("ClientOnly")
method void Init()
    -- This method proceeds with resetting ObjectPool Logic.

    -- Proceeds with resetting the item pool.
    self:InitItemPool()
end

@ExecSpace("ClientOnly")
method number GetTileNumber(Entity entity)
    -- This method extracts the tile number.

    -- Parameter 1 Entity entity: Receives the target entity.

    -- Extracts the entity's name.
    local tileSetName = entity.Name
    -- Extracts the last string of the name. (Extract 14th, 15th character from
    the 'Model_TileSet02' string > 02)
    local numberStr = string.sub(tileSetName,14,15)
    -- Converts strings to number values and returns them.
    return tonumber(numberStr)
end

end

```

- ObjectPoolLogic manages the floor tile sets and each item element that's regularly generated and destroyed.

- The created GetTile serves the role of bringing out floor tiles that are in standby in the tile pool and is called when the ScrollingController component spawns a new map chunk.
- When a floor tile that leaves the screen collides with the DeadZone, the ReturnTile method is used by the TileDeadZone script to return the target tile to the pool (deactivate) instead of destroying them.

Minimum Implementation Standards

- Implemented so that the tile set entities aren't repeatedly generated and destroyed while a map is scrolling, but instead create a cycle of deactivation and reactivation through the object pool system.
- Implemented in a form so that all tile sets are created in the object pool or returning usable entities.

Usage and Extension Direction

- Edit the code so that entities are **created in advance** and used as one of the principles of an object pool.
- Confirm that not only the floor tiles but also the obstacles and item elements that frequently appear during the game are reused as intended through the object pool.
- The goal isn't to create a large pool from the start, but to flexibly increase the size of the pool (additional generation) when there aren't enough standby entities in the object pool and to implement an expansion logic that further optimizes memory usage.

[💡 Basic Training] Implementing Player Characters

Video Timeline

- You can follow along by using the timeline under the training video.
- Implementing Player Characters: 00:57:50

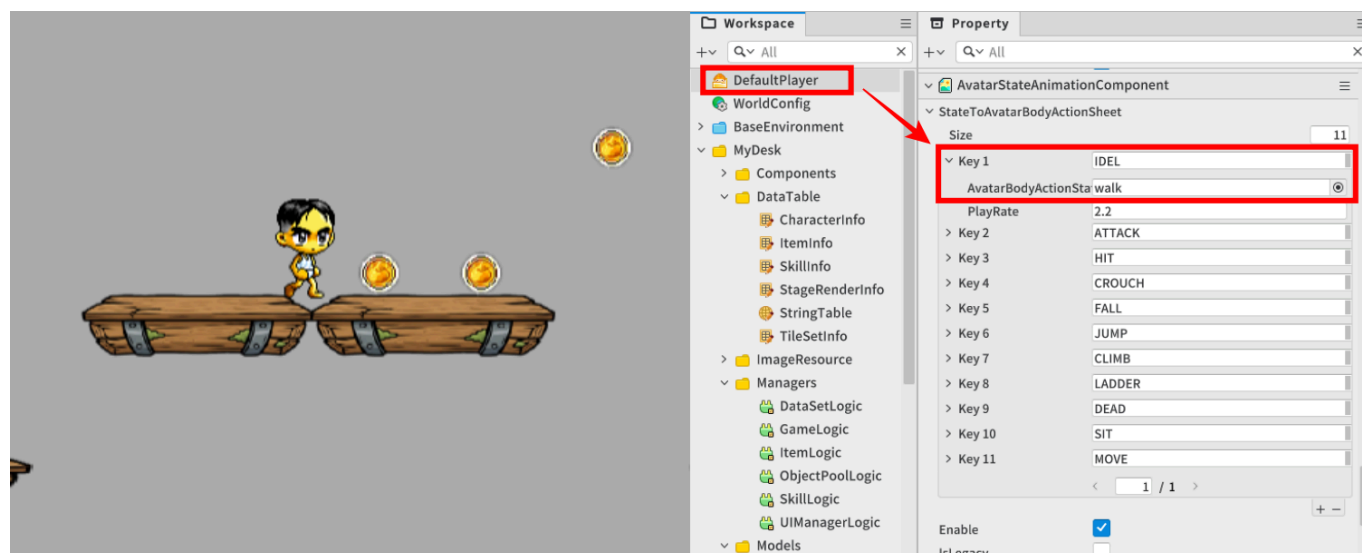
Learning Goals

- You can implement a jump function that detects keyboard input and applies a physical force to the character.
- Learn how to create various characters.
- Define the available skills for each character and learn how to implement their effects when using the skills.
- You can give your players an environment that allows them to select a variety of characters.

Practical Exercises to Do After Watching the Video

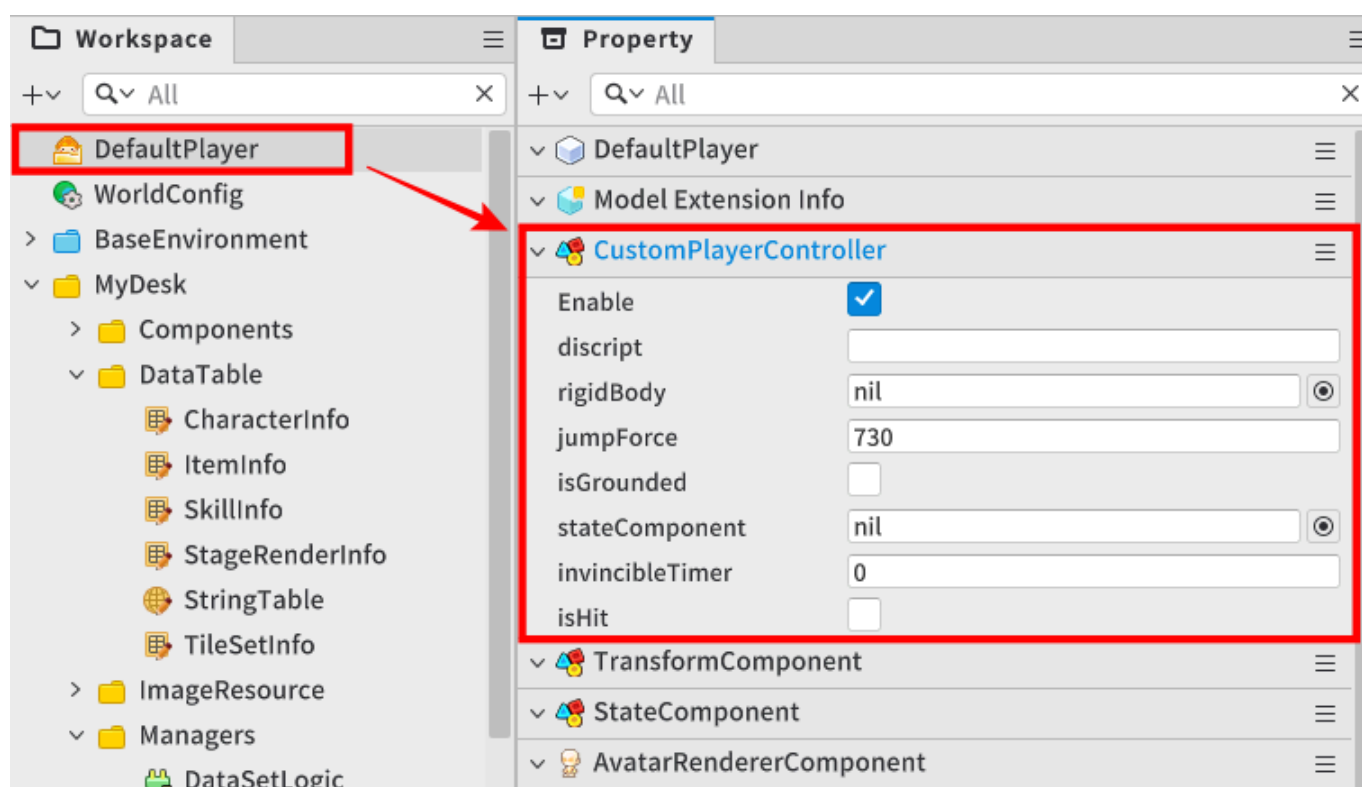
- Try setting up your skills to be triggered by inputs other than the skill keys provided in the sample world.
- Try changing the force of your character's jump to change the physical appearance you want.
- Add a new character to the CharacterInfo DataSet and verify that you can play as the new character in the Character Selection window.

Setting Character Default State



- The game is running with the character in the sample world not actually moving, so by default, the running animation should play instead of the standing animation.
- In the AvatarStateAnimationComponent of the DefaultPlayer's component that is the character's model, change the animation that plays for the standing state (IDLE) by default to a (walk) animation.

Controlling Character Functions



- Create and attach a CustomPlayerController component script to control various features of the DefaultPlayer, which is the character's model, directly in code.

Writing a CustomPlayer Script

- Write the following code.
- The target code is based on Plain Text.

```
@Component
script CustomPlayerController extends Component

property string discript = "" -- The component needed to define a
character's function and state.

property PhysicsRigidbodyComponent rigidBody = nil

property number jumpForce = 730 -- The strength value that's needed for a
character jumping.

property boolean isGrounded = false -- The variable that checks if the
character is currently on the ground.

property StateComponent stateComponent = nil

property number invincibleTimer = 0 -- The timer variable needed to control
the invincibility duration after colliding with an obstacle.

property boolean isHit = false -- The variable that checks if the character
is currently hit by an obstacle.

@ExecSpace("ClientOnly")
method void Init()
-- This method resets the CustomPlayerController controller.

-- Executes an internal variable reference link.
self:ConnectRefs()

-- Proceeds with the default state as MOVE.
self:ChangeState("MOVE")

end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- This method reference links the members that are internally required.

-- Links the Rigidbody component that's attached to the character.
self.rigidBody = self.Entity.PhysicsRigidbodyComponent
-- Links the State component that's attached to the character.
self.stateComponent = self.Entity.StateComponent
end
```

```

@ExecSpace("ClientOnly")
method void Jump()
-- This method executes the character's jump animation.

-- If the floor detection fails,
if self.isGrounded == false then
    -- it halts the jump execution.
    return
end

-- Sets the speed value applied on the character's Y-axis to 0 and removes
the Y-axis activity.
self.rigidBody:SetLinearVelocityY(0)

-- Applies the Force value through rigidbody in the Y-axis direction.
self.rigidBody:ApplyForce(Vector2.up * self.jumpForce)

-- When not in a prone state while executing the jump logic,
if self.stateComponent.CurrentStateName ~= "CROUCH" then
    -- converts into the jump state.
    self:ChangeState("JUMP")
end

_SoundService:PlaySound("24ee4f2425924a1abd4f655e8faf2791",1)

-- Sets that the character isn't on the floor, since the jump was executed.
self.isGrounded = false
end

@ExecSpace("ClientOnly")
method void ChangeState(string state)
-- This method changes the character's state as the state that was
received. (This is to control a character's animation.)

-- Parameter 1 string state: Receives the state that will be changed into
string format.

-- Sets the state with the state component that was declared as a member.
self.stateComponent:ChangeState(state)

-- Trigger Collider Box Size Adjustment: This is needed to categorize the
collider state for when the character is prone and when the character is
standing.

-- If the state delivered from a parameter value is CROUCH,
if state == "CROUCH" then
    -- The trigger Box size of the Trigger component that's included in its
own (character) entity is set to how it is in Config.
    self.Entity.TriggerComponent.BoxSize = _Config.crouchTriggerSize
    -- Sets the offset value that indicates how far the Trigger component's
trigger Box needs to be from the Pivot (base).
    self.Entity.TriggerComponent.BoxOffset = _Config.crouchTriggerOffset

-- If the state delivered from a parameter value is MOVE,

```

```
elseif state == "MOVE" then
    -- The trigger Box size of the Trigger component that's included in its
    own (character) entity is set to how it is in Config.
    self.Entity.TriggerComponent.BoxSize = _Config.standTriggerSize
    -- Sets the offset value that indicates how far the Trigger component's
    trigger Box needs to be from the Pivot (base).
    self.Entity.TriggerComponent.BoxOffset = _Config.standTriggerOffset
end
end

@ExecSpace("ClientOnly")
method void ContactGround()
    -- This method is executed when the floor is detected.

    -- /Branch 1/ If already touching the floor,
    if self.isGrounded == true then
        -- nothing is executed and the method is ended.
        return
    end

    -- /Branch 2/ Upon touching the floor after previously not being on the
    floor,

    -- changes the current state as being on the floor.
    self.isGrounded = true

    -- Plays the move animation upon detecting the floor.
    self:ChangeState("MOVE")
end

@ExecSpace("ClientOnly")
method void ContactItem(Item item)
    -- Proceeds with the actions that need to be executed for each item.

    -- Parameter 1 Item item: Receives the Item component that was included in
    the item object.

    -- Plays a sound effect upon obtaining an item.
    _SoundService:PlaySound("ef4235b158414f5283c068bad71f403a",1)

    -- If the name variable of the received item component is Coin,
    if item.name == "Coin" then
        -- executes the game logic's score addition method and delivers the
        Value value of the received item.
        _GameLogic:AddScore(item.value)
    end

    -- If the name variable of the received item component is Health,
    if item.name == "Health" then
        -- executes the game logic's HP addition method and delivers the Value
        value of the received item.
        _GameLogic:AddHP(item.value)
    end
end
```

```

-- If the name variable of the received item component is Booster,
if item.name == "Booster" then
    -- calls the booster item usage method from item logic.
    _ItemLogic:UseBooster()
end

-- Moves a far distance upon obtaining the item.
-- (Moves to a location where the item object position is temporarily not
visible, since there was an issue with recreating from the object pool.)
item.Entity.TransformComponent.Position = Vector3.one * 100

-- Executes a method that returns the item object to the object pool.
_ObjectPoolLogic:ReturnItem(item.Entity)
end

@ExecSpace("ClientOnly")
method void ContactObstacle()
-- This method is executed upon colliding with an obstacle.

-- If there is still some invincibility time left,
if self.invincibleTimer > 0 then
    -- it ends without doing anything.
    return
end

-- Delivers the obstacle collision damage defined in Config to the game
log's HP addition method.
_GameLogic:AddHP(_Config.obstacleDamage)

-- Sets the state to being hit.
self.isHit = true

-- Sets the invincibility duration timer to the default value set in
Config.
self.invincibleTimer = _Config.invincibleTime

-- Plays the take hit effect.
self:PlayOnDamagedEffect()

-- Sets the avatar renderer's transparency. (To deliver the being hit
state.)
self.Entity.AvatarRendererComponent.SetAlpha(0.5)
end

@ExecSpace("ClientOnly")
method void PlayOnDamagedEffect()
-- This method plays the effect that's triggered when being hit.

-- Saves the being hit effect resource ID value.
local effectRUID = "41e13add2ba746d9bbe624a31205a848"
-- Loads the map entity.
local mapEntity = _EntityService:GetEntityByPath("/maps/MainGame")
-- Sets the effect trigger point.
local position = self.Entity.TransformComponent.Position

```

```

-- Plays the effect using the effect service.
_EffectService:PlayEffect(effectRUID,mapEntity,position,0,Vector3.one * 3)
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- This method is called for each frame.

-- While being hit,
if self.isHit == true then
    -- if there is invincibility time left,
    if self.invincibleTimer > 0 then
        -- the duration of the being hit timer is reduced.
        self.invincibleTimer -= delta
    -- If there is no time left in the being hit timer,
    else
        -- changes to being hit removed state.
        self.isHit = false
        -- Resets the avatar renderer's transparency value to 1.
        self.Entity.AvatarRendererComponent:SetAlpha(1)
    end
end
end

@ExecSpace("ClientOnly")
method void ResetCharacter()
-- This method proceeds with setting the default values when restarting the
game.

-- Sets the target entity's position values to absolute coordinates x = 0,
y = 1.
self.Entity.TransformComponent.Position = Vector3(0,1,0)
-- Sets the Rigidbody's BodyType to Dynamic, so it can be flexible.
self.rigidbody.BodyType = BodyType.Dynamic

-- Sets the character as visible.
self.Entity.Visible = true
end

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- This method is called when the world is launched.

-- Resets the target component.
self:Init()
end

@EventSender("Service", "InputService")
handler HandleKeyDownEvent(KeyDownEvent event)
-- This method is called when the user makes a key input.

-- If the game is in the end state,

```

```
if _GameLogic.isEnd == true then
-- This method is halted.
    return
end

-- Saves the input key information.
local key = event.key

-- When the space bar is input,
if key == KeyboardKey.Space then
    -- executes the jump animation.
    self:Jump()
end

-- When the down arrow key is input,
if key == KeyboardKey.DownArrow then
    -- sets the character state as 'CROUCH'.
    self:ChangeState("CROUCH")
end

-- When the 'K' key is input,
if key == KeyboardKey.K then
    -- the character skill is used.
    _SkillLogic:UseSkill()
end

end

@EventSender("Service", "InputService")
handler HandleKeyUpEvent(KeyUpEvent event)
-- This method is called when the user's key input ends.

-- If the game is in the end state,
if _GameLogic.isEnd == true then
-- This method is halted.
    return
end

-- Saves the input key information.
local key = event.key
-----

-- When the down arrow key is no longer input,
if key == KeyboardKey.DownArrow then
    -- Crouch state is removed.
    self:ChangeState("MOVE")
end
end

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- This method is called when the character's trigger box collides with
another trigger.
```

```

-- If the game is in the end state,
if _GameLogic.isEnd == true then
-- This method is halted.
    return
end

-- Saves the target entity of the trigger collision.
local TriggerBodyEntity = event.TriggerBodyEntity
-- Saves the group name of the collided trigger.
local TriggerGroup =
tostring(TriggerBodyEntity.TriggerComponent.CollisionGroup.Id)
-----

-- If the target trigger is the trigger of an item object,
if _Config.triggerGroup[TriggerGroup] == "Item" then
    -- the logic for when an item is detected is executed.
    self:ContactItem(TriggerBodyEntity.Item)
end

-- If the target trigger is an obstacle element object's trigger,
if _Config.triggerGroup[TriggerGroup] == "Obstacle" then
    -- the logic for when an obstacle is detected is executed.
    self:ContactObstacle()
end

-- If the target trigger is a DeadZone element object's trigger,
if _Config.triggerGroup[TriggerGroup] == "DeadZone" then

    -- since the condition which the current character meets the dead zone
    is when the character falls below the screen,
    -- the end game command logic is required.

    -- Sets the Rigidbody's BodyType to Static so no physics-based
    activities take place.
    self.rigidbody.BodyType = BodyType.Static
    -- Executes the game logic's end game method.
    _GameLogic:EndMainGame()
end
end

@EventSender("Self")
handler HandlePhysicsContactBeginEvent(PhysicsContactBeginEvent event)
-- This event is triggered when a physics collision is detected with a
Physics rigid body.

-- Loads the entity of the collided rigid body.
local ContactedBodyEntity = event.ContactedBodyEntity

-- Loads the TileSet component of the parent object of the collided rigid
body.
local tileSet = ContactedBodyEntity.Parent.TileSet

-- When a tile set is detected from the object that made contact,

```

```

if tileSet ~= nil then

    -- the Transform component of the object that made contact is loaded.
    local contactTransform = ContactedBodyEntity.TransformComponent
    -- Loads the character's Transform component.
    local playerTransform = self.Entity.TransformComponent

    -- If the colliding target's body object position has a Y-axis value
    that's higher than the character's Y-axis value,
    if contactTransform.Position.y >= playerTransform.Position.y then
        -- This method is halted.
        return
    end

    -- delivers that it's grounded.
    self.isGrounded = true
    -- If the current state is 'JUMP',
    if self.stateComponent.CurrentStateName == "JUMP" then
        -- converts to the 'MOVE' state.
        self:ChangeState("MOVE")
    end
end
end

end

```

Understanding the Jump Function Execution Process

- One way to utilize jumping in a set environment is to utilize the physical functions of the PhysicsRigidbody registered to the character.
- To control the physics of the character within the CustomPlayerController component, you declare a variable of PhysicsRigidbodyComponent type and associate a component reference to it.
- To apply an upward force to an entity, utilize the ApplyForce method of the PhysicsRigidbody.
- Have the character play a jump animation to indicate that they are jumping.

Example of Implementing Character Jumping

```

PhysicsRigidbodyComponent rigidBody;
StateComponent state;

number jumpForce = 730
boolean isGrounded = false

void OnBeginPlay()
{
    -- Connect PhysicsRigidbodyComponent to apply the physical force.
    self.rigidBody = self.Entity.PhysicsRigidbodyComponent
    -- Connect StateComponent for the character's state transitions.
    self.state = self.Entity.StateComponent
}

```



```
void Jump()
{
    if self.isGround == false then
        return -- If the character is not on the ground, exit without
        performing the jump function.
    end

    -- At the beginning of the jump performance, the character's Y-axis
    velocity and force are canceled out.
    self.rigidBody:SetLinearVelocityY(0)

    -- Apply a force equal to jumpForce in the upward direction of the
    Vector2.up character.
    self.rigidBody:ApplyForce(Vector2.up * self.jumpForce)

    -- Ask the StateComponent to change to the Jump state to apply jump
    animation.
    self.state:ChangeState("Jump")

    -- Performs a jump, so the character switches state to not currently on
    the ground.
    self.isGround = false;
}

HandleKeyDownEvent(KeyDownEvent event)
{
    -- A method that is called when a key on the keyboard is pressed.

    -- Saves the input key information.
    local key = event.key

    -- If the key entered is the Space key
    if key == KeyboardKey.Space then
        -- Performs the Jump method.
        self:Jump()
    end
}
```

How Character State Transitions upon Floor Detection

- The boolean isGround state variable is utilized to prevent the character from performing consecutive jumps.
- Toggle the variable to false when the character starts jumping.
- When the jump is complete and gravity has brought the character to the ground, isGrounded state variable needs to be transitioned back to true to restore the jumpable state.
- State transitions are possible through PhysicsCollider and EventHandler.

Example Code for Floor Collision State Transition

StateComponent state

```
void OnBeginPlay()
{
    -- Connect StateComponent for the character's state transitions.
    state = self.Entity.StateComponent
}

HandlePhysicsContactBeginEvent(PhysicsContactBeginEvent event)
{
    -- A method that is called when a collision with another collider is
    detected by the PhysicsCollider.

    -- Save the entity for the collider target.
    local ContactedBodyEntity = event.ContactedBodyEntity
    -- Retrieve the TileSet component from the collider target.
    local tileSet = ContactedBodyEntity.TileSet

    -- If you've checked the TileSet component (checked it's a floor tile):
    if tileSet ~= nil then
        -- Toggle the isGround state to true.
        isGround = true
        -- Switch the animation from a jumping state to a walking animation
again.
        self.state.ChangeState("Move")
    end
}
```

Creating Different Characters

Create Information Tables per Character

	1	2	3	4	+
ID	Name	Speed	SkillId	RUID	
1	TestA	3	1	ac753a90b9814	
2	TestB	4	2	b65a754c25174	
+					

- DefaultPlayer
- WorldConfig
- > BaseEnvironment
- MyDesk
 - > Components
 - DataTable
 - CharacterInfo**
 - ItemInfo
 - SkillInfo
 - StageRenderInfo
 - StringTable
 - TileSetInfo
 - > ImageResource
 - Managers
 - DataSetLogic

Target	Description
Name	The character name that will be given to the player.
Speed	This value is added to set different scrolling speeds for different characters.
Skill id	A skill number that is unique to each character.
RUID	The resource ID value that will be registered to the BodyParts of the character's registered CostumeManagerComponent.

- The created CharacterInfo DataSet will store each character's information as a table using the InitCharacterInfoTable method in DataSetLogic.

Registering Skill Data

ID	1	2	3	4
Name	Value	EffectRUID	UI_RUID	
1	Heal	20	db4e6e7fcb1b4	c99e612dbd54
2	Bonus	500	db4e6e7fcb1b4	eb0b26a38476

Target	Description
Name	The name of the skill that will be provided to the player.
Value	Unique value per effect for each skill.
EffectRUID	The effect resource ID value for each skill.
UI_RUID	The Sprite resource ID value that will be displayed in UI for each skill.

- In the sample world, the features that a player can trigger by using specific skill keys includes additional points gain and HP recovery.
- The CharacterInfo Dataset will store each skill's information as a table using the InitSkillInfoTable method in DataSetLogic.

Skill Link Method per Character

CharacterInfo

	1	2	3	4	+
ID	Name	Speed	SkillId	RUID	
1	TestA	3	1	ac753a90b9814	
2	TestB	4	2	b65a754c25174	
+					

SkillInfo

	1	2	3	4	+
ID	Name	Value	EffectRUID	UI_RUID	
1	Heal	20	db4e6e7fcb1b4	c99e612dbd544	
2	Bonus	500	db4e6e7fcb1b4	eb0b26a384764	
+					

- The SkillId for each character registered in the CharacterInfo DataSet refers to the respective ID number in the SkillInfo DataSet.
 - For example, if TestB character's SkillId is 2, the Bonus skill corresponding to index 2 in the SkillInfo dataset is assigned as the character's skill.

Skill Logic's UseSkill Operation Principles

- In the HandleKeyDownEvent method of the CustomPlayerController, UseSkill method of SkillLogic is called when inputting the skill activation key (K key).
- The CustomPlayerController clearly separates the roles so that it only takes key inputs and passes commands to SkillLogic to activate skills, but does not handle the specific skill implementation logic.
- Skill Logic activates the skill that matches the skillId number variable registered to it.
- The skillId number changes value when you select a character on the title screen.

The UseSkill Principle in Skill Logic

```
number skillId;

void UseSkill()
{
    local skillName = _DataSetLogic.skillInfoTable[self.skillId].Name

    if skillName == "Heal" then
        -- Define a specific sequence of actions when using the HP Recovery
        skill.
    elseif skillName == "Bonus" then
        -- Define a specific sequence of actions when using bonus skills.
    end
}
```

Writing a SkillLogic Script

- Write the following code.
- The target code is based on Plain Text.

```
@Logic
script SkillLogic extends Logic

property number discript = 0 -- The logic declared to execute the detailed
actions when a character's unique skill is used.

property number skillId = 0 -- The variable needed to set the character's
unique SkillId value.

property boolean useable = true -- The state variable that is used to save
the skill availability state.

@ExecSpace("ClientOnly")
method void UseSkill()
-- This method executes the detailed actions of the currently registered
(SkillId).

-- If not available when this method is called,
if self.useable == false then
    -- the method ends.
    return
end

-- Saves the current character skill name.
local skillName = _DataSetLogic.skillInfoTable[self.skillId].Name
-- Saves the unique value of the current character skill.
local skillValue = _DataSetLogic.skillInfoTable[self.skillId].Value

-- If the skill name is 'GetLife',
if skillName == "Heal" then
    -- restores HP equal to the skill's unique value.
    _GameLogic:AddHP(skillValue)

-- If the skill name is 'GetScore',
elseif skillName == "Bonus" then
    -- earns an additional score equal to the skill's unique value.
    _GameLogic:AddScore(skillValue)
end

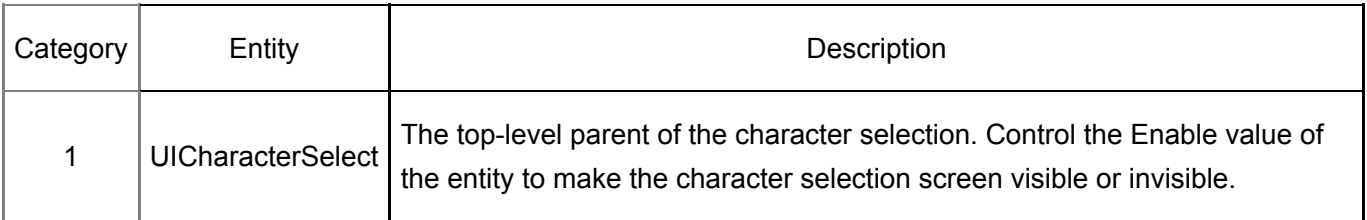
-- Plays skill effect.
self:PlaySkillEffect()

-- Play Skill Sound Effect
_SoundService:PlaySound("56c1b5e3c7fc4abda561c1cc00813bda",1)

-- Converts the state to unavailable, since the skill was used.
self.useable = false

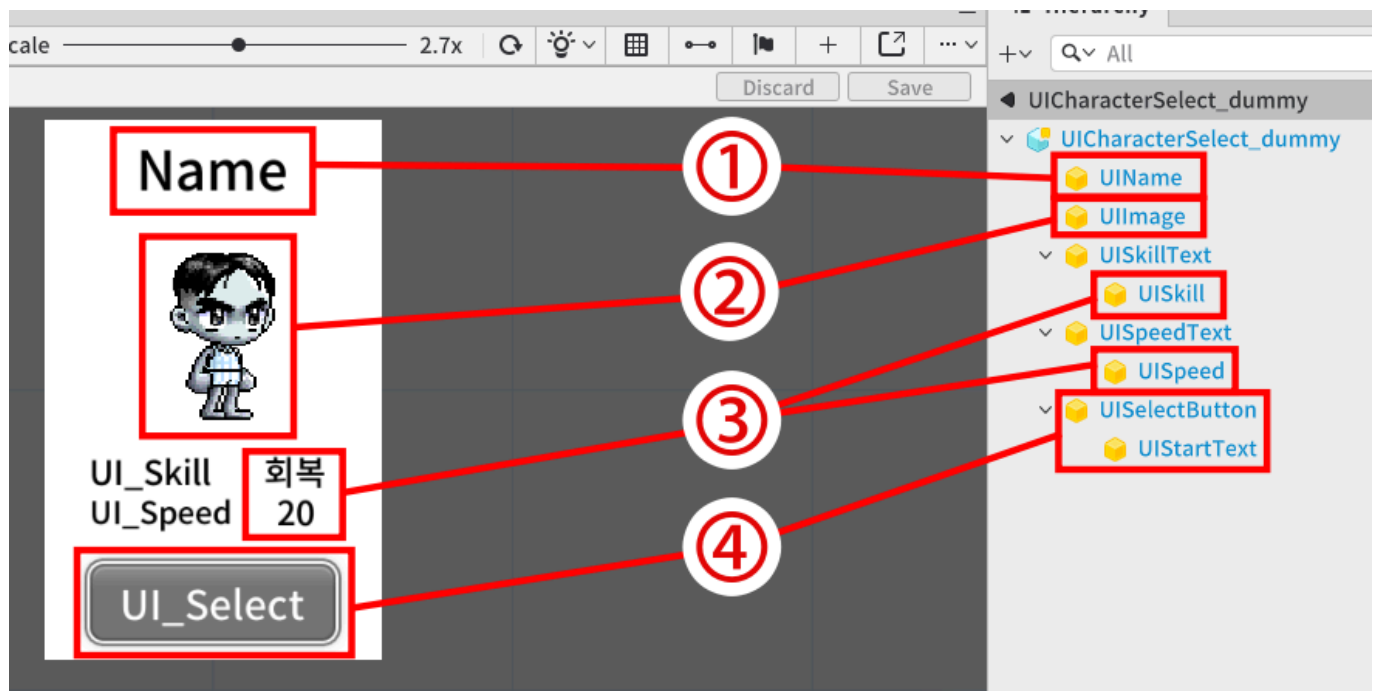
-- Sets the skill icon UI's renderer transparency value to be low.
_UIManagerLogic.skillRenderer.Color.a = 0.3
end
```

Implementing Character Selection



Category	Entity	Description
2	UIBackground UISelectText	This is the section that shows the title text indicating that the popup is the character selection screen.
3	Scroll_Layout	This is a space that lists the 'UICharacterSelect' slots as the area where the various character windows will be displayed. When the item elements exceeds the range of this area, the view area can be controlled by dragging the mouse left and right.

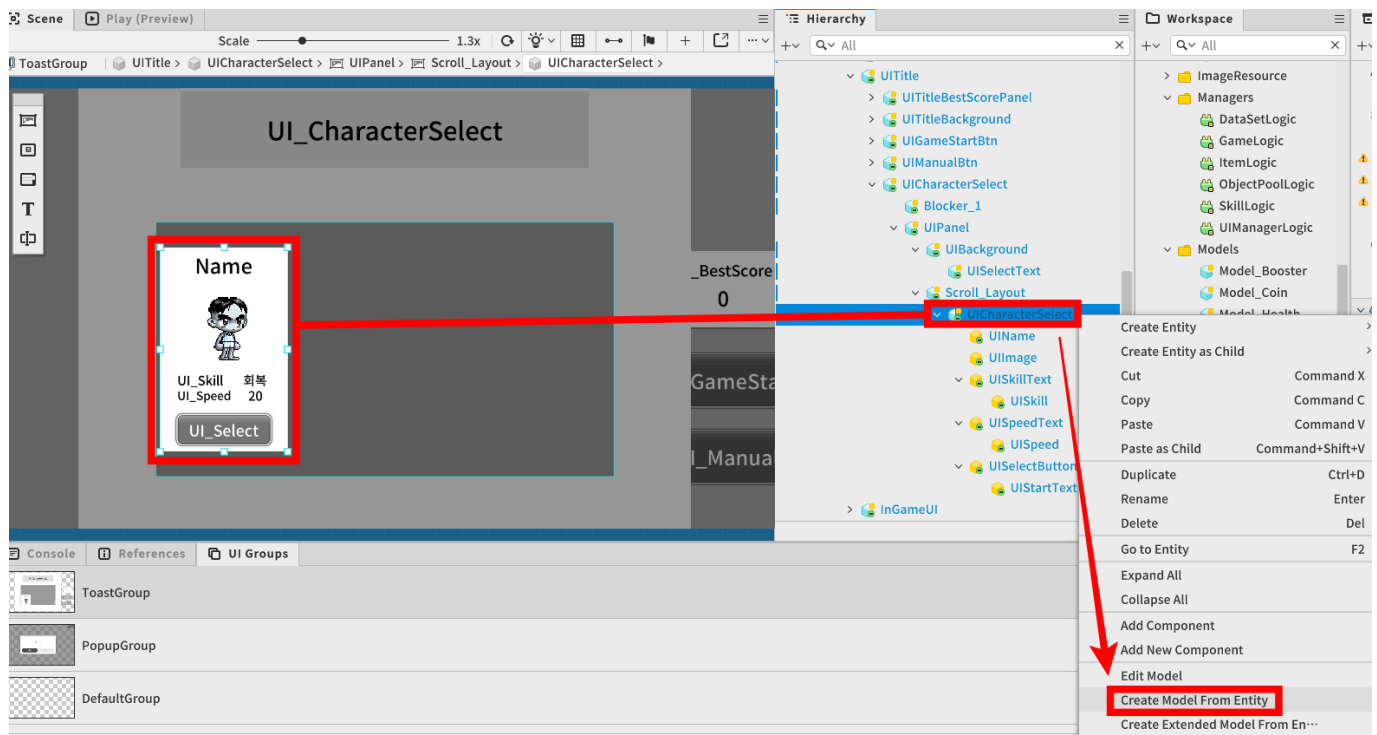
Creating UI Slots to Represent Character Information



Category	Entity	Description
1	UIName	Indicate the character name at the top of the Slot.
2	UIImage	Provides a visual representation of the character's appearance to the player.
3	UISkill UISpeed	Indicates the available skills per character and the scroll speed per character.
4	UISelectButton	If the player wants to play with a character registered to the Slot, provide a button that allows them to play the game as that character.

- A UI Slot is a self-contained UI unit that holds an individual character's unique information (portrait, name, stats, etc.) in a single entity and displays it on screen.
- When listing multiple characters in a list, use this UI Item as a 'framework (module)' that is reused over and over again, replacing only the data.

Creating Slot Item Model



- When creating a single character slot, it is recommended to play around with the UI placement directly within the slot to get an intuitive feel for the actual UI placement or size.
- Once a single unit Slot is created, right-click the entity and select Create Model From Entity to register it as a model to reuse in Workspace.

Registering Information per Character in Character Slot

- Generate all Slot items for all characters in advance when executing the world.
- Each slot must output all elements of the CharaterInfo DataSet and all of the character's data.
- The process for registering character information for a Slot is as follows.
 - Execute World -> Make sure to call the RegisterCharacter method within the automatically called OnBeginPlay.
 - The RegisterCharacter method features are as follows.

RegisterCharacter Method of UICharacterSelectPopup

Entity scrollViewEntity; -- The parent entity where the Slot item should be placed.

```
void RegisterCharacter()
{
```

```
    -- Stores the ModelID of the Slot item registered in Workspace.
```

```
    local modelId = _EntryService.GetModelIdByName("UICharacterSelect")
```

```
    -- Stored as a local variable to simplify writing code that calls the
    character information table.
```

```
    local characterTable = _DataSetLogic.characterInfoTable
```

```
    -- Iterates through the Character Info table a number of times
    corresponding to the registered count (the number of characters registered
```



```
in the CharacterInfo DataSet).
    for i = 1, #characterTable, 1 do

        -- Create a single Slot item using the SpawnByModelID method of the
        SpawnService.
        local characterInfo = _SpawnService:SpawnByModelID(

modelId,"characterInfoPopup",Vector3.zero,scrollViewEntity).UICharacterInfo
Component

        -- Sets the text in the UI that displays the character's name to
        the character name.
        characterInfo.nameText.Text = characterTable[i].Name

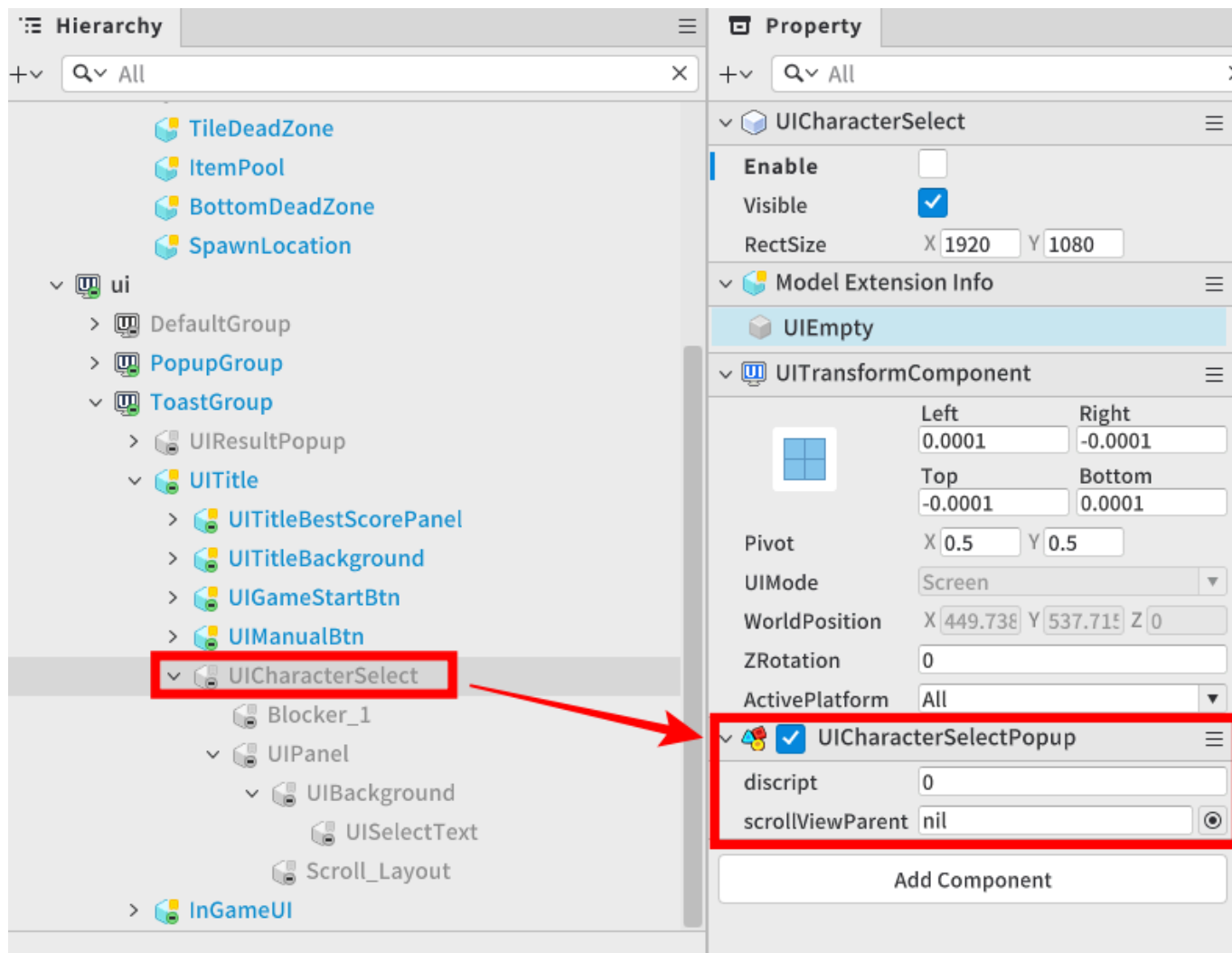
        -- Sets the resource ID value of the character appearance
        (CustomBodyEquip) pre-registered in the Slot.
        characterInfo.costumImage.CustomBodyEquip = characterTable[i].RUID

        -- Extract the SkillId number stored in the CharacterInfo DataSet.
        local skillId = tonumber(characterTable[i].SkillId)
        -- Retrieves the Skill information stored in the SkillInfo DataSet
        and registers the name of the skill.
        characterInfo.skillText.Text =
        _DataSetLogic.skillInfoTable[skillId].Name

        -- Set the text in the UI that indicates the character's scroll
        rate to the character's unique scroll rate.
        characterInfo.speedText.Text = characterTable[i].Speed

    end
}
```

Creating a Character Selection Screen



- In the top-level entity of the Character Selection window, register a component that performs the function of creating character slots when executing the world.
- When building is complete, leave the entity disabled, as the player shouldn't see the popup until they click the Start Game button.
- Write the following code.
- The target code is based on Plain Text.

```
@Component
script UICharacterSelectPopup extends Component

property number discript = 0 -- This component defines the function needed
when displaying the character selection window pop-up.

property Entity scrollViewParent = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
This method is called when the world starts.

Links the entity that will be the parent of the objects that were created
from the model (prefab).
self.scrollViewParent =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UICharacterSelect/UI
```

```
Panel/Scroll_Layout")
```

Proceeds with character information registration.

```
self:RegisterCharacter()
end
```

```
@ExecSpace("ClientOnly")
```

```
method void RegisterCharacter()
```

This method registers the character information from the character selection window.

This is to set the data values that are **input** internally when a pop-up displaying the character information is created from a model (prefab).

```
-- Loads the default model (prefab) element's model Id.
```

```
local modelId = _EntryService:GetModelIdByName("UICharacterSelect")
```

```
-- Loads the character information table from the data set.
```

```
local characterTable = _DataSetLogic.characterInfoTable
```

Executes a loop a number of times equal to the number of characters registered in the **table**.

```
for i = 1, #characterTable, 1 do
```

Creates a UI pop-up **and** saves the character info component using the spawn service.

```
local characterInfo =
_SpawnService:SpawnByModelId(modelId,"characterInfoPopup",Vector3.zero,self
.scrollViewParent).UICharacterInfoComponent
```

Loads the skillId value needed to classify the skill info **table**.

```
local skillId = tonumber(characterTable[i].SkillId)
```

Sets the character skill number that will be set inside character info.
characterInfo.skillId = skillId

Sets the character name that will be set in character info.

```
characterInfo.nameText.Text = characterTable[i].Name
```

Sets the costume body RUID value in character info.

```
characterInfo.costumeImage.CustomBodyEquip = characterTable[i].RUID
```

Sets the skill name in character info (loads skill name from the skill info **table** based on the saved SkillId number).

```
characterInfo.skillText.Text =
_DataSetLogic.skillInfoTable[skillId].Name
```

Sets the speed text in character info.

```
characterInfo.speedText.Text = characterTable[i].Speed
```

Sets the character ID (index) value.

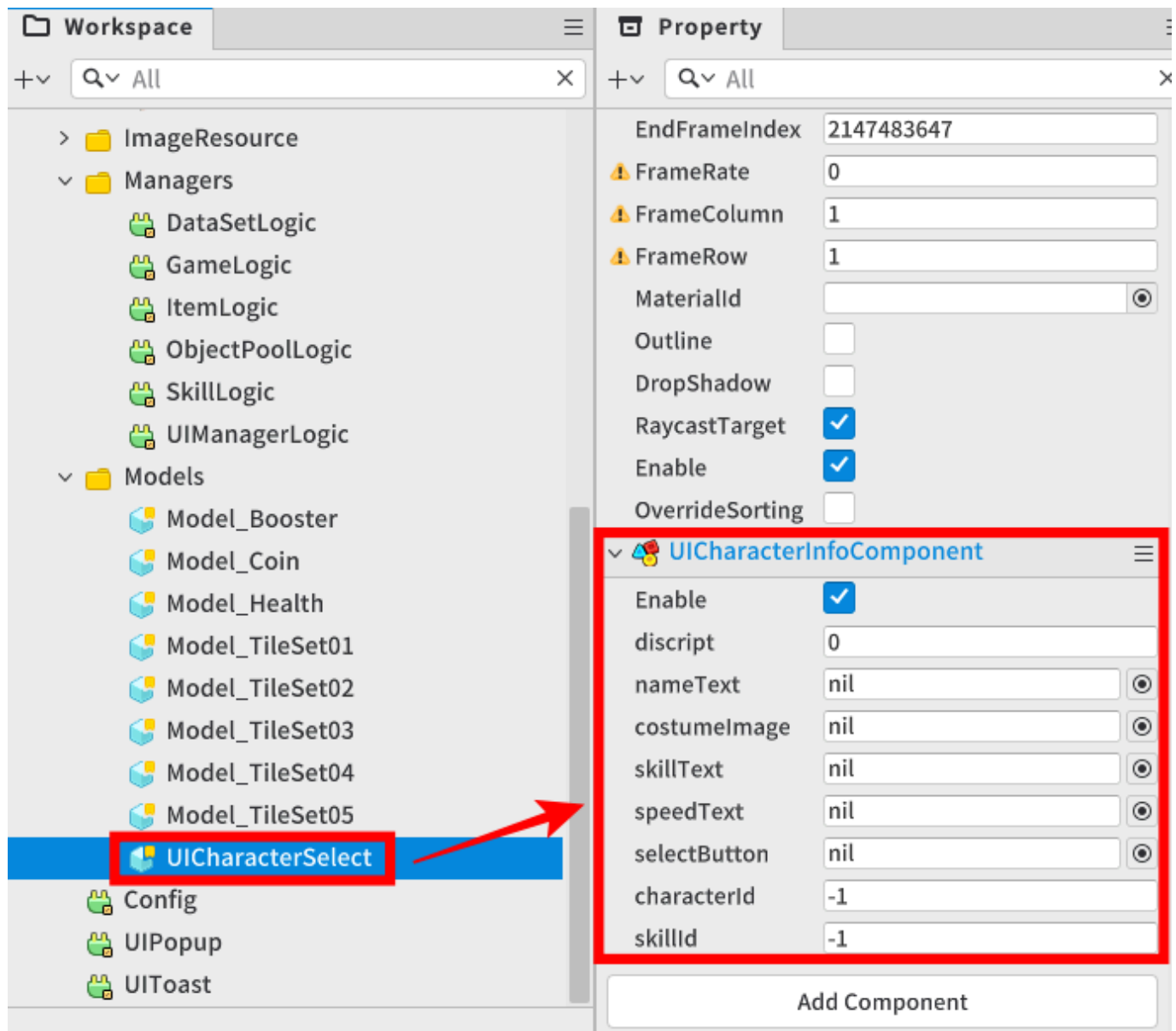
```
characterInfo.characterId = i
```

```
end
```

```
end
```

```
end
```

Creating a Character Slot



- Create a script to trigger the function of a button placed at the bottom of the Slot that will perform the character selection function.
- Make the button's event able to call the CloseSelectPopup method.
- The CloseSelectPopup method performs the following actions.
 - Disables UI elements that make up the title screen that is visible at the start of the game.
 - Change the appearance of the character used by the player.
 - Pass the character's skill number to SkillLogic.
 - Conveys the current character's number to GameLogic and calls the game start function.
- Write the following code.
- The target code is based on Plain Text.

```
@Component
script UICharacterInfoComponent extends Component

property number discript = 0 -- The component needed to display each
character's information in the character selection window.
```

```
property TextComponent nameText = nil

property CostumeManagerComponent costumeImage = nil

property TextComponent skillText = nil

property TextComponent speedText = nil

property ButtonComponent selectButton = nil

property number characterId = -1 -- The variable needed to classify the
character index.

property number skillId = -1 -- The character's unique skill number.

@ExecSpace("ClientOnly")
method void OnBeginPlay()
This method is called when the world starts.

-- Resets the target component.
self:Init()
end

@ExecSpace("ClientOnly")
method void Init()
-- This method resets the CharacterInfo component.

-- Executes the model (prefab) child object component link.

-- Loads the text component that will enter the character name from the
child object.
self.nameText = self.Entity:GetChildByName("UIName").TextComponent
-- Loads the costume component that will register the character image from
the child object.
self.costumeImage =
self.Entity:GetChildByName("UIImage").CostumeManagerComponent
-- Loads the text component that will input the skill name from the child
object.
self.skillText = self.Entity:GetChildByName("UISkill",true).TextComponent
-- Loads the text component that will input the speed value from the child
object.
self.speedText = self.Entity:GetChildByName("UISpeed",true).TextComponent
-- Loads the button component that will detect clicks from the child
object.
self.selectButton =
self.Entity:GetChildByName("UISelectButton").ButtonComponent

-- Registers the method (CloseSelectPopup) that will be executed when the
character select button is clicked.
self.selectButton.Entity:ConnectEvent(ButtonClickEvent,self.CloseSelectPopu
p)

end
```

```

@ExecSpace("ClientOnly")
method void SetInfo(string name, string ruid, string skill, string speed)
-- This method is needed to set the required information and content needed
for the character information window.

-- Parameter 1 string name: Receives the character name.
-- Parameter 2 sting ruid: Receives the character torso RUID value.
-- Parameter 3 string skill: Receives the character skill name.
-- Parameter 4 string speed: Receives the character scrolling speed value.

-- Sets the character name text.
self.nameText.Text = name
-- Sets the costume body part resource.
self.costumeImage.CustomBodyEquip = ruid
-- Sets the skill name text.
self.skillText.Text = skill
-- Sets the scrolling speed value text.
self.speedText.Text = speed
end

@ExecSpace("ClientOnly")
method void CloseSelectPopup()
-- This method deactivates the character selection window pop-up.

-- Loads the title panel (start button, controls, character selection
window's parent) entity before the game starts.
local TitlePanel = _EntityService:GetEntityByPath("/ui/ToastGroup/UITitle")

-- Deactivates the panel object.
TitlePanel.Enable = false

-- Sets the character's costume body resource value.
_UserService.LocalPlayer.CostumeManagerComponent.CustomBodyEquip =
self.costumeImage.CustomBodyEquip

-- The character that will be used in the game sets the unique skill Id.
_SkillLogic.skillId = self.skillId

-- Requests game progression with the selected character Id number to the
game logic.
_GameLogic:GameStart(self.characterId)
end

end

```

Minimum Implementation Standards

- The player character should correctly detect collisions with floor tiles, and the limiting feature should work correctly so that jumps are only performed while on the ground and not jump infinitely

in the air.

- The character selection window should be built to tell GameLogic to call GameStart when a button registered to a character Slot is inputted.

Usage and Extension Direction

- In addition to scrolling speeds for each character, add detailed stat columns to the CharacterInfo DataSet, such as Jump, Max HP, and invincibility time, to provide different play styles for different characters.
- Design and add a variety of action skills, such as 'invincible dash' to run through obstacles or allow players to grab nearby coins all at once.
- When selecting a character, go beyond swapping just a single body costume and bundle different parts data, such as hair, face, cape, and more, to make changes and advance the script.

[💡 Basic Training] Implementing a Life System

Video Timeline

- You can follow along by using the timeline under the training video.
- Implementing a Life System: 01:20:58

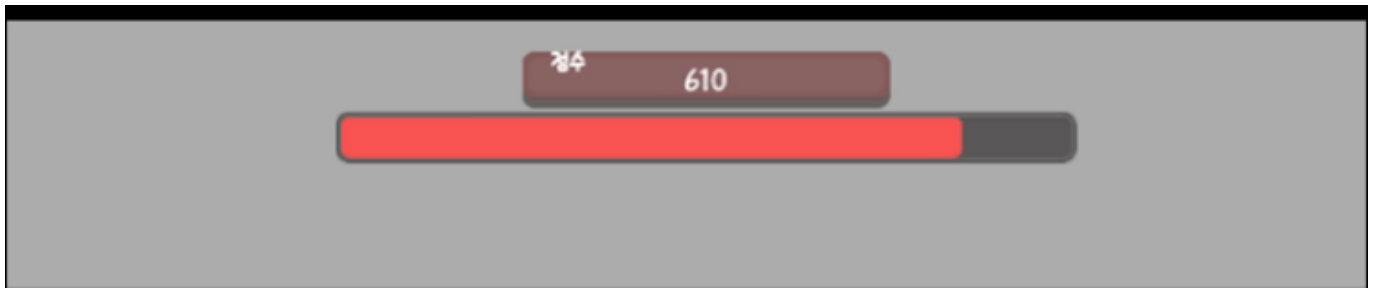
Learning Goals

- Understand the OnUpdate method and the concept of delta time, and implement logic that decreases HP over time, independent of device frames.
- Learn a state control flow that detects when a character's HP drops below 0 and performs a game over.
- Learn how to attach a SliderComponent to a UI Entity to create a visual HP bar and synchronize internal data with the on-screen UI in real-time.

Practical Exercises to Do After Watching the Video

- Try changing the value of the decreaseHpValue variable set in the Config Logic script to set how quickly the HP decreases over the course of the test.
- In-game, test by pressing the Play button to see if the HP gauge in the UI decreases smoothly over time, and if it transitions properly to the results window when it reaches zero HP.

HP Decrease per Time Implementation



- HP control was built to include both immediate changes when picking up items and hitting obstacles as well as periodic HP decreases over time.
- The ability to decrease HP in code is defined as a function of GameLogic's AutoDecreaseHp, AddHP, and SetHP methods, each of which are performed within GameLogic's OnUpdate method.
- If the contents of the GameLogic code are needed, the GameLogic script is attached in the '2. Understanding Logic' section.

HP Setting Function

- The ability to add and decrease HP utilizes GameLogic's AddHP method.

AddHP Method in GameLogic

```
number hp;
```

```
void AddHP(number addValue)
```

```
{
```

```
    -- Adds or subtracts the value passed as a parameter to the current HP as is.
```

```
    -- When calling the AddHP method, there's a flexibility to pass a positive or negative number to perform both recovery and reduction functions.
```

```
    self.hp += addValue;
```

```
    -- Request that the information in the sliders managed by UIManagerLogic be updated because a change in the HP value has occurred.
    _UIManagerLogic.SetHpSliderValue(self.hp)
```

```
    -- If HP drops below 0 (when the Game Over condition is met)
```

```
    if self.hp <= 0 then
```

```
        -- Perform the End Game function.
```

```
    end
```

```
}
```

Continuous HP Decrease Function Implementation

- By utilizing the OnUpdate method, which is called every frame, the feature of gradually decreasing HP can be achieved.

- However, if this is not multiplied by delta, which is the amount of time change between frames, there will be a fatal problem where the HP will evaporate unusually quickly depending on the device's frame rate (FPS).
- If frames per second is 60 FPS, the delta value of the time change between frames is $1\text{Sec}/60\text{FPS} = 0.0167\text{Sec}$.
- If the desired effect is to decrease 5 HP per sec, the value of $5 * \text{delta}$ will be calculated and reduced for every frame. Whether the device is running at 60 frames or 30 frames, the cumulative reduction at the end of one second will be exactly 5.

Example of the HP reduction function part of GameLogic's OnUpdate method:

```
void OnUpdate(number delta)
{
    -- Config : If the value is not changed once set, it is registered and
    managed in the corresponding Logic script.
    -- Reflects HP by the set HP loss per second * delta
    self:AddHP(delta * _Config.decreaseHpValue)
}

void AddHP(number addValue)
{
    -- Adds or subtracts the value passed in as a parameter as it is to the
    HP value.
    -- When utilizing the AddHP method, pass the positive and negative
    values to perform the intended function.

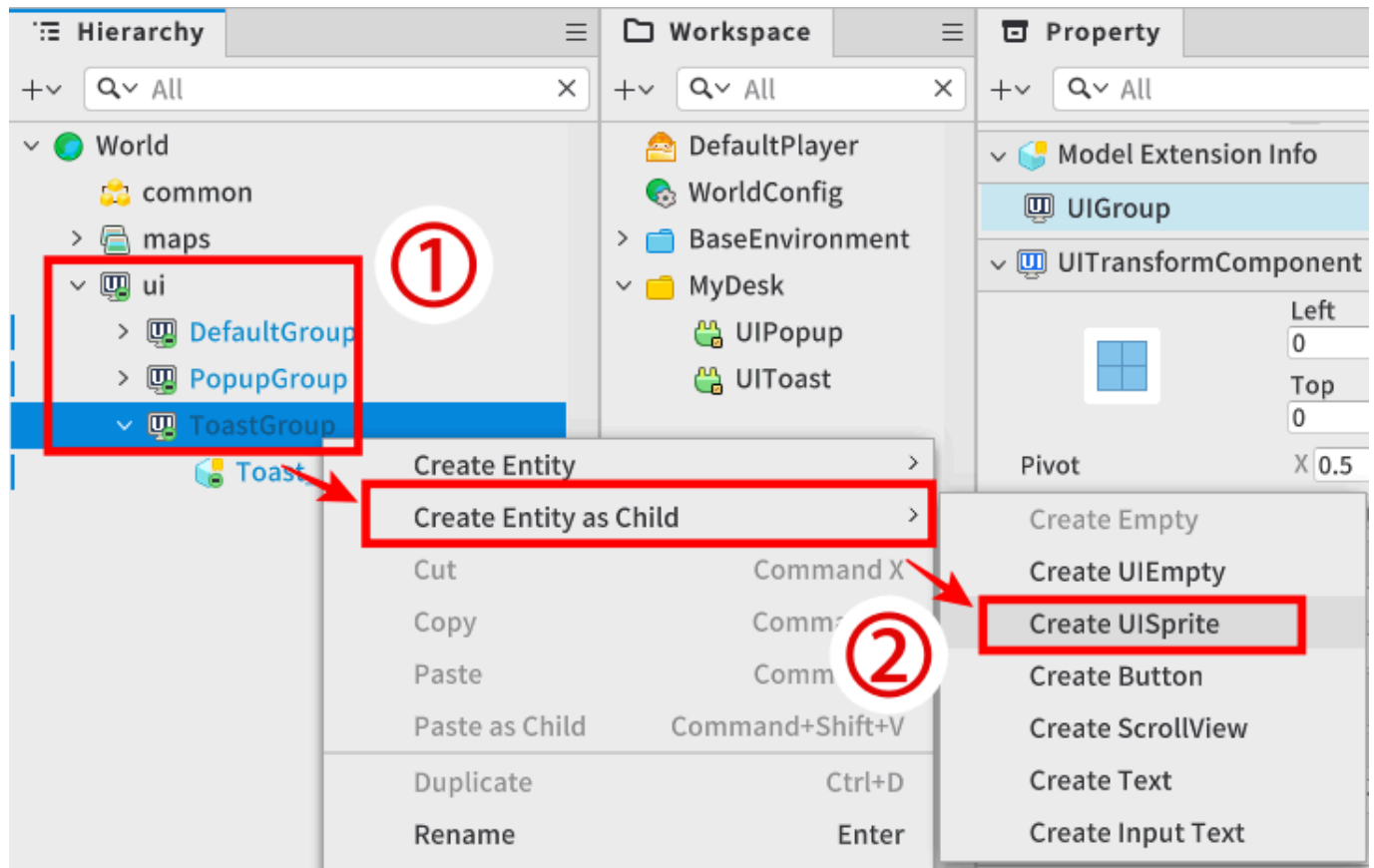
    self.hp += addValue;

    -- Request that the information in the sliders managed by
    UIManagerLogic be updated because a change in the HP value has occurred.
    _UIManagerLogic.SetHpSliderValue(self.hp)

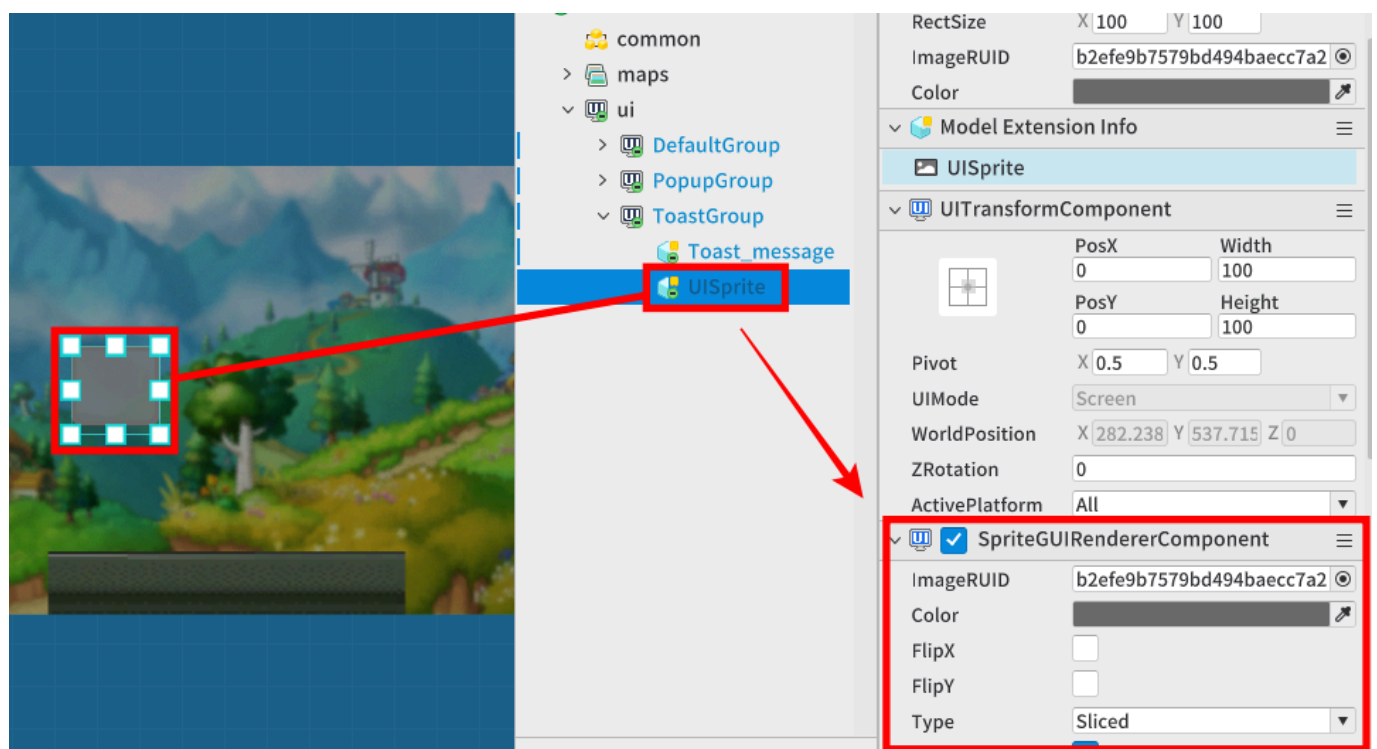
    -- When HP reaches 0 (when game can no longer proceed)
    if self.hp <= 0 then
        -- Perform the End Game function.
    end
}
```

Integrate the HP Bar (UI) Utilizing SliderComponent

Creating HP Bar UI

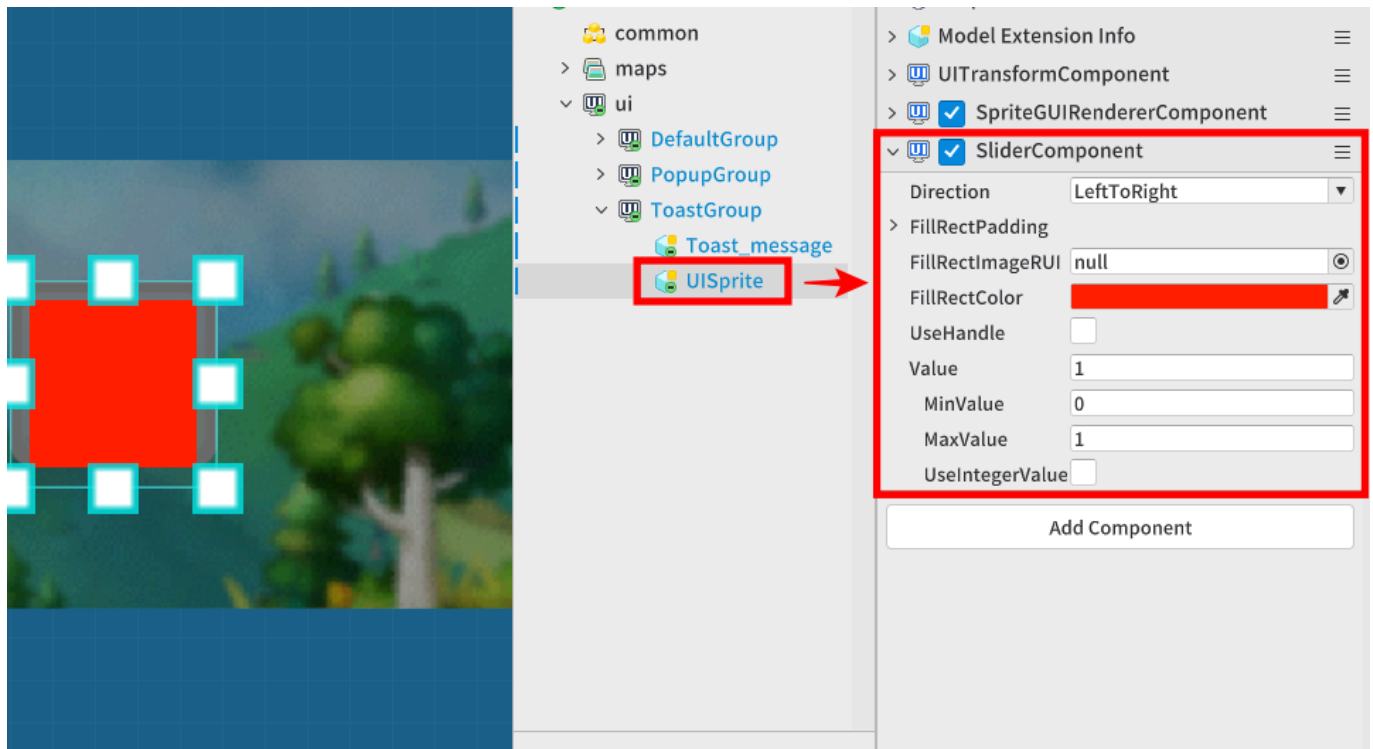


1. Within the Ui sub-item, select the entity that serves as the canvas.
2. Under Create Entity as Child, select Create UISprite to create a UI that will be utilized as a Sprite.
3. The UI is a Sprite image that will be the background for the HP bar.



- In the created entity, set the image RUID and color in the SpriteGUIRendererComponent to be used as the background for the HP bar.

✅ If the created UI entity is not visible, check if the coordinates set for UITransformComponent, the Component that is registered to the entity by default, are incorrect.



- Register a SliderComponent to cover the background of the HP bar.
- Set the FillRectColor color and verify that it covers the background image normally.
- Try changing the component's Value, which is the internal property and see if it works normally as a gauge.
- The sample world is built with a max HP value of 100.

Controlling HP Bar UI with UIManagerLogic

- The entity that will act as the gauge is registered in UIManagerLogic.
- If the contents of the UIManagerLogic code are needed, the UIManagerLogic script is attached in the '2. Understanding Logic' section.
- Establishes a reference to the entity to which the SliderComponent is attached to and calls this logic whenever the HP changes in GameLogic to overwrite the Slider's Value property with the latest HP value in real time.

Example of implementing the SetHpSliderValue method of UIManagerLogic:

```
SliderComponent hpSlider;
```

```
void SetHpSliderValue(number value)
```

```
{
```

```
    -- Set the value of the slider to be set using the math.clamp method so  
    that it is never less than 0 and never more than 100.
```

```
    self.hpSlider.Value = math.clamp(value, 0, 100)
```

```
}
```

Minimum Implementation Standards

- Changes in HP data (hp) within the script should be passed through UIManagerLogic and synchronized in real-time to the on-screen SliderComponent gauge UI.
- As soon as HP drops to 0 or lower, it should be detected and prevented from going any further into negative HP, and game over handling should be called successively.

Usage and Extension Direction

- The gauge color of the SliderComponent can be changed to flash red when the HP drops to a critical state below a certain number (for example, 20%) to provide visual tension and warning feedback to the player.

[💡 Basic Training] Composing UI

Video Timeline

- You can follow along by using the timeline under the training video.
- Composing UI: 01:31:40

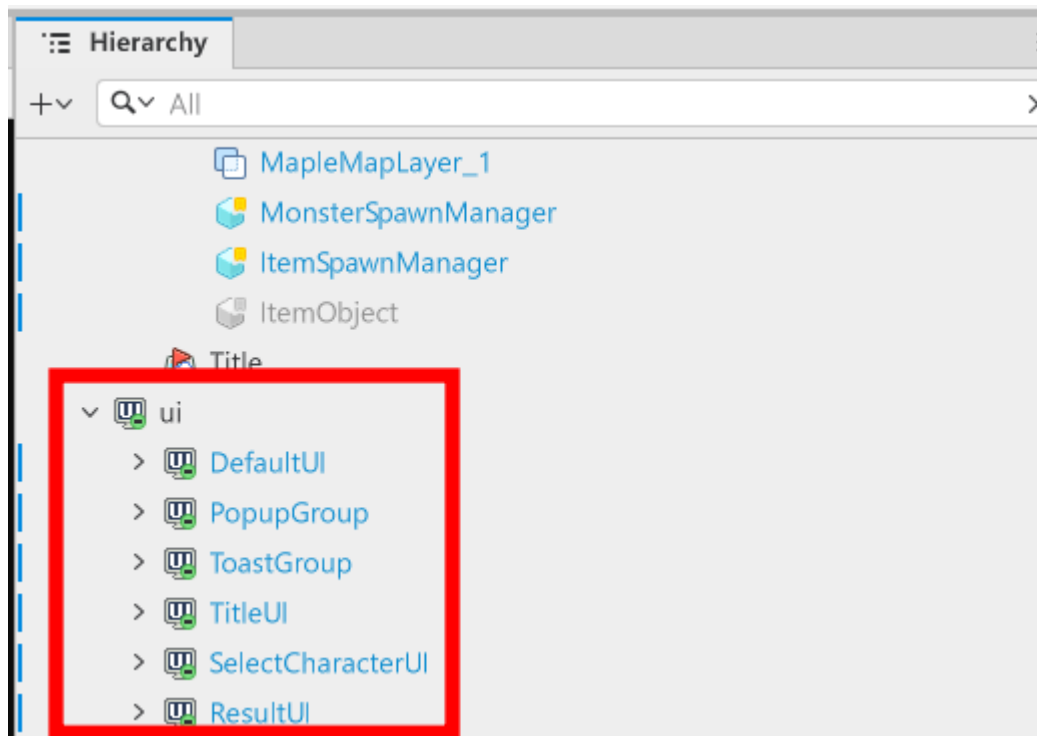
Learning Goals

- Instead of controlling individual UI elements one by one, understand how to bundle them into a single UI entity to efficiently transition screens based on the state of the game (title, in-game, result).
- Create a Result Popup that is called at the end of the game, and learn how to integrate GameLogic with UIManagerLogic to synchronize the current score and best score to UI text.

Practical Exercises to Do After Watching the Video

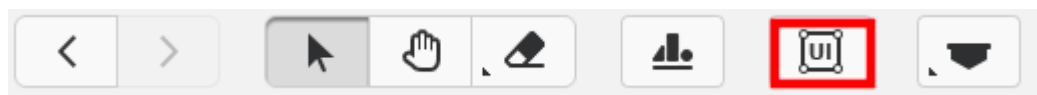
- In the Hierarchy window, group the UI elements needed for the Title, In-Game, and Results screens into their respective parent entities (UI Group), toggle the Enable property of the top-level entity on and off, and test to see if the screens transition properly.
- In-game, intentionally hit an obstacle to end the game, then check to see if the results popup correctly displays the score just earned and the existing best score in the text UI.

Understanding UI Groups

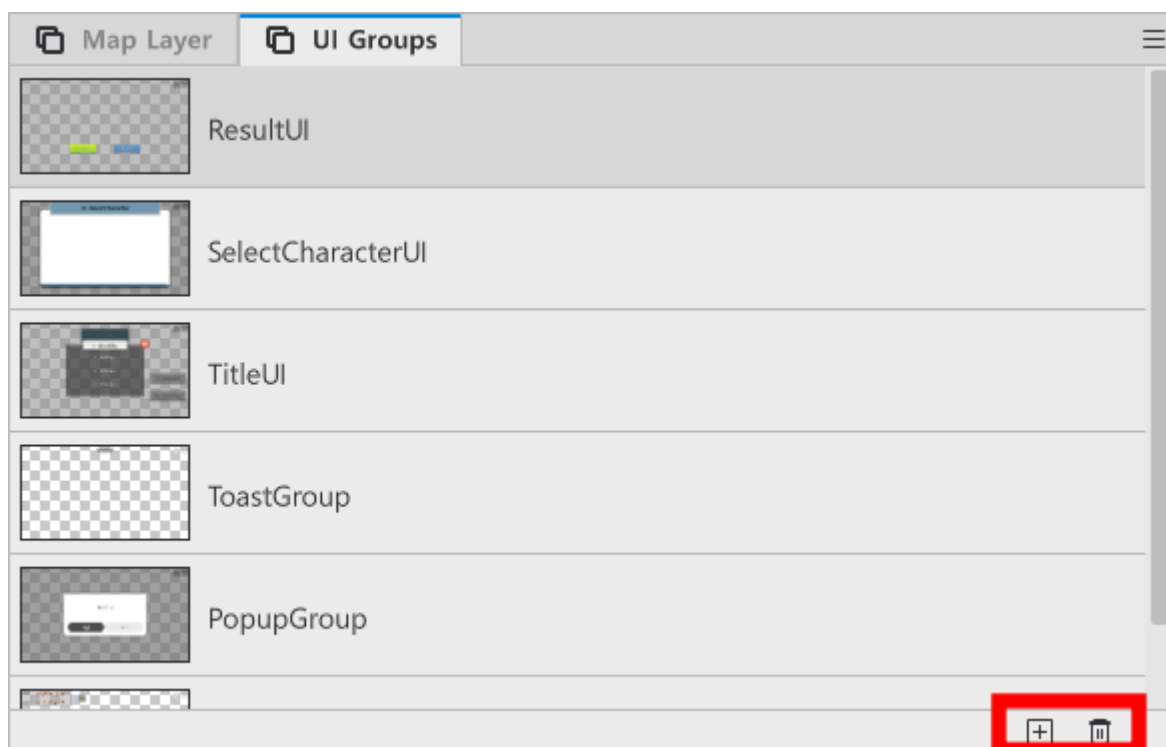


- Looking at the bottom of the Hierarchy window, entities organized under the name UI can be seen.
- Those entities are a ui group that outputs the ui.
- Edit the ui group to change or create the ui that needs to be output.

Creating a UI Group



- Click the UI button on the top of the editor to change to UI editing mode.

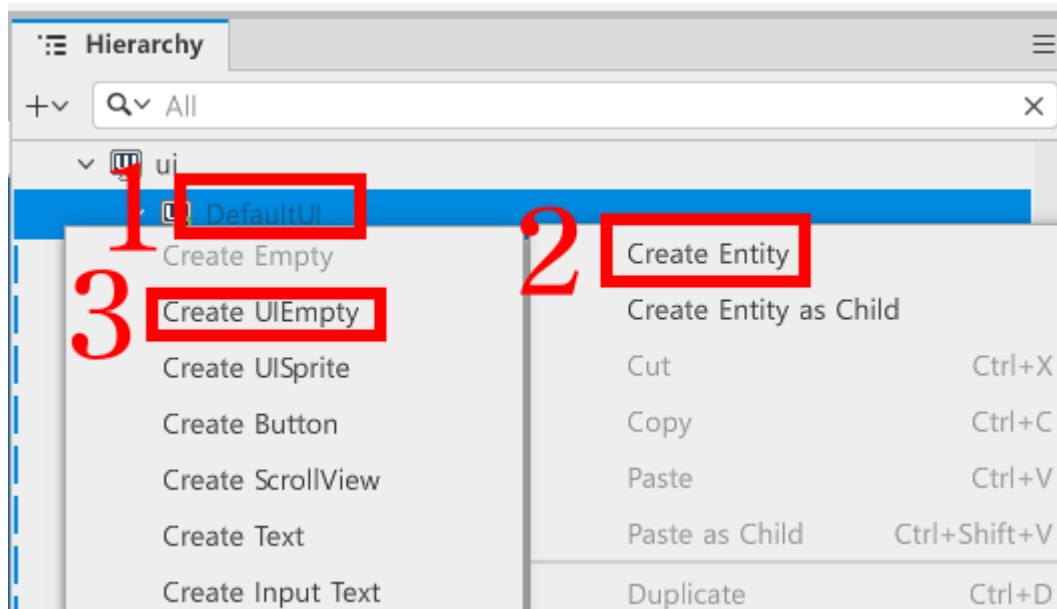


- Click on the + button from UI Groups on the bottom of the editor window to add a new UI Group.

- If you have UI Groups that needs to be deleted, click the UI Group you want to delete and click on the recycling bin icon to delete it.

Creating Empty UI Entities

- You can create entities in the UI like you did with existing entities.
- The difference is when creating an empty entity, you must use **Create UIEmpty** instead of Create Entity.

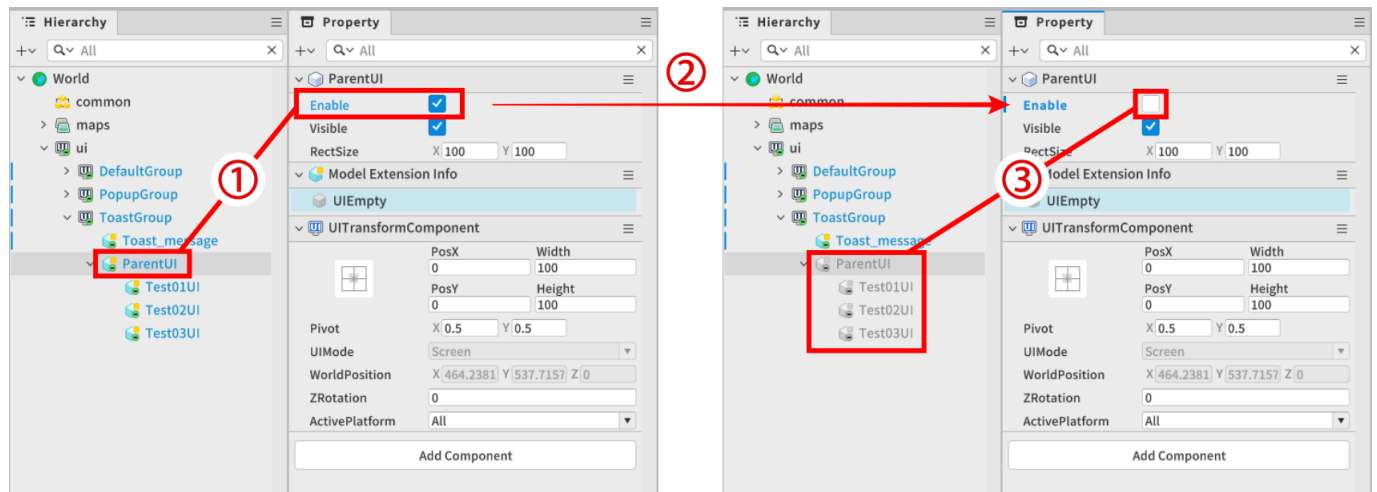


- Right-click the ui group you want to add an entity to from the Hierarchy window.
- Click Create Entity.
- Click Create UIEmpty to create an empty UI entity.

Applying a Component to a UI Entity

- Like normal entities, you can add Components to UI entities to execute additional functions.
- Adding Components is done the same way as the Add Component function for regular entities.
- This lesson will talk about the often used Components.
 - **SpriteGUIRendererComponent**: A Component used when outputting an image for the UI. It's recommended to use this Component rather than SpriteRendererComponent when outputting images in the UI.
 - **TextGUIRendererComponent**: A Component used when outputting text on the UI.
 - **ButtonComponent**: A Component used when the UI needs a button function.

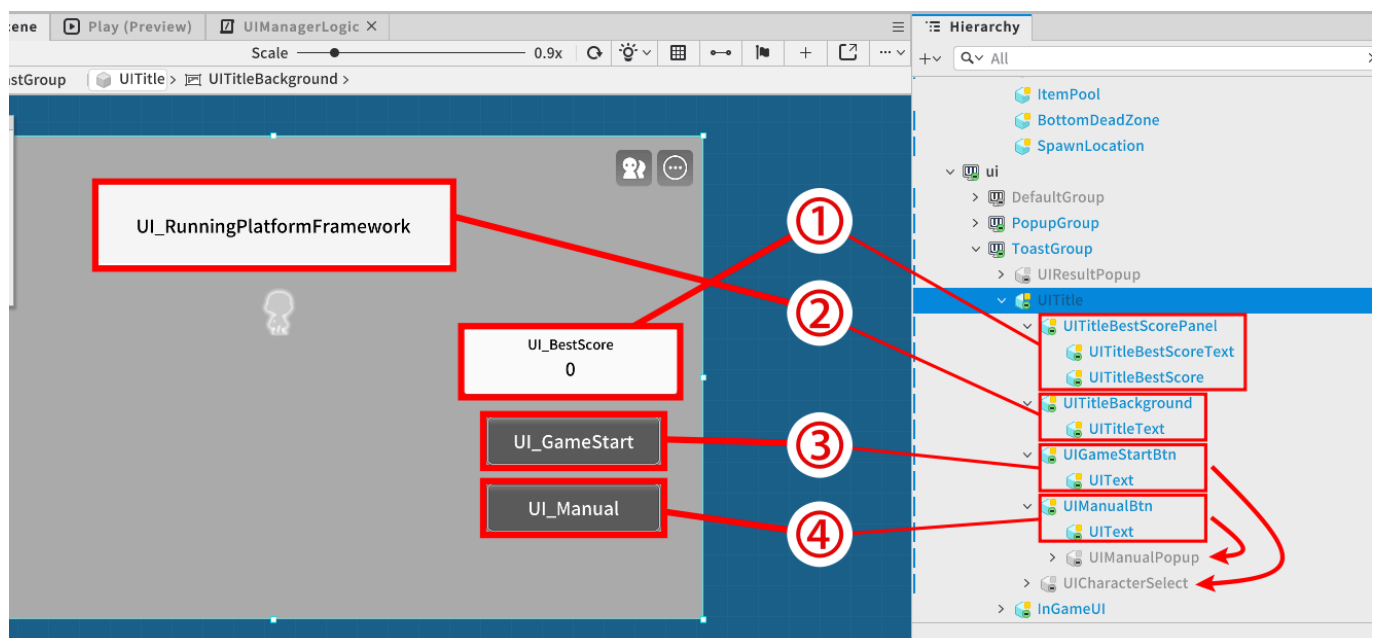
Group-Level Control Utilizing UI Groups



1. When the Enable value of the highest-ranking entity element of a group is checked to true, child entities can individually toggle Enable to true or false.
2. Clear the Enable state by toggling the top-level entity element to false.
3. When the parent entity is deactivated, the child entities are automatically deactivated as well.

- Due to the nature of entities, disabling a parent entity will automatically disable any child entities that belong to the entity.
- This makes it easy to control the UI that should be displayed in different situations in the game by enabling and disabling only the top-level entities in a group, rather than controlling each element individually.

Creating Title UI Group



Category	Group	Description
1	UITitleBestScorePanel	This is the area that displays the player's best score at the start of the world. UITitleBestScoreText - Text that displays the 'best score' at the top of the panel.

Category	Group	Description
		UITitleBestScore - Text that is intended to display the actual best score only.
2	UITitle	This is the area that displays the title of the game at world start. UITitleText - The text that displays the title of the game.
3	UIGameStartBtn	For the player to start the game, proceed with this button. This button is linked to activate the character selection screen popup (UICharacterSelect). UIText - Text that says 'Start Game'.
4	UIManualBtn	If the player wants to learn how to play the game, they can use this button to see the controls. This button is linked to activate the controls popup (UIManualPopup). UIText - Text that represents the 'Controls'.

- For the TitleUI group, players should be able to see them immediately at the start of world, so the default Enable is set to true.
- The time to activate the TitleUI group directly is when the full game has ended and transitioning from the results window back to the main menu.
- The time to disable the TitleUI group directly is when the player selects a character and enters the actual game state.

Situations where the game starts with the character **select** button **in** the character slot (UICharacterSelect) **in** the character selection screen.
CloseSelectPopup method of UICharacterInfoComponent

```
void CloseSelectPopup()
{
    -- Stores the top-level entity of the Title UI group.
    local TitlePanel =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle")

    -- Disables the top-level entity.
    TitlePanel.Enable = false

    -- Asks GameLogic to call Start Game.
    _GameLogic:GameStart(self.characterId)
}
```

-- A situation where the player is returned to the title screen via the
'Main Menu' button in the game results window.
-- ReturnToMenu method of UIManagerLogic

Entity mainTitlePanel


```

Entity resultPopup
Entity resultToMenuButton

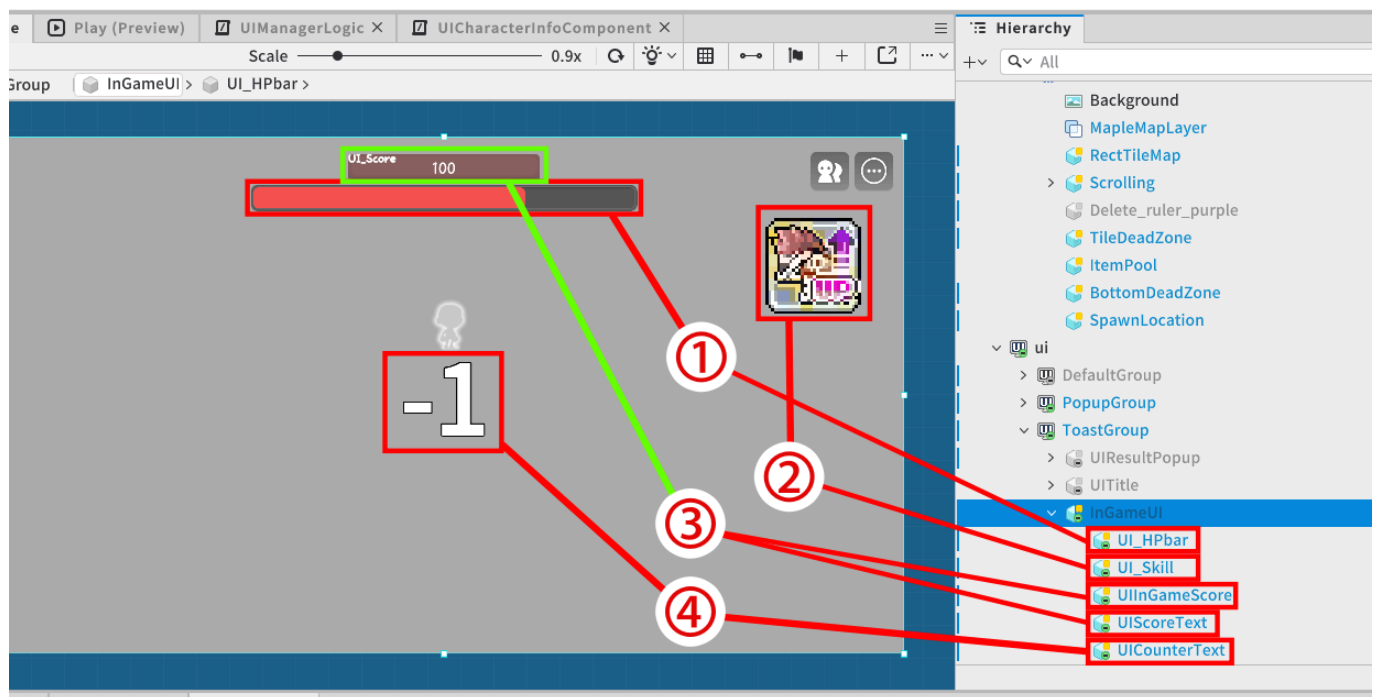
void OnBeginPlay()
{
    -- Register an event on the 'Main Menu' button in the results window
    popup when executing the world.

    self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
}

void ReturnToMenu()
{
    -- Disables the results window popup.
    self.resultPopup.Enable = false
    -- Enable the TitleUI top-level entity.
    self.mainTitlePanel.Enable = true
}

```

Creating InGame UI Group



Category	Group	Description
1	UI_HPbar	A UI that allows the player to see the current HP status of their character.
2	UI_Skill	This is the UI that shows what skills the character has available to them, and checks whether they have been used.

Category	Group	Description
3	UI_Score	UIInGameScore - A UI that displays the actual score value the player has earned as the game progresses. UIScoreText - A UI that displays the "Score" text in the upper left corner of the panel.
4	UICounterText	Just before the actual game begins, let the player know that scrolling will begin in N seconds.

- The time to activate the InGameUI group directly is when the player has completed character selection and the actual game is starting.
- The only time to disable the InGameUI group directly is when the character dies and goes from the results window to the main menu.

```

-- Calling the necessary in-game UIs when a player finishes selecting a
character
-- ShowPlayerInfo method of UIManagerLogic

SliderComponent hpSlider
Entity inGameCurrentScoreText
Entity inGameScoreText
SpriteGUIRendererComponent skillRenderer

void ShowPlayerInfo(boolean active)
{
    -- Transition to active state, to receive the entity representing the
    HP gauge.
    self.hpSlider.Entity.Enable = active
    -- Transition to active state, to receive the entity representing the
    player's score value.
    self.inGameCurrentScoreText.Entity.Enable = active
    -- Toggle the UI element that displays the 'score' text to active.
    self.inGameScoreText.Entity.Enable = active
    -- Transition to the active state to receive the entity representing
    the character's skill information.
    self.skillRenderer.Entity.Enable = active
}

GameStart method in GameLogic

void GameStart()
{
    _UIManagerLogic:ShowPlayerInfo(true)

    -- Perform any necessary initialization before starting in-game.
}

```

```

-- A situation where the player is returned to the title screen via the
'Main Menu' button in the game results window.
-- ReturnToMenu method of UIManagerLogic

Entity mainTitlePanel

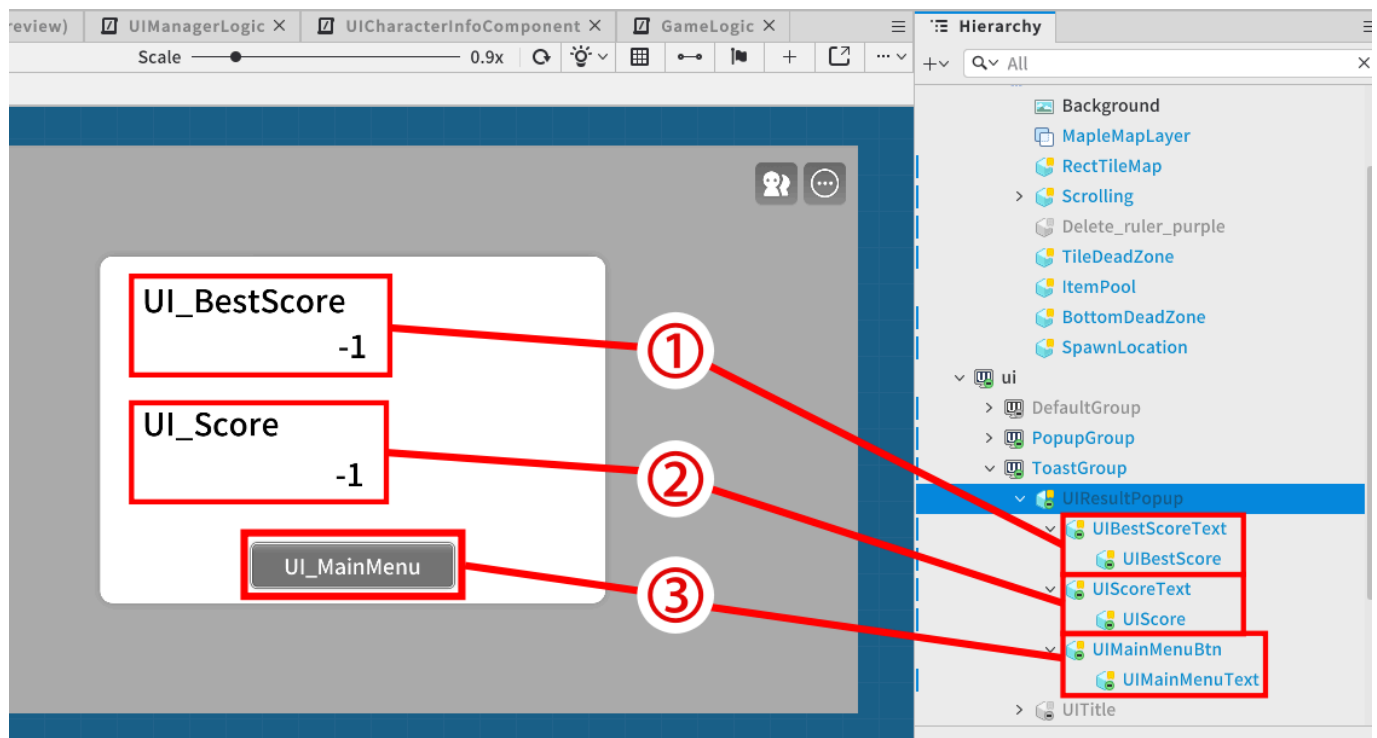
void OnBeginPlay()
{
    -- Register an event on the 'Main Menu' button in the results window
    popup when executing the world.

    self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
}

void ReturnToMenu()
{
    -- Disables all UI elements required for in-game.
    self:ShowPlayerInfo(false)
    -- Enable the TitleUI top-level entity.
    self.mainTitlePanel.Enable = true
}

```

Creating a Results Popup



Category	Group	Description
1	UIBestScoreText	A UI that displays the 'Best Score' text within the results window. UIBestScore: A text UI that displays the actual existing best score value for the player.
2	UIScoreText	The UI that displays the 'Score' text within the results window. UIScore: The text UI that displays the actual player's score at the end of the current game.
3	UIMainMenuBtn	Button UI that performs the function of closing the results window popup and returning to the Title state. UIMainMenuText: UI that represents the 'main menu' text within the button.

- The time to activate the ResultPopup group directly is when the actual game proceeds and the character is no longer playable.
- The time to disable the ResultPopup group directly is when the 'MainMenu' button is selected within the results screen to switch to the Title state.

```
-- In-game ends and a results screen pops up.
-- ShowResultPopup method of UIManagerLogic

-- A Text entity that registers the actual previous high score registered
in the results screen.
Entity resultBestScoreText
-- A Text entity that registers the actual current score value registered
in the results screen
Entity currentScoreText

void ShowResultPopup(string bestScore, string currentScore)
{
    -- Change the highest score value text to the text passed as a
    parameter.
    self.resultBestScoreText.TextComponent.Text = bestScore

    -- Change the current score value text to the text passed as a
    parameter.
    self.currentScoreText.TextComponent.Text = currentScore

    -- Activates the results screen.
    self.resultPopup.Enable = true;
}

EndMainGame method of GameLogic called when a character dies

number bestScore
number currentScore

void EndMainGame()
```

```
{
    -- Perform necessary functions at end game.

    _UIManagerLogic:ShowResultPopup(tostring(self.bestScore),tostring(self.currentScore))
}
```

```
-- A situation where the player is returned to the title screen via the
'Main Menu' button in the game results window.
-- ReturnToMenu method of UIManagerLogic

Entity resultPopup
Entity mainTitlePanel

void OnBeginPlay()
{
    -- Register an event on the 'Main Menu' button in the results window
    popup when executing the world.

    self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
}

void ReturnToMenu()
{
    -- Disables the Results screen popup.
    self.resultPopup.Enable = false
    -- Enable the TitleUI top-level entity.
    self.mainTitlePanel.Enable = true
}
```

Minimum Implementation Standards

- The UI of each button to switch between the three states, TitleUI, InGameUI, and ResultPopup, as well as the ability to start the game, exit the game, and return to the main menu should work properly.
- Make sure that the currentScore and bestScore values calculated by GameLogic are passed to each UI text component correctly.

Usage and Extension Direction

- Adds effects to the ResultPopup so that when it appears, it doesn't just appear, but gradually grows in size or changes transparency.
- Play sound effects when clicking a button or when a result window pops up to increase the sense of control.

[📖Advanced Training] Implementing Obstacle Objects

Video Timeline

- You can follow along by using the timeline under the training video.
- Implementing Obstacle Objects: 01:46:25

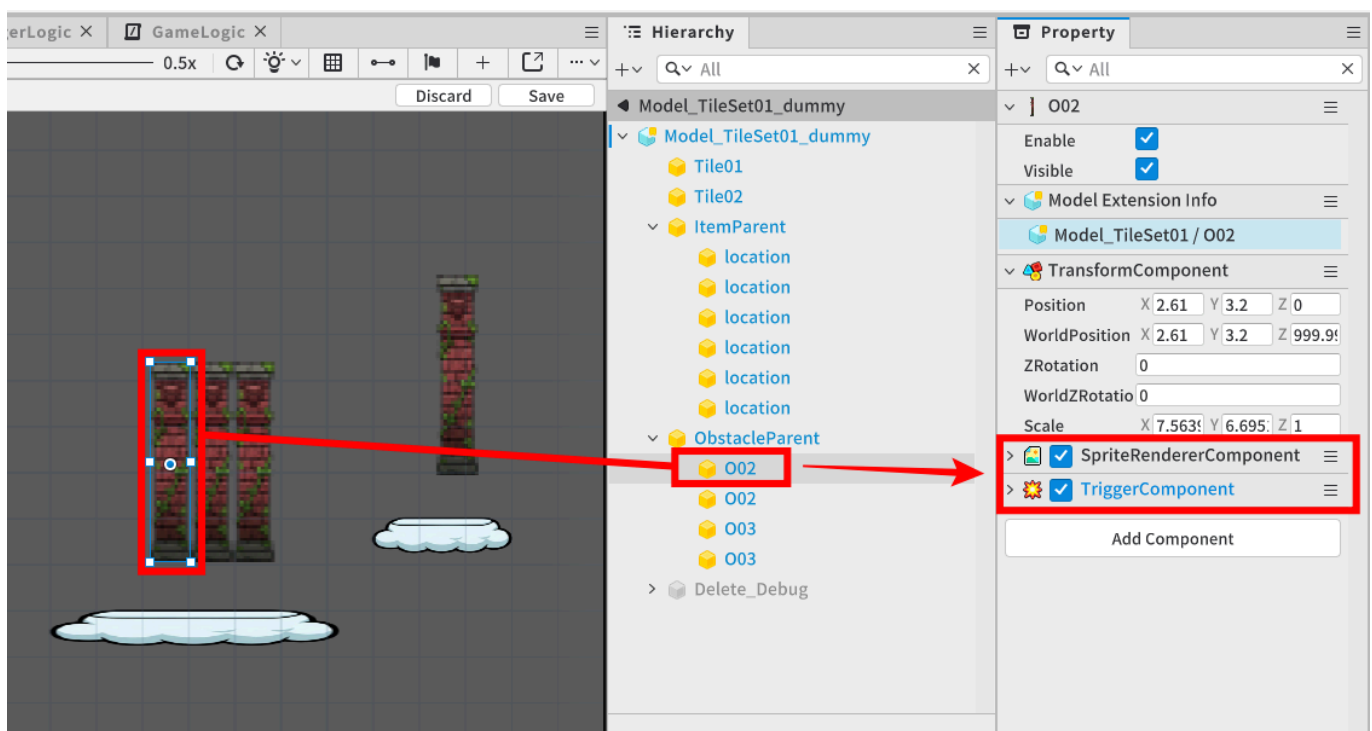
Learning Goals

- Understand how to pre-position obstacles within a tileset, activate them probabilistically as the game progresses, and manage resources efficiently.
- Implement features that reduce a character's HP when an obstacle collision is detected, or make the character invincible for a period of time.

Practical Exercises to Do After Watching the Video

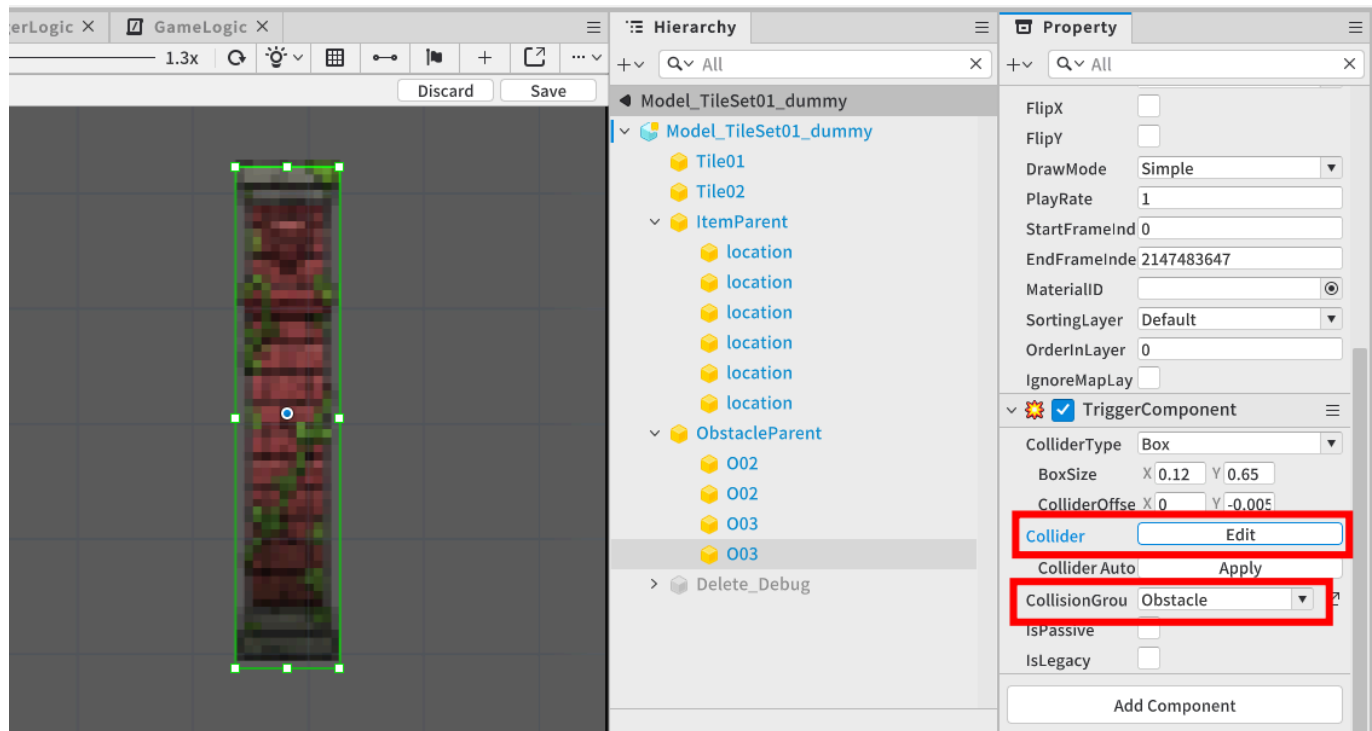
- Try placing the obstacle entity as a child entity of the tileset to see if the obstacle moves with the tiles when they are moved without separate coding.
- Test that the character's HP is reduced when hitting an obstacle and that they do not take additional damage for the set invincibility duration.

Creating Obstacles



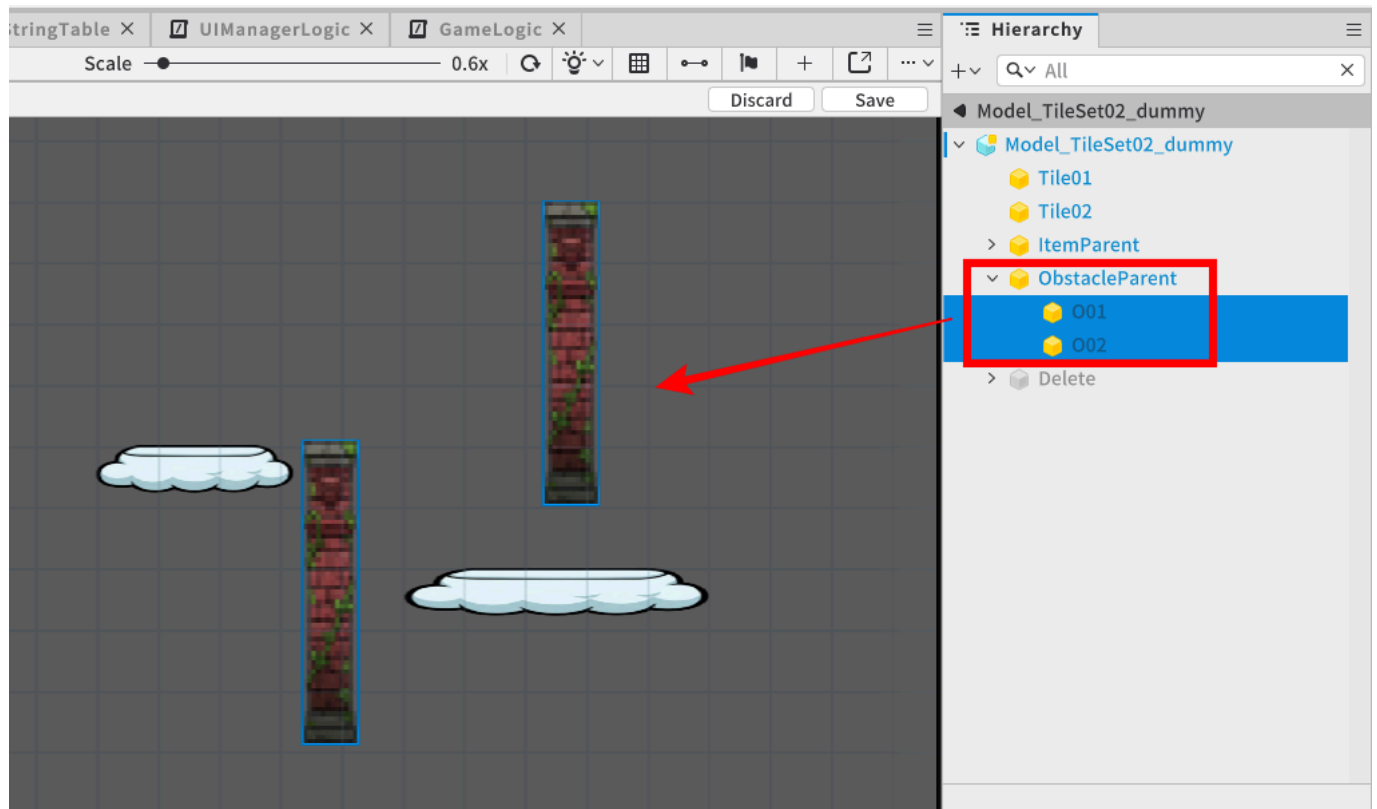
- The components required for the obstacle entity are SpriteRendererComponent and TriggerComponent.
- The SpriteComponent is intended to represent the appearance of the obstacle.
- The TriggerComponent is intended to detect collisions with the character but not to cause an actual physical collision, such as the character being blocked and pushed.

Setting up the TriggerComponent



- Set the Collider range of the TriggerComponent registered in the obstacle entity.
- In CollisionGroup, add a new collision group 'Obstacle' and set the matrix so only collisions with 'Player' are detected.
- Adjust the difficulty of the game by widening or narrowing the range of the Collider.

Obstacle Placing Method



- It's difficult to control the position of obstacles in code, so it proceeds with production by pre-positioning the obstacle entities.
- Play around with placement and modifications, testing to see if the player can avoid obstacles.
- Prepare the parent entities of obstacles so that the way the obstacle entities are managed is seamless.

Randomly Placing Obstacles

- If the prepared obstacles are placed on every tile, the game could become monotonous, so utilize the Random feature to make the obstacles appear in an irregular pattern.
- The ability to randomly place pre-placed obstacles occurs when the tileset is activated from a TileSet Component script registered to the floor tileset.
- The random obstacle placement feature can be handled with the following code.

SpawnObstacle method in the TileSet script

-- The parent entity of the obstacle entities.
Entity obstacleParent

```
void SpawnObstacle()
{
    -- Check the number of pre-installed obstacle objects in the tileset.
    -- Determine the number of children (obstacle entities) of the parent
    entity that hold the obstacles.
    local spawnLocationCount = self.obstacleParent.Children.Count

    -- Iterate through the child objects of the obstacleParent object
    for index , child in pairs(self.obstacleParent.Children) do
```



```

-- and generate a random number from 0 to 1 (implementation intent:
50% chance of placing obstacle, 0 if 0 and X if 1).
local randomNumber = math.random(0,1)

-- /branch 1/ If the number is zero
if randomNumber == 0 then
    -- Enable the obstacle object.
    child.Enable = true
    -- Overwrite with a resource image that matches the stage
number.
    child.SpriteRendererComponent.SpriteRUID =
_DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].ObstacleRUID

-- /branch 2/ If the number is not zero,
else
    -- Disable the obstacle object.
    child.Enable = false
end
end
}

```

Obstacle Hit Detection



- If the character's TriggerComponent and the obstacle's TriggerComponent are properly registered, they can utilize each other's Trigger events.
- Utilize the HandleTriggerEnterEvent event method within the CustomPlayer script created to directly control the character's functionality and action.
- If the contents of the CustomPlayer code is needed, the CustomPlayer script is attached in the '6. Implementing Player Characters' section.
- Collision handling can be handled with the following code.

```
-- Collision Handling with event handlers in CustomPlayer

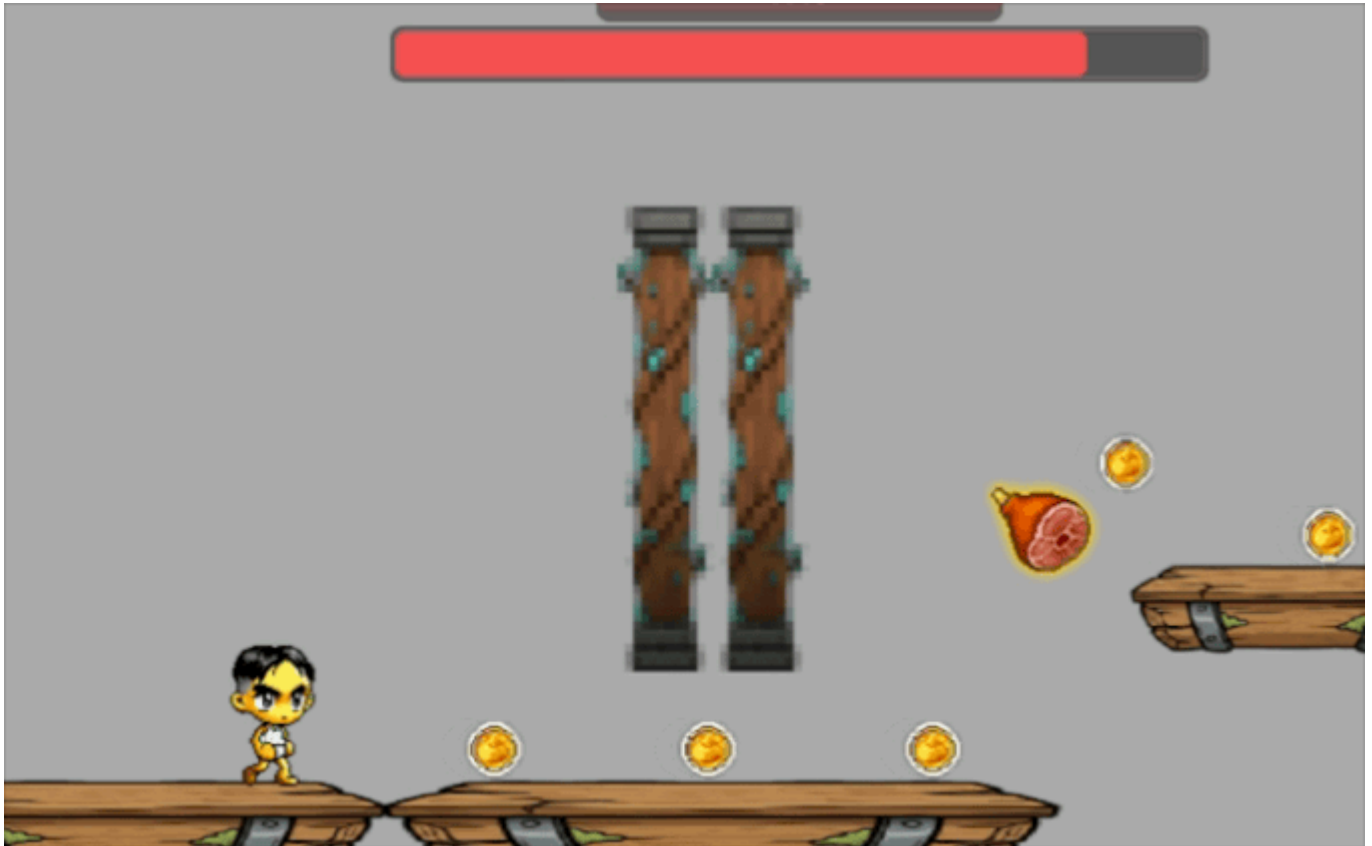
HandleTriggerEnterEvent(TriggerEnterEvent event)
{
    -- Save the entity information of the collided target.
    local TriggerBodyEntity = event.TriggerBodyEntity

    -- Extracts the collision group ID number set in the target entity's
    CollisionGroup.
    local TriggerGroup =
    tostring(TriggerBodyEntity.TriggerComponent.CollisionGroup.Id)

    -- If the group name of the collision object is the same as "Obstacle"
    if _Config.triggerGroup[TriggerGroup] == "Obstacle" then
        -- Perform collision detection processing.
        self:ContactObstacle()
    end
}

void ContactObstacle()
{
    -- Tells GameLogic to reduce HP by the obstacle damage value set in
    Config.
    _GameLogic:AddHP(_Config.obstacleDamage)
}
```

Add Character Invincibility



- If an obstacle with a `HandleTriggerStayEvent` event method that doesn't have a fixed invincibility time is created, the character may end up taking damage every frame, which can quickly bring them to zero health.
- By blocking additional damage for a short period of time immediately after hitting an obstacle, it gives the player more time to recognize the situation and readjust their stance.
- The ability to ignore hits for a period of time can be handled with the following code.

```
-- Utilizing invincibleTimer of CustomPlayer

-- Declare a temporary invincibility timer variable.
number invincibleTimer
-- Declare a variable that indicates the hit state.
boolean isHit

void ContactObstacle()
{
    -- If the value of the temporary invincibility timer variable is
    greater than 0 (the temporary invincibility timer is running)
    if self.invincibleTimer > 0 then
        -- Terminates the method without performing any hit detection due to
        obstacles.
        return
    end

    -- If the above conditions have not been executed (the temporary
    invincibility timer is not running)

    -- Decreases HP.
    _GameLogic:AddHP(_Config.obstacleDamage)
```

```

    -- Toggles the hit state to true.
    self.isHit = true;

    -- Reduce the alpha value of the character's appearance by half to make
    it see-through, so the player knows they are in invincible state.
    self.Entity.AvatarRendererComponent:SetAlpha(0.5)

    -- Updates the invincibility time value to set to allow the temporary
    invincibility timer to perform an action.
    self.invincibleTimer = _Config.invincibleTime
}

void OnUpdate(number delta)
{
    -- Utilizes the isHit value to prevent the code below from being
    executed when not in an isHit situation.
    if self.isHit == true then

        -- If the value of the temporary invincibility timer is greater
        than zero,
        if self.invincibleTimer > 0 then
            -- Decreases the value of the timer by the delta value.
            self.invincibleTimer -= delta

            -- If the temporary invincibility timer has no duration left,
            else
                -- Increases the alpha value of the character's appearance to its
                original value (1) to make it opaque, so the player knows that
                invincibility has ended.
                self.Entity.AvatarRendererComponent:SetAlpha(1);
            end
        end
    end
}

```

Minimum Implementation Standards

- The obstacle entity should be moved the same as the floor tiles.
- Collisions between player characters and obstacles should result in a hit detection, and the obstacle should not push the character backward.
- Temporary invincibility must be activated when hit by an obstacle and must not take damage while the timer is running.

Usage and Extension Direction

- Can add static-looking obstacles as well as active obstacles with dynamic animations that rise up and down in place.
- As the game progresses (over time), the difficulty of the game can be adjusted by making obstacles more or less likely to spawn.

[📖 Advanced Training] Implementing Items

Video Timeline

- You can follow along by using the timeline under the training video.
- Implementing Items: 01:55:02

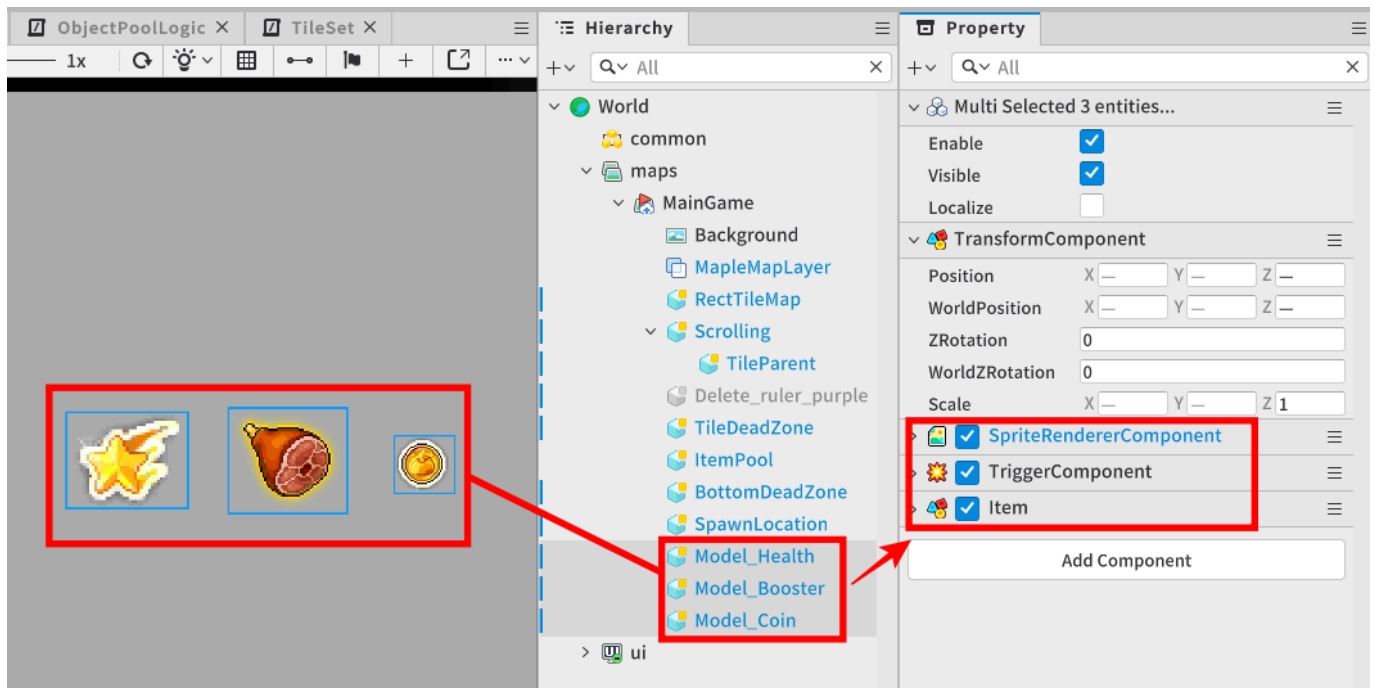
Learning Goals

- Understand the process of defining an item's unique information (effect value, spawn probability, etc.) in a Dataset and transforming it into a table to utilize it.
- By applying an object pooling system, implement a method to probabilistically spawn items driven by random number generation and associate them with specific map tiles.
- Detect collisions between characters and items and learn flows that use ItemLogic to trigger specific effects (score increases, HP recovery, boosters, etc.) based on the type of item picked up.

Practical Exercises to Do After Watching the Video

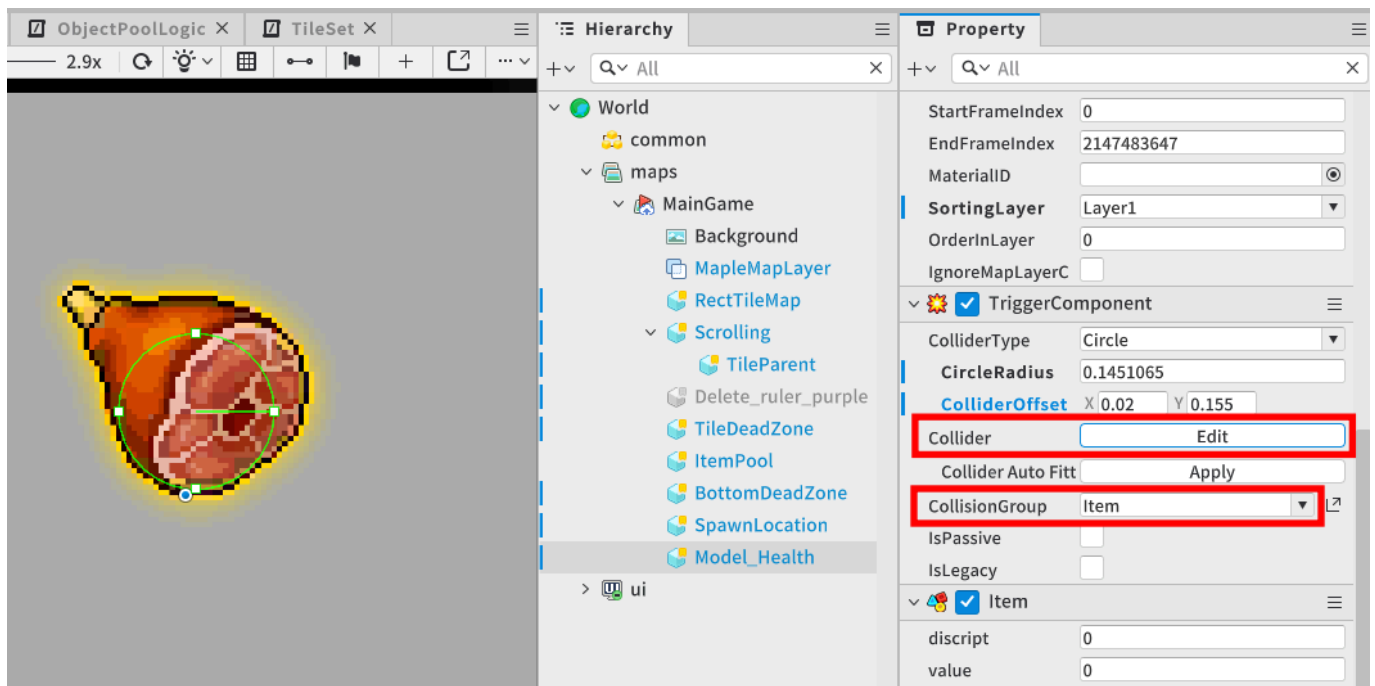
- Add data for a new feature item (such as an invincibility item) to the ItemInfo Dataset and verify that it is properly registered to the table via logic when executing the world.
- Try changing the item's (SpawnRate) value to an extremely high or low value and test whether the item spawns in-game at the intended frequency.
- Play to verify whether the stats of acquired items are correctly reflected in the UI and system.

Creating Items



- In the sample world, the components registered as item are SpriteRendererComponent, TriggerComponent, and Item.
- The SpriteRendererComponent is a component for registering the appearance of an item.
- The TriggerComponent is intended to detect collisions with the character, not to cause actual physical collisions such as the character being pushed.
- The Item component is a custom-made script designed to classify the specific type of item whenever a collision with a character is detected.

Setting up the TriggerComponent



- To change the shape of an item's collision box, utilize the drop-down menu for ColliderType.
- To enable interaction with the character, use the Collider's Edit function within the TriggerComponent registered to the item entity to configure the collision boundaries.
- In CollisionGroup, add a new collision group Item and set it to only collide with the Player.

Creating Item script

- The Item script simply registers the item name and the item's unique effect value variables.
- Write the following code.
- This code is written based on Plain Text.

```
@Component
script Item extends Component

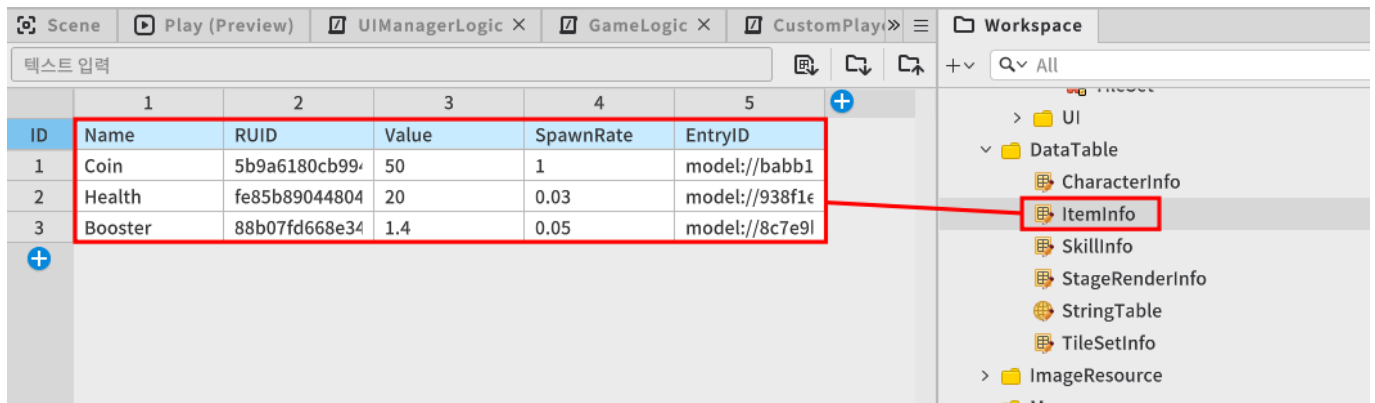
property number discript = 0 -- The component that contains the basic
functionality and the variables of the items.

property number value = 0 -- The unique numeric value of the item.

property string name = "" -- A variable to store the name of the item.

end
```

Item Dataset



ID	Name	RUID	Value	SpawnRate	EntryID
1	Coin	5b9a6180cb99	50	1	model://babb1
2	Health	fe85b89044804	20	0.03	model://938f1e
3	Booster	88b07fd668e34	1.4	0.05	model://8c7e9l

Target	Description
Name	Define the name of the item.
RUID	Record the item's appearance and resource ID value.
Value	Define a unique effect value for each item.
SpawnRate	Defines the spawn probability of the item and registers a value between 0 and 1 when recorded.
EntryID	Stores the EntryID value of the source (Model) needed when creating a new item entity through the SpawnService.

- Learn how to systematically organize by entering item names, unique effect values, spawn rate, and appearance data into a DataSet.

Registering Item Dataset Table

- Item-specific information created with a DataSet is registered in the itemInfoTable of DataSetLogic.
- The ability to register as a data table is done in the InitItemInfoTable method of the DataSetLogic script.
- If the contents of the DataSetLogic code are needed, the DataSetLogic script is attached in the '2. Understanding Logic' section.

```
-- InitItemInfoTable method of DataSetLogic

table itemInfoTable

void InitItemInfoTable()
{
    -- The number of rows in the ItemInfo table.
    local rowCount = _DataService:GetRowCount("ItemInfo")

    -- A variable to hold the data for each line.
    local rowData

    -- Start iteration, ranging from 1 up to the number of rows in the
    dataset.
    for i = 1 , rowCount , 1 do
        rowData =
        {
            -- Save the item name.
            Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
            -- Stores the item RUID.
            RUID = tostring(_DataService:GetCell(dataSetName,i,"RUID")),
            -- Stores the item effect's unique value.
            Value = tonumber(_DataService:GetCell(dataSetName,i,"Value")),
            -- Stores the item creation rate.
            SpawnRate =
            tonumber(_DataService:GetCell(dataSetName,i,"SpawnRate")),
            -- Stores the EntryID value of the item's model (prefab).
            EntryID =
            tostring(_DataService:GetCell(dataSetName,i,"EntryID"))
        }

        -- Saves the data stored as rowData to a table.
        self.itemInfoTable[i] = rowData
    end
}
```

Utilizing Item Data

- After saving data from an ItemInfo DataSet to DataSetLogic, data can be retrieved from DataSetLogic.
- When creating an item in the sample world, a data table for the item is needed.
- Pre-create item entities at the start of world from the Object Pool.
- The following code is the CreateItem method of ObjectPoolLogic that creates an item at the start of the world.

CreateItem of ObjectPoolLogic

```

table itemDataSetIDTable = {"Coin" = 1, "Health" = 2, "Booster" = 3}

void CreateItem(string itemName, number createCount)
{
    local tempTable

    local dataSetID = self.itemDataSetIDTable[itemName]

    local entryID = _DataSetLogic.itemInfoTable[dataSetID].EntryID
    -- Retrieves the Name value of the target item model in the dataset.
    local name = _DataSetLogic.itemInfoTable[dataSetID].Name
    -- Retrieves the Value of the target item model in the dataset.
    local value = _DataSetLogic.itemInfoTable[dataSetID].Value

    -- Performs validation on the target table.

    -- /branch 1/ If no target pool (table) exists within the target item
    pool
    if isValid(self.itemPool[name]) == false or self.itemPool[name] == nil
    then
        -- Create a new temporary table.
        tempTable = {}
        -- Set the inner pool (table) in the item pool as a temporary table.
        self.itemPool[name] = tempTable

    -- /branch 2/ If the target pool already exists within the target item
    pool
    else
        -- Declare the temporary table (pool) as the table already being
        utilized.
        tempTable = self.itemPool[name]
    end

    -- Performs a loop as many times as parameter2 createCount.
    for i = 1, createCount, 1 do
        -- Utilizes SpawnService to create the object.
        local entity = _SpawnService:SpawnByModelId(entryID, name, Vector3.up *
100, self.ItemPoolParent)

        -- Proceed with configuring the created object.

        -- Sets the item's unique value of the created object.
        entity.Item.value = value
    end
}

```

```

-- Sets the name variable of the item of the created object.
entity.Item.name = name

-- Inserts the entity created for the target table (pool).
table.insert(tempTable,entity)

-- Disable the created entity object.
entity.Enable = false
end
}

```

Creating Items Based on Item Spawn Rate

ID	Name	SpawnRate
1	Coin	1
2	Health	0.03
3	Booster	0.05

Random = 0.98 ⇒ **"Coin"**

Random = 0.04 ⇒ **"Coin"**

Random = 0.02 ⇒ **"Health"**

- There are three types of items implemented in the sample world: coins that award points, healing items that restore HP to the character, and booster items that temporarily increase scrolling speed.
- If all items spawn with the same probability, this can lead to balance issues where recovery items appear too often and the game never ends.
- As such, how often an item appears can be adjusted by setting the creation probability.
- The probability logic is designed to extract a random number between 0 and 1 when an item spawns, compare it to the SpawnRate value in the ItemInfo dataset, and spawn that item if the random number is lower or equal.

SpawnItem method of TileSet

```

-- Proceed with a randomized spawn. (Generates a random number).
local randomRate = math.random(0,100) * 0.01

-- Declare the local variable to be set to the entity type.
---@type Entity
local tempEntity

-- /branch 1/ If the random probability is less than or equal to the
value of healthSpawnRate
if _DataSetLogic.itemInfoTable[2].SpawnRate >= randomRate then
-- Get the Health item from the object pool.
tempEntity = _ObjectPoolLogic.GetItem("Health")

-- /branch 2/ If the random probability value is less than or equal to
boosterSpawnRate
elseif _DataSetLogic.itemInfoTable[3].SpawnRate >= randomRate then

```

```

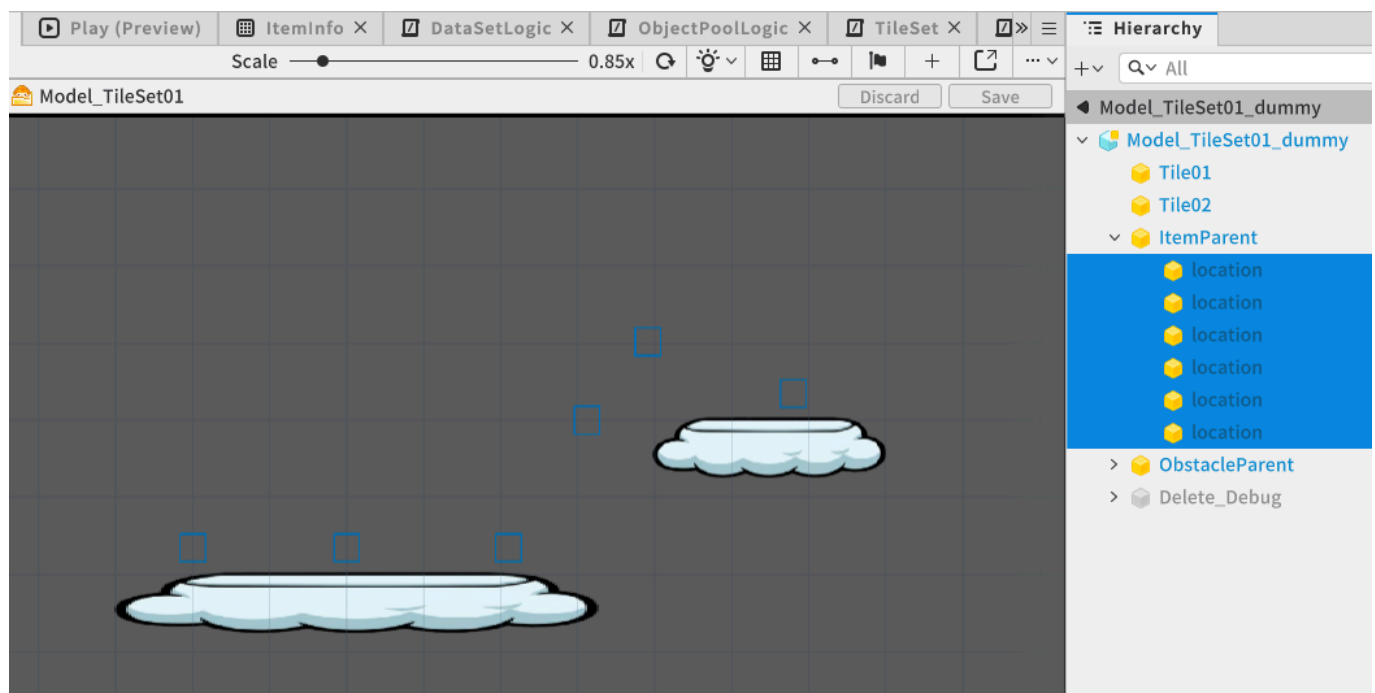
-- Get the Booster item from the object pool.
tempEntity = _ObjectPoolLogic:GetItem("Booster")

-- /branch 3/ Spawns a default item (coin) if it's not within the
probability of spawning any of the other set items.
else
    tempEntity = _ObjectPoolLogic:GetItem("Coin")

end

```

Placing Items in Specified Locations



- Within the floor tileset, entities are registered with ItemParent to set the location of items.
- Items spawned from the object pool should be moved the same as floor tiles.
- The spawned item is (Attach)ed as a (Child) of the entity that serves as the reference point, then has its local coordinates reset to the origin (0, 0, 0) so that it is placed in the exact same location as its parent.
- Repeat item spawning by iterating through the child elements of ItemParent.

SpawnItem method of TileSet

Entity itemParent -- The parent entity of the entities for setting the item location.

```

void SpawnItem
{
    -- Iterate the children of the entity to set the item's position.
    for index, child in pairs(self.itemParent.Children) do

```

```
local tempEntity

-- Process of setting the item entity of tempEntity by comparing the
random probability to the SpawnRate value

-- Sets the parent of the item entity to child.
tempEntity.AttachTo(child)

-- Sets the local coordinates of the item entity to the same position as
its parent, using the parent's location as the origin.
tempEntity.TransformComponent.Position = Vector3(0,0,0)
}
```

Minimum Implementation Standards

- When an item spawns, it should attach to the floor tileset and move normally to the left side of the screen with the scroll movement of the map.
- When a character collides with an item's Trigger, the item entity should disappear from the screen and be returned to the object pool.

Usage and Extension Direction

- Toward the end of the game, the difficulty of the game can be adjusted by randomly spawning items that have positive effects as well as trap items that will blind the character or lower the score if picked up.
- Design items for new gimmicks, such as a shield item that ignores obstacle collisions for a period of time or a magnet item that grabs nearby coins at once, and extend them with dedicated scripts.

[📖 Advanced Training] Implementing Save Data System

Video Timeline

- You can follow along by using the timeline under the training video.
- Implementing Save Data System: 02:13:50

Learning Goals

- Understand how to utilize the `DataService` to store a player's best score on the server and retrieve them upon reconnection.

- Identify the reasons for setting the Sync attribute of variables to synchronize data between the client and the server.
- Utilize the OnSyncProperty event to pinpoint when data has been successfully retrieved from the server and understand how to use it safely.

Practical Exercises to Do After Watching the Video

- Record the new best score value manually, then verify that the old score information is loaded correctly when the world restarts.

Saving High Score Value on Server Side

- Instead of communicating with the server every time the score goes up, reduce server load by storing data only when it's absolutely necessary, such as when the game ends.
- The BestScore value managed from the server is only refreshed if the current score is higher than the existing best score at the time of game over (0 HP).

How to update BestScore value managed from the server
EndGame method called at the **end** of the game from GameLogic

```
[Sync] number bestScore;
[None] number currentScore;

[client] -- client-only method
void EndMainGame()
{
    -- Function called on game over.

    -- If the current score is higher than the previous best score
    if self.currentScore > self.bestScore then
        -- Update the bestScore managed on the client (local environment).
        self.bestScore = self.currentScore
        -- Request score updates server-side as well.
        self:UpdateScore(self.bestScore)
    end
}

[server] -- server-only methods
void UpdateScore(number score)
{
    -- Update the bestScore value managed by a non-local server.
    self.bestScore = score
}
```

Store Player High Scores with DataStorage

- Even if data values are managed on the server side, if they are not explicitly saved and the player reconnects to the world, the player's previous data values cannot be retrieved.
- To save player data or game data, utilize the DataStorage service.
- It is recommended to utilize the functionality of DataStorage within the HandleUserLeaveEvent and HandleUserEnterEvent.

Storing player and game data with DataStorage
SaveScore Method of GameLogic

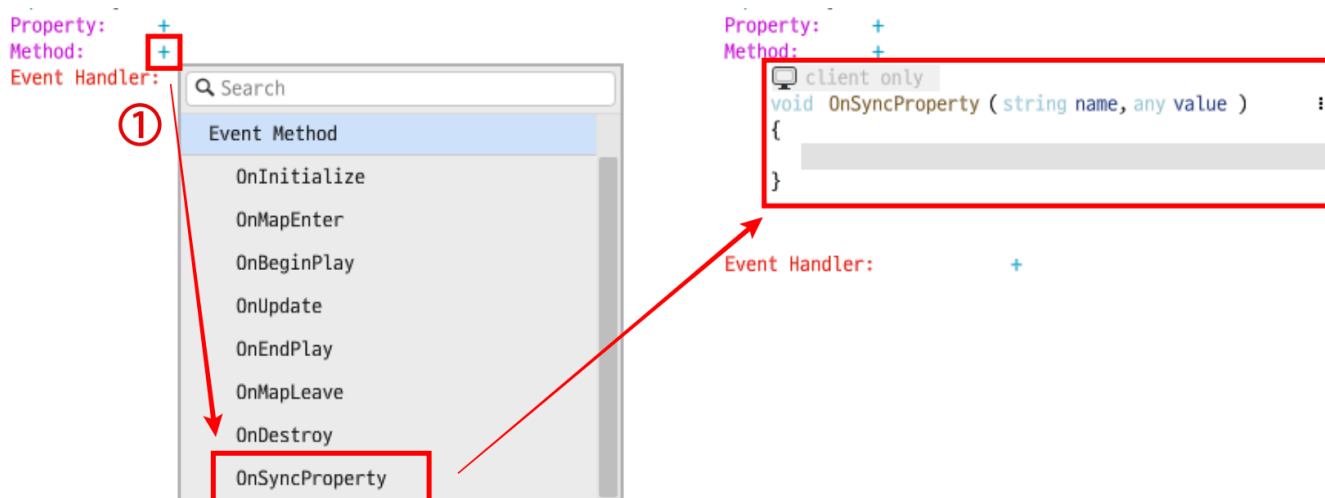
```
HandleUserLeaveEvent(UserLeaveEvent event)
{
    self:SaveScore(self.bestScore)
}

[server] -- server-only methods
void SaveScore(number score)
{
    local data = _DataStorageService:GetGlobalDataStorage("MyData")
    local errorCode = data:SetAndWait("BestScore", tostring(score))
}
```

Retrieving High Score Value

- As with data storage, only request data once, upon first entering the world, to avoid unnecessary server communication.
- If there was a previously-recorded best score when the player entered the world, it will be retrieved.

Retrieving Previous Score Values Using DataStorage and OnSyncProperty



1. In the Method tab of the script, right-click the '+' button and create a method via OnSyncProperty.
- To retrieve player data or game data, utilize the DataStorage service.

- Variables set to Sync can use the OnSyncProperty method to detect exactly when a value has been synchronized from the server.

Loading player and game data with DataStorage
LoadData Method of GameLogic

[Sync] number bestScore

```
HandleUserEnterEvent(UserEnterEvent event)
{
    self:LoadData()
}
```

[server] -- server-only methods

```
void LoadData()
{
    -- Utilize the data storage service to get data storage named 'MyData'.
    local data = _DataStorageService:GetGlobalDataStorage("MyData")

    -- Get the Value corresponding to the 'BestScore' key within that data
    storage.
    local errorCode,value = data:GetAndWait("BestScore")

    -- Register a temporary score value variable.
    local tempScore

    -- If no data is stored
    if value == nil then
        -- Set the temporary score value to 0.
        tempScore = 0

    -- If there is any stored data
    else
        -- Set the temporary score value to the existing score value.
        tempScore = tonumber(value)
    end

    -- Update the bestScore value.
    -- When a property declared with Sync changes its value,
    -- The value is synchronized on the client side as well, and the
    OnSyncProperty method is called automatically.
    self.bestScore = tonumber(tempScore)
}
```

[client only]

```
void OnSyncProperty(string name, any value)
{
    -- When a change in a variable (property) is detected

    -- If the changed variable is named "bestScore"
    if name == "bestScore" then
        -- Ask UIManagerLogic to update the best score text on the main title
    end
}
```

```
screen.  
  _UIManagerLogic:UpdateTitleBestScore(tonumber(value))  
end  
}
```

Minimum Implementation Standards

- When a player achieves a new best score in the game, quits the world completely, and then logs back in, the title screen should display the previous best score correctly instead of resetting to zero.
- The timing control via OnSyncProperty must be fully functional, so the UI text doesn't update first when the score data hasn't finished syncing from the server and display empty values or errors.

Usage and Extension Direction

- Extend this to a system that bundles and stores not just bestScore, but a variety of other data, such as cumulative currency, character lists, stage information, and more.